VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018

A PROJECT REPORT

ON

"AI SMART ATTENDANCE SYSTEM USING FACE RECOGNITION"

A Dissertation Submitted in partial fulfillment of the requirement for the degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by

MAHAMMAD YUNUS B	**(1RG23CS027)**
MIR NAQI RAZA	**(1RG23CS035)**
MUHAMMAD ADAM KHAN	**(1RG23CS038)**
MOHAMMED MUZAMMIL	**(1RG24CS409)**

Under The Guidance of

Mrs. Bhagyashri Wakde

Assistant Professor, Dept. of CSE

RGIT, Bengaluru-32

Department of Computer Science & Engineering

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

Cholanagar, R. T. Nagar Post, Bengaluru-560032

2026-2027

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

(Affiliated to Visvesvaraya Technological University)

Cholanagar, R.T. Nagar Post, Bengaluru-560032

Department of Computer Science & Engineering

CERTIFICATE

Phase- II

This is to certify that the Project Report entitled **"AI SMART ATTENDANCE SYSTEM USING FACE RECOGNITION"** is a Bonafide work carried out by **Mr. MAHAMMAD YUNUS B (1RG23CS027), Mr. MIR NAQI RAZA (1RG23CS035), Mr. MUHAMMAD ADAM KHAN (1RG23CS038), Mr. MOHAMMED MUZAMMIL (1RG24CS409)** in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** under **Visvesvaraya Technological University, Belagavi,** during the year **2026-2027.** It is certified that all corrections/suggestions given for Internal Assessment have been incorporated in the report. This report has been approved as it satisfies the academic requirements in respect of project work (BCS786) prescribed for the said degree.

Signature of guide	Signature of HOD	Signature of Principal
Mrs. Bhagyashri Wakde	**Dr. Arudra A**	**Dr. D G Anand**
Assistant Professor	Associate Professor, HOD	Principal, RGIT,
Dept. of CSE	Dept. of CSE	Bengaluru-32
RGIT, Bengaluru-32	RGIT, Bengaluru-32	

External Viva

Name of the Examiners	Signature with date
1.	
2.	

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

DECLARATION

We hereby declare that the project work entitled **"AI SMART ATTENDANCE SYSTEM USING FACE RECOGNITION"** submitted to the **Visvesvaraya Technological University, Belagavi** during the academic year **2026-2027,** is record of an original work done by us under the guidance of **Mrs. Bhagyashri Wakde,** Assistant Professor, Rajiv Gandhi Institute of Technology, Bengaluru and this project work is submitted in the partial fulfillment of requirements for the award of the degree of **Bachelor of Engineering in Computer Science & Engineering.** The results embodied in this report have not been submitted to any other University or Institute for award of any degree or diploma.

MAHAMMAD YUNUS B	**(1RG23CS027)**
MIR NAQI RAZA	**(1RG23CS035)**
MUHAMMAD ADAM KHAN	**(1RG23CS038)**
MOHAMMED MUZAMMIL	**(1RG24CS409)**

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude and respect to the **Rajiv Gandhi Institute of Technology, Bengaluru** for providing us an opportunity to carry out our project work. We express our sincere regards and thanks to **Dr. D G ANAND,** Principal, RGIT, Bengaluru. Also we would like to thanks **Dr. ARUDRA A**, Associate Professor and HOD, Department of Computer Science & Engineering, RGIT, Bengaluru, for their encouragement and support throughout the Project.

With profound sense of gratitude, we acknowledge the guidance and support extended towards us by project coordinators **Mrs. BHAGYASHRI WAKDE,** Assistant Professor, Department of CSE, RGIT, Bengaluru. **Mrs. SONIYA KOMAL V,** Assistant professor, Department of CSE, RGIT, Bengaluru. Their incessant encouragement and valuable technical support have been of immense help in realizing this project. Their guidance gave us the environment to enhance our knowledge, skills and to reach the pinnacle with sheer determination, dedication and hard work.

We extend our heartfelt gratitude to **Mrs. BHAGYASHRI WAKDE,** Assistant Professor, Department of Computer Science & Engineering, RGIT, Bengaluru, for their unwavering support and guidance in our project. Their expertise has been instrumental in shaping its success. We also appreciate the collaborative efforts of other coordinators and team members for enriching this learning experience.

We also extend our thanks to the entire faculty of the Department of CSE, RGIT, Bengaluru, who have encouraged throughout the course of Bachelor Degree.

We extend our heartfelt thanks to our family for their full support and unwavering encouragement, which provided us with a safe and conducive environment for the completion of this project. Their assistance and belief in our abilities have been instrumental in this achievement.

MAHAMMAD YUNUS B (1RG23CS027)
MIR NAQI RAZA (1RG23CS035)
MUHAMMAD ADAM KHAN (1RG23CS038)
MOHAMMED MUZAMMIL (1RG24CS409)

ABSTRACT

With the rapid advancement of artificial intelligence and computer vision technologies, automated systems are increasingly being adopted to improve efficiency and accuracy in various domains. Attendance management is a crucial task in educational institutions and organizations, yet traditional methods such as manual registers or card-based systems are time-consuming, error-prone, and susceptible to manipulation. This creates a need for an intelligent and reliable system capable of automating attendance tracking. This project proposes an AI Smart Attendance System using Face Recognition, which utilizes advanced machine learning techniques to identify individuals based on their facial features. The system captures real-time images through a camera, detects faces, and compares them with a pre-trained database to mark attendance automatically. Face recognition algorithms extract unique facial features and ensure accurate identification even under varying lighting conditions and facial expressions. Unlike conventional methods, the proposed system eliminates the need for physical contact or manual intervention, thereby increasing efficiency and reducing the chances of proxy attendance. The system is designed to preprocess captured images, perform feature extraction, and classify faces using trained models for reliable performance. A user-friendly interface is developed to display attendance records, manage student data, and generate reports. The system provides real-time attendance monitoring, maintains digital records, and allows easy access to attendance data. It can also be extended to notify parents or guardians about student attendance, enhancing transparency and communication. This solution is highly suitable for schools, colleges, and organizations aiming to improve accuracy, security, and automation in attendance management. Future enhancements may include integrating deep learning models for improved recognition accuracy and deploying the system in large-scale environments with cloud-based support.

CONTENTS

ACKNOWLEDGEMENT			**i**
ABSTRACT			**ii**
LIST OF FIGURES			**vii**
LIST OF TABLES			**ix**
CHAPTER NO		**TITLE**	**PAGE NO**
1		**INTRODUCTION**	**1**
1.1		**Overview**	**1**
	1.1.1	**Face recognition in attendance systems**	**2**
	1.1.2	**Why face recognition alone is not enough**	**2**
1.2		**Motivation**	**2**
1.3		**Problem Identification**	**3**
1.4		**Scope of the Project**	**3**
1.5		**Objective and Methodology**	**4**
1.6		**Existing System**	**5**
1.7		**Proposed System**	**5**
1.8		**Outcome of the Project**	**7**
1.9		**Report Organization**	**8**
1.10		**Introduction Summary**	**8**
2		**LITERATURE SURVEY**	**9**
2.1		**Related Work**	**9**
	2.1.1	**Reference papers**	**9**
	2.1.2	**Attendance and campus-automation studies**	**9**
	2.1.3	**Biometric privacy**	**9**
2.2		**Eigenfaces for Recognition**	**10**
2.3		**Rapid Object Detection using a Boosted Cascade of Simple Features**	**10**
2.4		**HOG and LBP**	**10**
2.5		**FaceNet: A Unified Embedding for Face Recognition and Clustering**	**11**
2.6		**Deep Residual Learning for Image Recognition and Dlib-ml**	**11**
2.7		**Landmarks and EAR**	**12**
2.8		**ArcFace: Additive Angular Margin Loss for Deep Face Recognition**	**12**
2.9		**Literature Survey Summary**	**12**

3	**SYSTEM ANALYSIS**		**14**
3.1	**Introduction to System Analysis**		**14**
3.2	**Feasibility Study**		**14**
	3.2.1	**Technical Feasibility**	**14**
	3.2.2	**Economic Feasibility**	**14**
	3.2.3	**Operational Feasibility**	**14**
	3.2.4	**Schedule Feasibility**	**15**
	3.2.5	**Social Feasibility**	**15**
3.3	**Stakeholders and Usage Scenarios**		**15**
3.4	**System Analysis Summary**		**15**
4	**REQUIREMENT ANALYSIS**		**16**
4.1	**Functional Requirements**		**16**
4.2	**Non-Functional Requirements**		**17**
4.3	**Software Requirements**		**17**
	4.3.1	**Operating System**	**18**
4.4	**Software Requirement Summary**		**18**
4.5	**Hardware Requirements**		**18**
4.6	**Hardware Requirements Summary**		**19**
4.7	**Requirement Analysis Summary**		**19**
5	**SYSTEM DESIGN**		**20**
5.1	**System Architecture Overview**		**20**
	5.1.1	**Architecture**	**20**
	5.1.2	**System Flow Design**	**20**
	5.1.3	**Design Characteristics**	**21**
5.2	**Gantt Chart**		**21**
	5.2.1	**Gantt Chart of Phase One**	**21**
	5.2.2	**Gantt Chart of Phase Two**	**22**
5.3	**Life Cycle Model**		**22**
5.4	**Data Flow Diagram**		**23**
5.5	**Use Case Diagram**		**26**
5.6	**Sequence Diagram**		**26**
5.7	**Class Diagram**		**27**
5.8	**Database Design (ER Diagram)**		**28**

5.9 Security Design 29
5.10 System Design Summary 29
6 SYSTEM IMPLEMENTATION 30
6.1 Modular Approach 30
6.1.1 Vision Engine (vision.py) 30
6.1.2 Geofence Engine (geofence.py) 30
6.1.3 Timetable and Session Engine (timetable.py, main.py) 30
6.1.4 Authentication and Access Control (auth.py) 31
6.1.5 Multi-Tenant Data Layer (database.py, models.py) 31
6.1.6 Front-end Portals 31
6.2 Programming Code 32
6.2.1 Configuration (backend/rgit_app/config.py) 32
6.2.2 Vision engine (backend/rgit_app/vision.py) 32
6.2.3 Geofence engine (backend/rgit_app/geofence.py) 33
6.2.4 Timetable periods (backend/rgit_app/timetable.py) 34
6.2.5 Authentication and role guards (backend/rgit_app/auth.py) 34
6.2.6 Database models (backend/rgit_app/models.py) 35
6.2.7 Launching a teacher session (backend/rgit_app/main.py) 35
6.2.8 Marking attendance (backend/rgit_app/main.py) 36
6.2.9 Front-end check-in action (frontend/src/components/StudentPortal.jsx) 37
6.2.10 Multi-tenant database layer (backend/rgit_app/database.py) 37
6.2.11 Request schemas (backend/rgit_app/schemas.py) 38
6.2.12 Front-end API layer (frontend/src/services/api.js) 38
6.2.13 Face enrolment endpoint (backend/rgit_app/main.py) 38
6.2.14 Defaulters calculation (backend/rgit_app/main.py) 39
6.3 System Implementation Summary 40

7	**TESTING**		**41**
7.1	**Validation and System Testing**		**41**
	7.1.1	**Software Testing**	**41**
	7.1.2	**Validation**	**41**
	7.1.3	**Reasons for performing software validation**	**41**
7.2	**Test Cases**		**41**
7.3	**Operating Parameters and Evaluation Metrics**		**42**
7.4	**Testing Summary**		**43**
8	**SAMPLE OUTPUT**		**44**
8.1	**Snapshots**		**44**
	8.1.1	**Login portals**	**44**
	8.1.2	**Student self-registration**	**46**
	8.1.3	**Student portal**	**47**
	8.1.4	**Teacher portal**	**51**
	8.1.5	**HOD portal**	**55**
	8.1.6	**Master administrator portal**	**59**
	8.1.7	**Utilities**	**61**
8.2	**Sample Output Summary**		**62**
CONCLUSION			**65**
REFERENCES			**66**

LIST OF FIGURES/ILLUSTRATIONS

FIGURE NO.	FIGURE NAME	PAGE NO.
1.1	System Architecture	6
1.2	Workflow Diagram	7
5.1	Gantt Chart for Phase One	21
5.2	Gantt Chart for Phase Two	22
5.3	Life Cycle Model (Spiral Model)	23
5.4	DFD Level 0 (Context Diagram)	23
5.5	DFD Level 1	24
5.6	DFD Level 2 for Attendance Verification (Process 4.0)	25
5.7	Use Case Diagram	26
5.8	Sequence Diagram for Live Attendance Verification	27
5.9	Class Diagram	28
5.10	Entity Relationship Diagram	29
8.1	Student Portal Login Screen	44
8.2	Teacher Login Portal	45
8.3	HOD Login Portal	45
8.4	Master Admin (MOHD) Login Portal	46
8.5	Student Biometric Admission Enrollment Form	46
8.6	Registration Status Tracker	47
8.7	Student Main Dashboard	47
8.8	Student Dashboard with Check-in Options	48
8.9	Student QR Scanner Modal	48
8.10	Biometric Attendance Scanner with Session PIN	49
8.11	Session PIN Entry with Biometric Capture	49
8.12	PIN Auto-detected from the QR Link	50
8.13	Scanner with Phone-Camera Capture	50
8.14	Student Contact Edit Modal	51
8.15	Teacher Dashboard	51
8.16	Teacher Assigned Subjects	52
8.17	Launching an Attendance Session with Dynamic QR Code	52
8.18	Classroom Live QR Code and Session PIN	53
8.19	Teacher Session with Live QR Code and PIN	53
8.20	Projector Full-screen Session View	54
8.21	Excel Attendance Sheet	54

8.22	**CSV Attendance Sheet Preview**	**55**
8.23	**Day-to-Day Attendance Register**	**55**
8.24	**Faculty Export Panel**	**55**
8.25	**HOD Department Overview Dashboard**	**56**
8.26	**HOD Student Directory**	**56**
8.27	**HOD Portal in Mobile View**	**57**
8.28	**HOD Edit Credentials and Profile Modal**	**57**
8.29	**Registration Rejection Reasons**	**58**
8.30	**Assign Faculty Subjects Modal**	**58**
8.31	**Institutional Export Controls**	**58**
8.32	**Master Admin Dashboard**	**59**
8.33	**HOD Governance Directory**	**59**
8.34	**Campus Geofence Perimeter Studio**	**60**
8.35	**Security Audit Ledger**	**60**
8.36	**Google Calendar Holiday Synchronisation**	**61**
8.37	**Command Palette (Ctrl+K)**	**61**
8.38	**Theme and UI Studio**	**62**
8.39	**Campus Notice Board**	**62**

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
1.1	Existing Attendance Methods and Their Limitations	5
2.1	Eigenfaces for Recognition	10
2.2	Rapid Object Detection using a Boosted Cascade of Simple Features	10
2.3	HOG and LBP	10
2.4	FaceNet: A Unified Embedding for Face Recognition and Clustering	11
2.5	Deep Residual Learning for Image Recognition and Dlib-ml	11
2.6	Landmarks and EAR	12
2.7	ArcFace: Additive Angular Margin Loss for Deep Face Recognition	12
3.1	Stakeholders and Their Needs	15
4.1	Non-Functional Requirements	17
4.2	Software Requirements	18
4.3	Hardware Requirements	18
5.1	Data Flow Elements	25
5.2	Threats and Countermeasures	29
6.1	RGIT Seven-Period Daily Timetable	31
7.1	Test Cases of the Automated Test Suite	42
7.2	Operating Parameters of the System	43
7.3	Evaluation Metrics for the Vision and Geofence Modules	43

INTRODUCTION

CHAPTER 1

INTRODUCTION

1.1 Overview

Recording attendance is one of the most routine yet most important administrative activities of an engineering college. Student attendance decides eligibility to appear for the semester-end examinations of the Visvesvaraya Technological University (VTU), which mandates a minimum of 75% attendance in every subject. In spite of rapid progress in learning management systems and digital campuses, classroom attendance in most institutions is still taken by calling out names, passing a paper register or tapping an RFID card. These methods consume lecture time, depend on the honesty of every student in the room and produce records that have to be typed into a portal again at the end of the month.

This project, **AI Smart Attendance System Using Face Recognition**, replaces these manual practices with an automated, contact-free and proxy-resistant attendance platform built for Rajiv Gandhi Institute of Technology (RGIT), Bengaluru. A student marks attendance from a smartphone or laptop by (i) entering the six-digit session PIN that the subject teacher has launched for the ongoing class, (ii) allowing the application to read the GPS location of the device and (iii) showing the face to the camera. The back-end then checks the registered device, the campus geofence, the validity of the session, duplicate marking and finally the identity of the person by comparing 128-dimensional face embeddings. Only when all checks pass is an attendance record written to the database of the student's department together with a photographic proof, the distance from the campus centre and the confidence of the face match.

The platform is organised around four roles. **Students** register themselves, enrol their face and mark attendance. **Teachers** launch attendance sessions, monitor the live roster, correct records and export reports. **Heads of Department (HOD)** approve registrations, manage faculty, subjects and timetables of their branch. The **Master Administrator (MOHD)** governs all nine B.E. departments, appoints HODs, calibrates the campus geofence and inspects the security audit ledger. Each department keeps its data in its own SQLite database, which gives the system a simple multi-tenant structure.

The software is a decoupled full-stack application. The server is written in Python with

FastAPI and uses OpenCV, dlib and the face_recognition library for the computer-vision pipeline. The client is a React 19 single-page application built with Vite, which is also packaged as an Android application using Capacitor so that the device camera, GPS and device identity can be used natively.

1.1.1 Face recognition in attendance systems

Face recognition is a natural fit for attendance because the camera is already present in every phone and laptop, the capture is touch-free and the identity of the person is the thing being verified. Modern systems do not compare raw images. A deep convolutional network converts a face into a compact numeric vector (an embedding) in which images of the same person lie close together and images of different persons lie far apart. Recognition then reduces to measuring the Euclidean distance between two vectors and comparing it with a threshold. This project uses the 128-dimensional embedding produced by the dlib ResNet model through the face_recognition library.

1.1.2 Why face recognition alone is not enough

A practical attendance system must answer more questions than "whose face is this?". It must also decide whether the student is physically on the campus, whether a class is actually in session, whether the student belongs to that class, whether the record already exists and whether the phone being used is the one registered by the student. The proposed system therefore combines face verification with GPS geofencing, a teacher-controlled session PIN, hardware-device binding and a security audit trail.

1.2 Motivation

The motivation for this work comes from daily experience in the classroom and from the limitations of the systems that are in use today:

- **Loss of teaching time.** In a class of 60 students, calling names and collecting signatures can take several minutes of every period. Over a semester this adds up to many lost teaching hours.
- **Proxy attendance.** A student can answer for an absent friend, sign on his behalf or hand over an RFID card. Such records are not reliable for the 75% eligibility rule.
- **Delayed information.** Paper records are consolidated weeks later, so students and HODs learn about shortage of attendance only when it is too late to correct it.
- **Hygiene and queues.** Fingerprint scanners create queues at the door and need physical contact.

- **Departmental autonomy.** An engineering college has many branches with different subjects, faculty and timetables. A single shared register does not suit this structure, whereas isolated departmental data with a central administrator does.

1.3 Problem Identification

Existing attendance practices and many simple "face attendance" applications suffer from the following problems:

- **Impersonation.** Photographs, shared cards or a friend answering the roll call defeat the system.
- **Location spoofing.** Mobile check-in applications that do not verify location allow attendance from outside the college.
- **No class context.** Many systems record only that a person was seen, not for which subject, period or teacher.
- **Shared devices.** One phone can be used to mark attendance for several students.
- **Weak governance.** There is no clear separation between the student, subject teacher, HOD and administrator, and no audit trail of corrections and failed attempts.
- **Cost.** Commercial face terminals installed in every classroom are expensive and are not affordable for most institutions.

The problem addressed in this project is therefore to design and implement a low-cost, role-based attendance platform that identifies the student by face, confirms presence on the campus, ties every record to a teacher-launched class session and keeps an auditable history, using only ordinary phones and laptops.

1.4 Scope of the Project

The scope of the project covers the following:

- A web and Android application with four role-specific portals: Student, Teacher, HOD and Master Admin.
- Student self-registration with a 10-character VTU USN builder, personal details and a camera-captured face photograph, followed by an approval workflow for the HOD.
- Face verification using 128-d embeddings, with a Euclidean distance tolerance of 0.55 and a stored proof snapshot for each attendance.
- GPS geofence verification using the Haversine formula with a default radius of 250 m around the RGIT campus, a GPS-accuracy guard and a mock-location guard.
- A seven-period daily timetable (08:30 AM to 04:30 PM) with short and lunch breaks,

subject catalogues and editable timetable matrices for the nine departments.

- Teacher-launched attendance sessions with a six-digit rolling PIN and a dynamic QR code, live roster, manual override and defaulter analytics for the VTU 75% rule.
- Hardware-device binding, role-based access control with JSON Web Tokens and a security audit log.
- Reports in CSV format, department dashboards, an Android build and productivity utilities such as the command palette and theme studio.

The project does not attempt face recognition of whole classrooms from CCTV streams, and it does not integrate with the examination or fee modules of the university. These are listed as future enhancements.

1.5 Objective and Methodology

Objectives

1. To automate attendance marking using face recognition so that no roll call or paper register is needed.
2. To prevent proxy attendance through a combination of face verification, GPS geofencing, session PIN and device binding.
3. To link every attendance record to the correct department, semester, section, subject, period and teacher.
4. To provide separate, permission-controlled portals for students, teachers, HODs and the master administrator.
5. To give students and faculty an immediate view of attendance percentage against the VTU 75% requirement.
6. To keep an auditable record of successful and failed attempts, corrections and administrative actions.

Methodology

The project followed an iterative (spiral) approach. The main steps of the proposed attendance pipeline are:

1. **Enrolment.** The student registers with USN, name, e-mail, mobile number and a photograph. The server checks that exactly one face is present before the request is queued for the HOD.
2. **Approval.** The HOD approves or rejects the request (with standard rejection reasons). On approval the user account, student profile and reference photograph are created in

the department database.

3. **Session launch.** The teacher selects subject, section and period and launches a session. A random six-digit PIN and an expiry time are generated.
4. **Check-in.** The student opens the scanner, enters the PIN, and the client sends the camera frame, GPS coordinates and device UUID to the server.
5. **Verification.** The server verifies device binding, geofence, session PIN, duplicates and then the face (HOG detection, 128-d embedding, Euclidean distance against the enrolled photograph).
6. **Persistence and audit.** A PRESENT record with time, period, subject, coordinates, distance, confidence and snapshot is stored and an audit event is logged. Failures are also logged.
7. **Reporting.** Dashboards compute attendance percentages and flag defaulters below 75%.

1.6 Existing System

Attendance in most colleges is currently taken in one of the ways shown in Table 1.1.

Existing method	How it works	Disadvantages
Manual paper register	Teacher calls names; students respond or sign.	Time consuming; proxy signatures; records can be lost; late data entry.
RFID / smart card	Student taps a card on a reader.	Cards can be shared; reader hardware cost and maintenance.
Fingerprint scanner	Finger placed on optical sensor.	Queues, hygiene concerns, sensor wear, enrolment problems.
Basic face-photo apps	A photograph is matched to a stored image.	Can be fooled by photographs; no location, subject or period context; no teacher control.

Table 1.1 Existing Attendance Methods and Their Limitations

Disadvantages of the existing system

- Instructional time is wasted on roll calls.
- Proxy attendance cannot be detected reliably.
- Attendance is not linked to location, subject, period and teacher in real time.
- Defaulters are identified late.
- No audit trail exists for corrections.

1.7 Proposed System

The proposed system is a web and mobile application that verifies four things before an

attendance record is accepted: **who** the student is (face), **where** the student is (GPS geofence), **which class** is running (teacher-launched session PIN and timetable) and **with what device** the attempt is made (device binding). The overall architecture is shown in Figure 1.1 and the workflow in Figure 1.2.

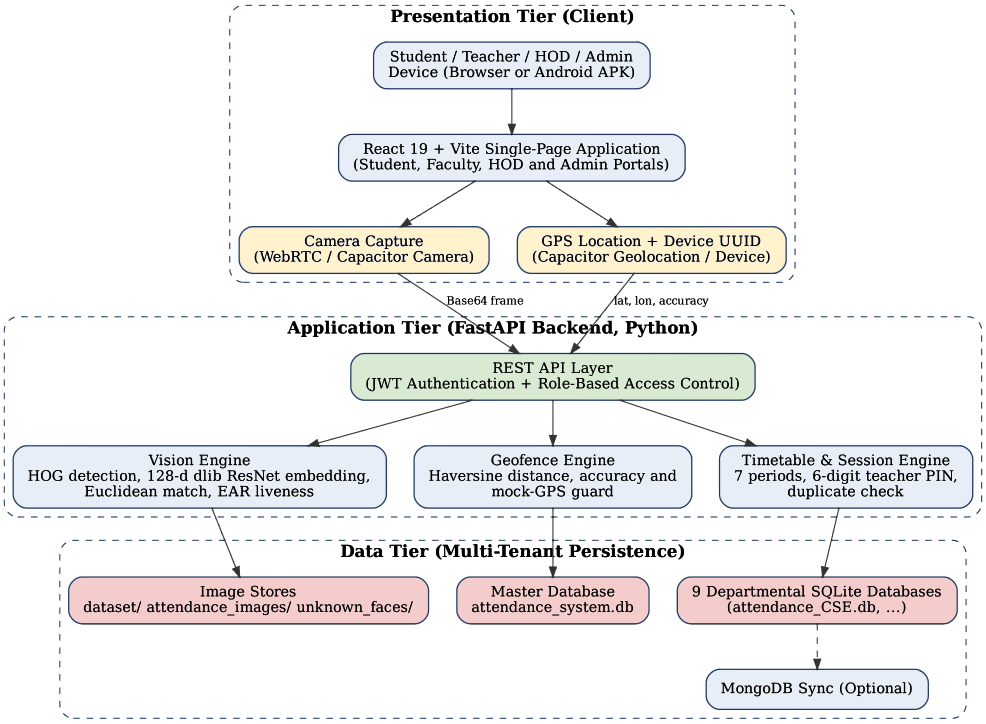


Figure 1.1 System Architecture

Description of the System Architecture

Figure 1.1 shows the three tiers of the system. The client tier is a React single-page application with separate portals for the student, the teacher, the Head of Department (HOD) and the master administrator. On phones the same application is packaged as an Android application with Capacitor so that the native camera, GPS and device-information plug-ins can be used. The client sends the camera frame, the GPS coordinates with their accuracy, the device UUID and the session PIN to the server.

The application tier is a FastAPI server. It authenticates every request with a JSON Web Token, checks the role of the user and then hands the request to the Vision Engine, the Geofence Engine and the Timetable and Session Engine. Each engine has a single responsibility, so a rule can be changed without touching the other modules.

The data tier consists of a master database and one SQLite database for each of the nine departments, together with folders for reference photographs, attendance proofs and mismatched faces. Separate databases isolate the branches, while the master administrator

retains oversight.

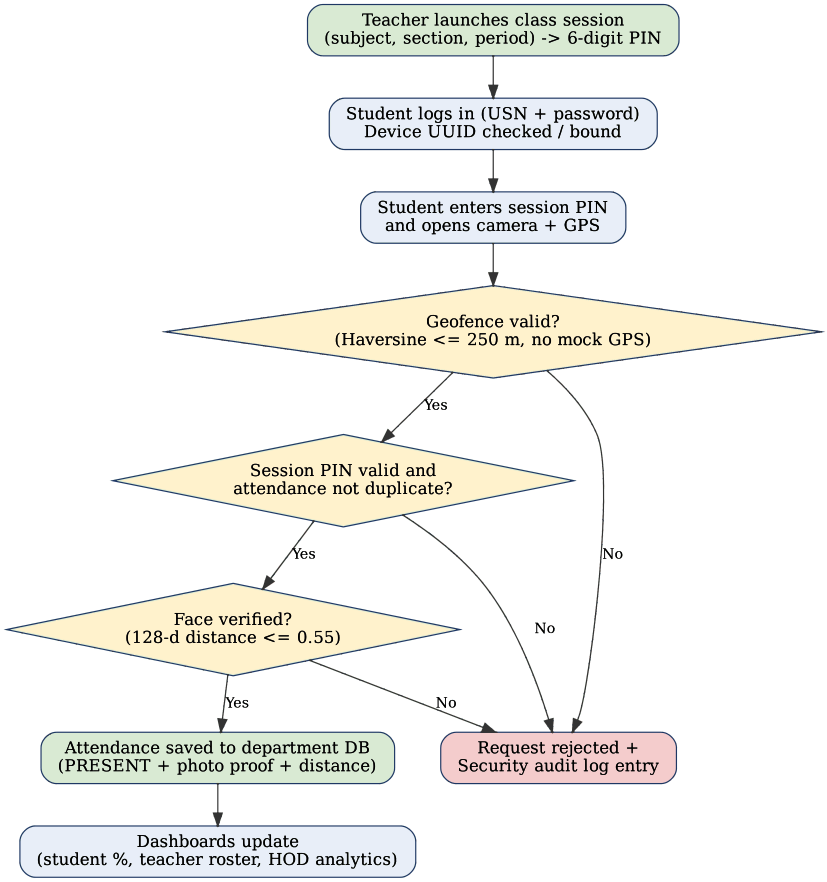


Figure 1.2 Workflow Diagram

Advantages of the proposed system

- Touch-free and fast: no roll call and no special hardware other than the phone or laptop already available.
- Layered protection against proxy attendance: face, geofence, PIN, device binding and duplicate check.
- Every record carries subject, period, teacher context, coordinates, distance and a photo proof.
- Role-based portals and a complete audit ledger support accountability.
- Department-wise databases keep data of the nine branches isolated while the master administrator retains oversight.
- Automatic VTU 75% analysis for students, teachers and HODs.

1.8 Outcome of the Project

The outcome of the project is a working AI Smart Attendance platform consisting of a FastAPI server with 51 REST endpoints, a React web client with four role portals, an Android package and nine departmental databases. The system registers and approves

students, launches class sessions, verifies attendance by face, geofence and PIN, records proof images, computes defaulters and exports reports. A paper describing the work has been published in the IRE Journals (Volume 10, Issue 4, October 2026; Paper ID 1723595), and the certificates of publication are included at the end of this report.

1.9 Report Organization

Chapter 1: gives the overall description of the project, its motivation, problem, scope, objectives, existing and proposed systems.

Chapter 2: reviews the research papers related to face recognition, liveness detection and attendance systems, and identifies the gaps.

Chapter 3: presents the system analysis, including the feasibility study and stakeholders.

Chapter 4: lists the functional, non-functional, software and hardware requirements.

Chapter 5: describes the system design: architecture, Gantt charts, life-cycle model, data flow diagrams, UML diagrams and database design.

Chapter 6: explains the modules and presents the important source code.

Chapter 7: describes testing and validation of the system.

Chapter 8: shows the screens of the system, portal by portal.

1.10 Introduction Summary

Manual and card-based attendance is slow and easy to defeat. This chapter introduced an AI-based smart attendance system that combines face recognition, GPS geofencing, teacher-controlled session PINs and device binding in a role-based, multi-department platform for RGIT. The following chapters present the supporting literature, analysis, design, implementation, testing and output of the system.

Key Contributions of the Work

- A five-layer check for every attendance request: face, geofence, session PIN, device binding and duplicate check.
- Role-based portals for students, teachers, HODs and the master administrator.
- Department-wise databases for the nine B.E. branches with central oversight and an audit ledger.
- Automatic VTU 75% analysis with defaulter lists, using free software and ordinary hardware.

LITERATURE SURVEY

CHAPTER 2

LITERATURE SURVEY

2.1 Related Work

2.1.1 Reference papers

Automatic face recognition has been studied for more than three decades. Early systems used linear projections of pixel intensities, later systems used hand-crafted texture descriptors, and today the best systems use deep convolutional networks trained with metric-learning objectives. For attendance applications the relevant literature has three strands: face detection and description, face embedding and verification, and presentation-attack (liveness) detection. Papers that are the technical basis of this project are summarised in Tables 2.1 to 2.7.

2.1.2 Attendance and campus-automation studies

Many student projects and journal papers describe attendance marking by face recognition using Haar cascades, LBPH or CNN models, usually with a single laptop camera at the classroom door. They report good recognition under controlled lighting but generally do not address proxy attendance by photographs, location verification, device sharing or integration with the college timetable and the roles of teacher and HOD. These gaps motivate the design of the present system.

2.1.3 Biometric privacy

Because face images are personal data, recent work stresses that systems should store as little as necessary, restrict access by role and keep an audit trail. The present system keeps reference photographs in a server-side dataset folder, stores attendance proof snapshots separately, and records access and failures in the audit log.

Observations from the Literature

Three observations recur in the studies. Recognition accuracy depends strongly on lighting and camera quality. A system using only a face image can be defeated by a photograph or video, so liveness cues such as eye blinks are recommended. Very few works link the biometric decision with the timetable, the location of the student and the roles of staff, which is the area addressed here.

2.2 Eigenfaces for Recognition

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
Eigenfaces (PCA)	1991	Turk and Pentland	Projects face images on the principal components of a face training set (eigenfaces) and matches in this low-dimensional space.	Simple and fast; first practical face recognition method.	Very sensitive to illumination, pose and expression changes.

Table 2.1 Eigenfaces for Recognition

The eigenface approach established that a face can be represented by a small number of coefficients. Its weakness under changes of lighting makes it unsuitable for classrooms with windows and uneven illumination.

2.3 Rapid Object Detection using a Boosted Cascade of Simple Features

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
Viola-Jones detector	2001	Viola and Jones	Integral image, AdaBoost feature selection and cascade of classifiers for real-time face detection.	Extremely fast detection on ordinary CPUs.	Detects only near-frontal faces; many false positives; gives no identity.

Table 2.2 Rapid Object Detection using a Boosted Cascade of Simple Features

The cascade detector made real-time face detection possible, but it cannot produce identity information and misses faces turned away from the camera. The present system uses the HOG-based detector of dlib, which is more tolerant of moderate pose changes.

2.4 HOG and LBP

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
HOG and LBP	2005 / 2006	Dalal and Triggs; Ahonen et al.	HOG captures edge orientation; LBP histograms capture local texture for recognition.	HOG is robust and fast (used here for face localisation); LBP is tolerant to monotonic lighting change.	Handcrafted features degrade with large pose, expression or ageing changes.

Table 2.3 HOG and LBP

HOG descriptors with a linear classifier give a dependable face locator on CPU, which is why

the project uses the HOG model for detection. LBP-based recognisers are lightweight but lose accuracy with expression and pose variation, so they were not chosen for identity matching.

2.5 FaceNet: A Unified Embedding for Face Recognition and Clustering

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
FaceNet	2015	Schroff, Kalenichenko and Philbin	Deep CNN trained with triplet loss maps a face to a compact Euclidean embedding; verification is a distance threshold.	State-of-the-art accuracy; same embedding for verification, recognition and clustering.	Needs large training data and substantial compute for training.

Table 2.4 FaceNet: A Unified Embedding for Face Recognition and Clustering

FaceNet showed that a direct mapping to a Euclidean space is sufficient for verification: two faces belong to the same person if the distance between their embeddings is below a threshold. The same principle is used here with the dlib embedding and a threshold of 0.55.

2.6 Deep Residual Learning for Image Recognition and Dlib-ml

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
ResNet and dlib	2016 / 2009	He et al.; King	Residual networks ease training of very deep models; dlib provides a ResNet-based 128-d face embedding and 68-point landmark predictor.	Accurate, open-source and usable on CPU via the face_recognition library.	Accuracy falls with extreme pose, occlusion and very low resolution.

Table 2.5 Deep Residual Learning for Image Recognition and Dlib-ml

The dlib model used by the face_recognition library is a ResNet-style network that outputs a 128-dimensional vector and reports high accuracy on the Labeled Faces in the Wild benchmark. It can be executed on an ordinary server CPU, which keeps the deployment cost low.

2.7 Landmarks and EAR

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
Landmarks and EAR	2014 / 2016	Kazemi and Sullivan; Soukupova and Cech	Regression trees locate 68 facial landmarks; the eye aspect ratio (EAR) computed from six eye landmarks drops sharply during a blink.	Lightweight liveness cue without special hardware.	A recorded video can still show blinks; unreliable at extreme head angles.

Table 2.6 Landmarks and EAR

Eye aspect ratio is computed as EAR = (|p2 - p6| + |p3 - p5|) / (2 |p1 - p4|) over the six landmarks of an eye. The value is stable while the eye is open and falls close to zero when the eyelid closes. The project implements EAR calculation and uses a threshold of 0.22 for blink detection.

2.8 ArcFace: Additive Angular Margin Loss for Deep Face Recognition

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
ArcFace	2019	Deng, Guo, Xue and Zafeiriou	Angular-margin loss improves discrimination of deep face embeddings.	Higher accuracy on public benchmarks than softmax or triplet training.	Heavier models; needs GPU for real-time performance on many faces.

Table 2.7 ArcFace: Additive Angular Margin Loss for Deep Face Recognition

ArcFace-type training gives more separable embeddings. It is noted as a possible upgrade path for the vision engine if larger populations or harder conditions require a stronger model.

2.9 Literature Survey Summary

The survey shows that deep metric-learning embeddings are the accepted basis for face verification, that HOG-based detectors are adequate for locating a single frontal face, and that eye-blink analysis through facial landmarks is a lightweight liveness cue. The following research gaps were identified in typical attendance systems and are addressed by this project:

- Most systems verify only the face; location, class session and device are not verified.
- Attendance is rarely tied to the subject, period and teacher through the college timetable.
- Administrative roles (teacher, HOD, administrator) and auditing are seldom designed together with the biometric module.
- Multi-department colleges need separate data stores with central oversight, which single-

database attendance projects do not provide.

Suitability of the Surveyed Techniques

The techniques reviewed in this chapter were compared for their suitability to a classroom attendance system that must run on an ordinary computer without a GPU. The conclusions are summarised below.

- **Eigenfaces:** a classical projection method that forms the historical basis of face recognition, but it is sensitive to lighting and pose and is therefore not used for the final verification.
- **Boosted cascade detection:** fast and still widely used for locating faces, and a useful reference for the detection stage.
- **HOG and LBP:** descriptors that work well for a single frontal face. The HOG detector is used in this project to locate the face in each camera frame.
- **FaceNet:** introduced the idea of comparing faces through embeddings and distances, which is the principle followed by the verification stage.
- **Deep residual learning and Dlib-ml:** provide the network and toolkit that produce the 128-dimensional face embedding compared by Euclidean distance with a tolerance of 0.55.
- **Landmarks and EAR:** a lightweight liveness cue; the eye aspect ratio is calculated with a threshold of 0.22 to detect a blink.
- **ArcFace:** gives more separable embeddings and is kept as an upgrade path if larger populations or harder conditions require a stronger model.

Position of the Present Work

The literature shows that accurate face verification is already available as open-source software. The remaining problem is to place this capability inside a complete attendance workflow. The present project therefore combines the proven HOG detection and 128-dimensional embedding pipeline with GPS geofencing, teacher-controlled session PINs, device binding, role-based portals and an audit ledger, so that the attendance record produced is not only correct about who the student is, but also about where, when, in which class and from which device the attendance was marked.

SYSTEM ANALYSIS

CHAPTER 3

SYSTEM ANALYSIS

3.1 Introduction to System Analysis

System analysis studies the existing way of working, the needs of the users and the technologies available, and decides whether the proposed system is feasible. For the AI Smart Attendance System the analysis considered the way attendance is recorded at RGIT, the roles involved, the data to be stored and the technical resources required to implement face verification, geofencing and session control reliably on ordinary hardware.

3.2 Feasibility Study

The feasibility study checks whether the proposed system can be developed and operated successfully with the available technology, time, money and people.

3.2.1 Technical Feasibility

The system is technically feasible because every component is available as mature open-source software: Python and FastAPI for the server, OpenCV, dlib and face_recognition for computer vision, SQLAlchemy with SQLite for storage, and React with Vite for the user interface. Capacitor allows the same web code to be packaged as an Android application with native camera and GPS access. The face pipeline runs on a standard CPU; no GPU is required.

3.2.2 Economic Feasibility

All libraries and tools used are free. The system runs on an existing computer or server in the institution and on the students' own smartphones, so there is no need for biometric terminals, smart cards or licences. The only recurring cost is the server and network already maintained by the college.

3.2.3 Operational Feasibility

The interfaces are simple. A student needs only to enter the PIN announced by the teacher and look at the camera. A teacher launches a session with three selections. HODs and the administrator use tabular dashboards with search and filter. Standard rejection reasons, quick-select rosters and a command palette reduce the effort of routine work.

3.2.4 Schedule Feasibility

The project was carried out in two phases over the academic year 2026-2027. Phase I covered the topic study, literature survey, requirement analysis, design and a core face-and-geofence prototype. Phase II covered the full-stack portals, multi-department databases, timetable and session engine, device binding, Android packaging, testing and documentation. The Gantt charts are given in Chapter 5.

3.2.5 Social Feasibility

Students are required to give a face photograph, which is personal data. The system limits its use to attendance, stores it on the institution's server, restricts access by role and logs administrative access. Students can see their own records and can update only their contact details. These measures make the system acceptable within the institution when used with the consent and policy of the college.

3.3 Stakeholders and Usage Scenarios

Stakeholder	Needs	How the system helps
Student	Quick attendance, visibility of percentage and shortage.	Phone check-in in seconds; dashboard gauge, history with photo proofs, VTU 75% forecaster, timetable, digital ID card.
Teacher	Reliable attendance without losing lecture time.	Launches session with PIN and QR; live roster; manual override; defaulter list; CSV export.
HOD	Department-level control and reports.	Approves registrations; manages faculty, subjects, timetable and devices; analytics.
Master Admin (MOHD)	Institution-wide governance and security.	Appoints HODs, resets credentials, calibrates geofence, reviews audit ledger and database health.

Table 3.1 Stakeholders and Their Needs

3.4 System Analysis Summary

The analysis shows that the project is technically, economically, operationally, socially and schedule-wise feasible. The needs of the four stakeholders define the portals and features, and the use of free software on commodity hardware keeps the solution affordable for the institution.

REQUIREMENT ANALYSIS

CHAPTER 4

REQUIREMENT ANALYSIS

4.1 Functional Requirements

Functional requirements describe what the system must do.

1. **Student registration.** The system shall accept USN, name, e-mail, mobile number, password and a face photograph; shall reject photographs with no face or with more than one face; and shall queue the request for approval.
2. **Approval workflow.** The HOD shall approve (single or batch) or reject registrations with a stated reason, and the student shall be able to check the status using the USN.
3. **Authentication.** The system shall authenticate students, teachers, HODs and the master administrator and issue a JSON Web Token valid for seven days; student logins shall bind or verify the hardware-device UUID.
4. **Session management.** A teacher shall launch a session for a subject, section and period with a configurable duration (default 15 minutes), generating a six-digit PIN and a QR code.
5. **Attendance marking.** The system shall accept a camera frame, GPS coordinates, device UUID and PIN, and mark attendance only after device, geofence, session, duplicate and face checks succeed.
6. **Geofencing.** The system shall compute the Haversine distance to the campus centre, reject locations beyond the configured radius, poor GPS accuracy (more than 65 m) and mock locations.
7. **Timetable.** The system shall map periods 1 to 7 to clock times, lock breaks, and let the HOD edit the department timetable matrix.
8. **Corrections.** A teacher shall be able to override attendance with a reason (medical leave, duty, device failure).
9. **Analytics.** The system shall compute attendance percentages per subject, flag students below 75% and export filtered reports to CSV.
10. **Administration.** The HOD shall manage faculty, subject assignments, passwords and device unbinding; the master administrator shall manage HODs, geofence and view

audit logs and department database status.

11. **Audit.** The system shall log logins, attendance success, face failures, geofence breaches, mock GPS and unauthorised device attempts.

4.2 Non-Functional Requirements

Requirement	Description
Performance	A check-in request (face detection, embedding and comparison) should complete within a few seconds on a standard CPU server.
Reliability	Writes use SQLite in WAL mode with a busy timeout; duplicate attendance for the same student, subject, date and period is prevented.
Security	JWT authentication (HS256); role checks on every protected endpoint; salted SHA-256 password hashing; audit logging; device binding.
Usability	Responsive interface for phones and desktops; keyboard shortcuts, theme studio and command palette.
Scalability	Separate database per department; optional MongoDB synchronisation; stateless API.
Maintainability	Modular back-end (vision, geofence, timetable, auth, models) and component-based front-end.
Portability	Runs on Windows and Linux servers; client works in modern browsers and as an Android application.
Privacy	Face images are stored in server folders accessible only through authenticated endpoints; audit trail of access.

Table 4.1 Non-Functional Requirements

4.3 Software Requirements

The software environment and libraries required for development and deployment are listed in Table 4.2.

Category	Software / Library	Purpose
Programming language	Python 3.10+	Back-end logic and computer vision
Web framework	FastAPI, Uvicorn	REST API server with automatic OpenAPI documentation
Computer vision	OpenCV, NumPy	Image decoding, drawing proof snapshots
Face recognition	face_recognition (dlib ResNet), HOG detector	128-d embeddings, landmarks, distance computation

Category	Software / Library	Purpose
Database	SQLAlchemy, SQLite (WAL); PyMongo (optional)	Departmental databases and optional cloud sync
Authentication	PyJWT, hashlib	Tokens and password hashing
Front-end	React 19, Vite 8, lucide-react, canvas-confetti	Single-page application and UI
Mobile packaging	Capacitor 8 (Android, Camera, Geolocation, Device)	Android app with native camera and GPS
Development tools	Visual Studio Code, Git, npm	Coding and build
Browsers	Chrome, Edge, Firefox, Android WebView	Client runtime

Table 4.2 Software Requirements

4.3.1 Operating System

The server was developed and tested on Windows 10/11 (64-bit). The same Python code runs on Ubuntu 22.04 LTS, which is recommended for production. Windows is convenient for development because of editor and tool support; Linux offers better stability, easier service management and lower overhead when the server runs continuously. The client side requires only a modern browser on Windows, Linux, macOS, Android or iOS, or the Android application package.

4.4 Software Requirement Summary

The software stack consists entirely of open-source components. Python with FastAPI provides the REST layer, OpenCV and the dlib-based face_recognition library provide the vision pipeline, SQLAlchemy with SQLite provides per-department persistence, and React with Capacitor provides the web and Android clients.

4.5 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 (8th Gen) / Ryzen 3	Intel Core i5/i7 / Ryzen 5 or higher
RAM	8 GB	16 GB
Storage	10 GB free	50 GB SSD (for images and databases)
Student device	Smartphone with front camera and GPS	Smartphone with 8 MP+ camera and high-accuracy GPS
Network	Campus Wi-Fi / LAN	Gigabit LAN, 5 GHz Wi-Fi
Display (staff)	1366 x 768	1920 x 1080

Table 4.3 Hardware Requirements

4.6 Hardware Requirements Summary

The server needs only a modest computer because the HOG detector and the embedding network run on the CPU. The students use their own smartphones, which already contain the camera and GPS sensor that the system needs.

4.7 Requirement Analysis Summary

This chapter listed the functional and non-functional requirements and the software and hardware needed to build and run the system. All requirements can be met with freely available software and ordinary hardware.

Assumptions and Constraints

The requirements described in this chapter are based on the following assumptions and constraints.

- Every student owns a smartphone with a working front camera and GPS sensor, and the campus Wi-Fi or mobile data is available during the class.
- A clear reference photograph of every student is available at enrolment, because the live face is compared with this photograph.
- The geofence centre and radius are configured by the master administrator, and a GPS reading with accuracy worse than 65 m is rejected.
- The face match is accepted only when the Euclidean distance between the two embeddings is at most 0.55.
- Each department keeps its data in its own SQLite database, which is sufficient for the load of a single college; an optional MongoDB mirror is available when cloud synchronisation is needed.

Priority of the Requirements

- **Essential:** face verification, geofence check, session PIN, device binding, role-based login and the audit ledger.
- **Important:** live roster, manual correction with a reason, defaulter analysis and report export.
- **Desirable:** optional cloud synchronisation, Android packaging and interface customisation such as themes and the command palette.

SYSTEM DESIGN

CHAPTER 5

SYSTEM DESIGN

The system design of the AI Smart Attendance System defines how the client, server, vision engine, geofence engine, timetable engine and databases work together. The design follows a three-tier, modular architecture in which each module has a single responsibility and communicates through well-defined REST interfaces.

5.1 System Architecture Overview

5.1.1 Architecture

The architecture (Figure 1.1) has three tiers:

1. **Presentation tier.** A React single-page application with four portals, running in the browser or in an Android package. It captures the camera frame, reads GPS coordinates and stores the device UUID.
2. **Application tier.** A FastAPI server that authenticates users, checks roles and runs the Vision Engine (face detection, embedding, distance, EAR), the Geofence Engine (Haversine distance, GPS accuracy and mock guard) and the Timetable and Session Engine (periods, PIN, duplicates).
3. **Data tier.** A master database for system-wide settings and nine departmental SQLite databases (EE, CSE, ECE, BME, ME, CSE-AI, CSE-IoT/CS/BT, ISE, AIML), plus folders for reference photographs, attendance proofs and mismatched faces. An optional MongoDB mirror is supported.

5.1.2 System Flow Design

The flow of a check-in is as follows (Figure 1.2):

1. The teacher launches a session; the server stores subject, section, period, six-digit PIN and expiry time.
2. The student logs in; the server checks or binds the device UUID.
3. The student enters the PIN and the client sends image, latitude, longitude, accuracy, mock flag, device UUID and PIN.
4. The server rejects the request if the device differs from the bound device, if the location is outside the geofence, if the PIN or session is invalid, or if attendance already exists.

5. The face in the live frame is compared with the enrolled photograph. A match is accepted if the Euclidean distance is at most 0.55.
6. The record is stored with a proof snapshot and an audit entry; the client shows the result.

5.1.3 Design Characteristics

- **Modularity:** vision, geofence, timetable, authentication and portals are independent modules.
- **Multi-tenancy:** each department has its own database; the master administrator has jurisdiction over all.
- **Defence in depth:** five independent checks must pass for every attendance.
- **Auditability:** every success and failure is stored with coordinates and details.
- **Configurability:** geofence centre and radius, session duration, timetable and subjects can be changed from the portals.

5.2 Gantt Chart

5.2.1 Gantt Chart of Phase One

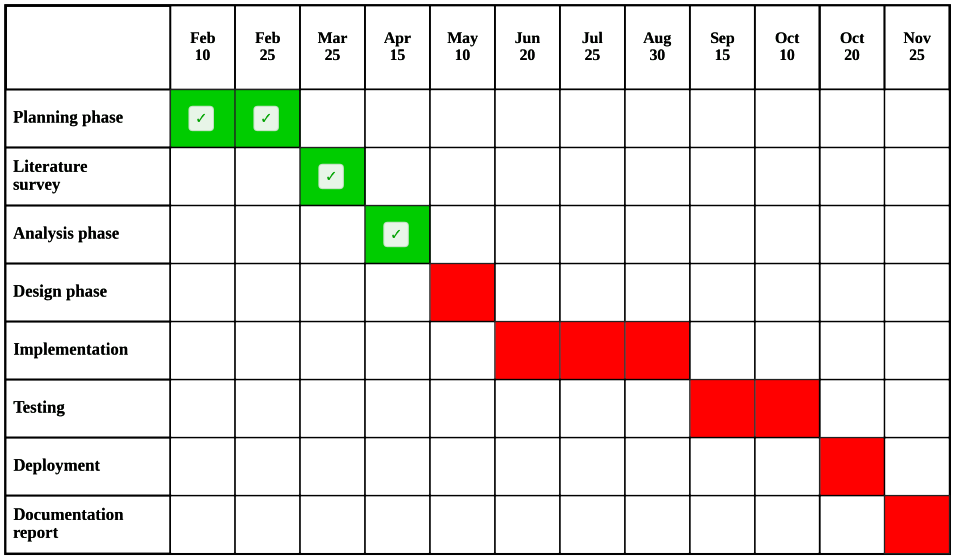


Figure 5.1 Gantt Chart for Phase One

Explanation of the Phase One Schedule

Figure 5.1 shows the schedule of the project from February to November. The planning phase (10 and 25 February), the literature survey (25 March) and the analysis phase (15 April) are completed in Phase One and are marked in green with a tick. The remaining activities, marked in red, are the design phase (10 May), implementation (20 June to 30 August), testing (15

September to 10 October), deployment (20 October) and the documentation report (25 November), which are carried out in Phase Two.

5.2.2 Gantt Chart of Phase Two

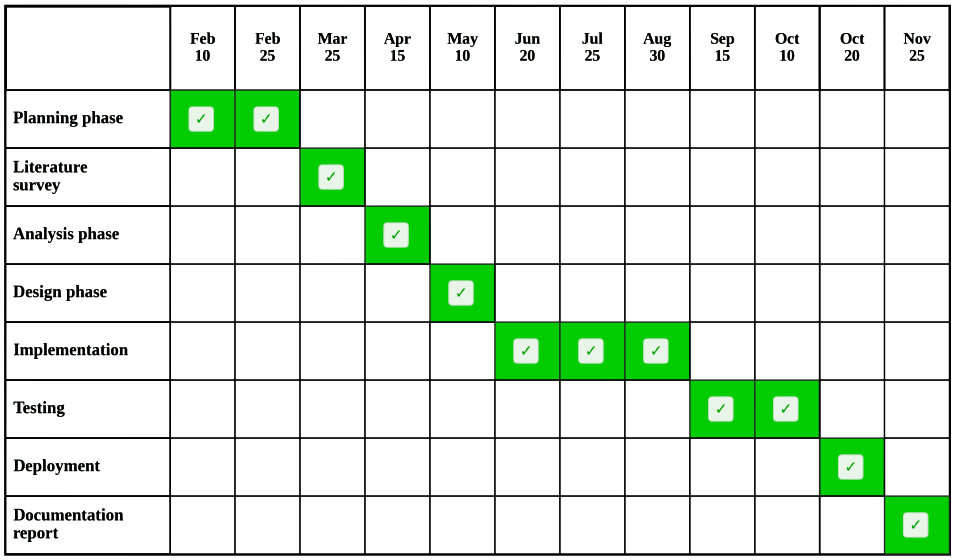


Figure 5.2 Gantt Chart for Phase Two

Explanation of the Phase Two Schedule

Figure 5.2 shows the same schedule at the end of Phase Two, where every activity is marked in green. The design phase was completed on 10 May, implementation ran from 20 June to 30 August, testing from 15 September to 10 October, deployment was done on 20 October and the documentation report was prepared by 25 November.

5.3 Life Cycle Model

The project was developed using the **Spiral model**, which combines iterative development with systematic risk analysis. Each loop of the spiral has four activities:

1. **Planning and requirement analysis:** identify requirements for face verification, geofence and portals of the current iteration.
2. **Risk analysis and prototyping:** evaluate risks such as photograph spoofing, GPS drift, phone camera quality and shared devices, and build prototypes to test them.
3. **Implementation and deployment:** implement the API routes, database models and React components for the iteration.
4. **Evaluation and testing:** test with sample students and refine thresholds such as face tolerance and geofence radius.

Advantage of the Spiral Model

Risks such as photograph spoofing, GPS drift and shared devices were examined early on prototypes, and each loop produced a working version that sample students could try.

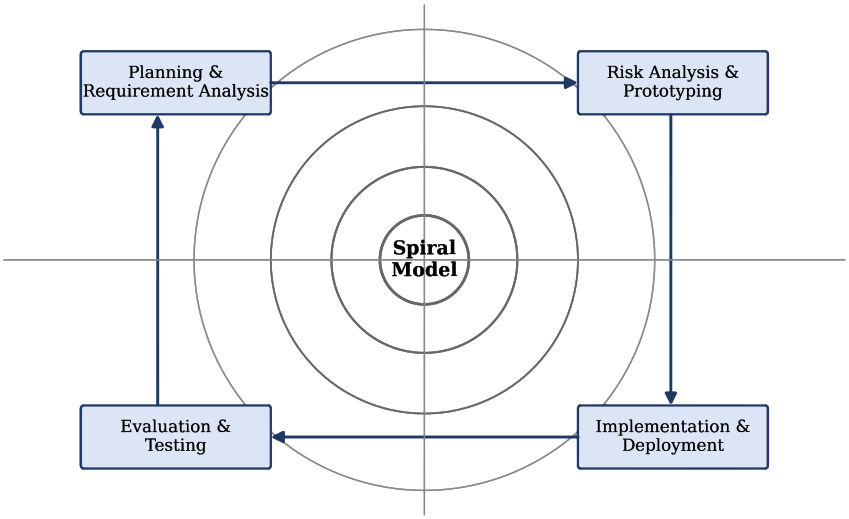


Figure 5.3 Life Cycle Model (Spiral Model)

5.4 Data Flow Diagram

The data flow diagrams show how data moves between the external entities, processes and data stores. Level 0 is the context diagram (Figure 5.4). Level 1 (Figure 5.5) decomposes the system into authentication, registration, session management, attendance verification and reporting. Level 2 (Figure 5.6) expands the attendance verification process.

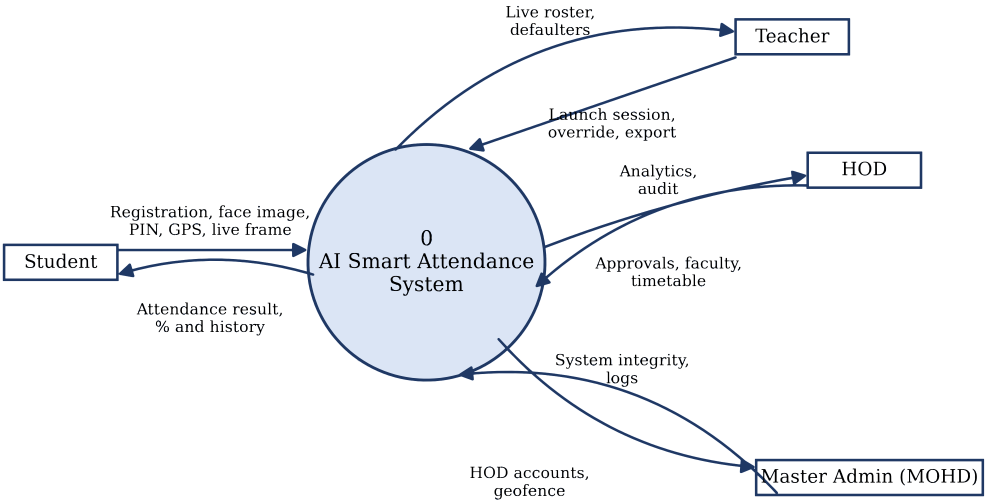


Figure 5.4 DFD Level 0 (Context Diagram)

Explanation of the Context Diagram

The context diagram (Figure 5.4) shows the whole system as a single process that exchanges data with its external entities. The student, the teacher, the HOD and the master administrator

send requests such as registration, login, session launch, attendance check-in and report queries, and receive approvals, results, dashboards and reports in return.

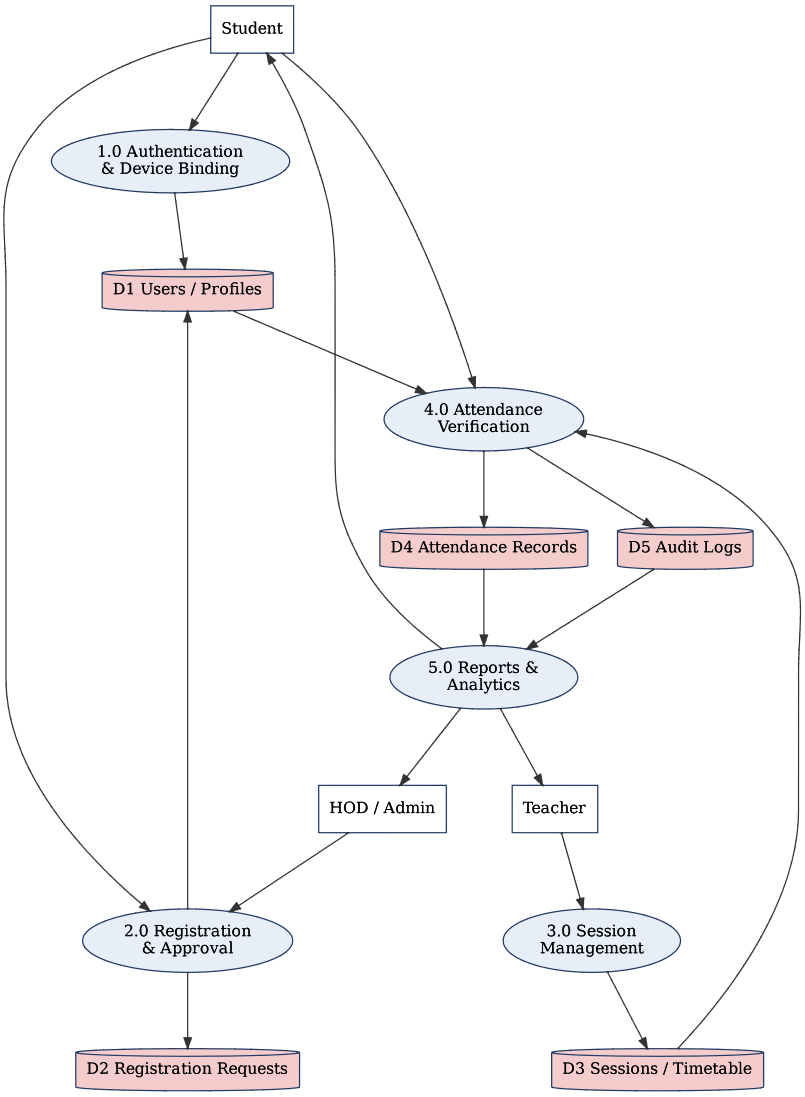


Figure 5.5 DFD Level 1

Explanation of the Level 1 Diagram

Figure 5.5 decomposes the system into five main processes. The data stores are the departmental databases, the audit log and the photograph and proof folders.

- **Authentication (1.0):** checks credentials and role, issues the token and binds the student device.
- **Registration (2.0):** stores the request as pending until the HOD decides.
- **Session management (3.0):** launches a session with the six-digit PIN and expiry time.
- **Attendance verification (4.0):** applies the device, geofence, session, duplicate and face checks (Figure 5.6).
- **Reporting (5.0):** computes percentages and flags defaulters below 75%.

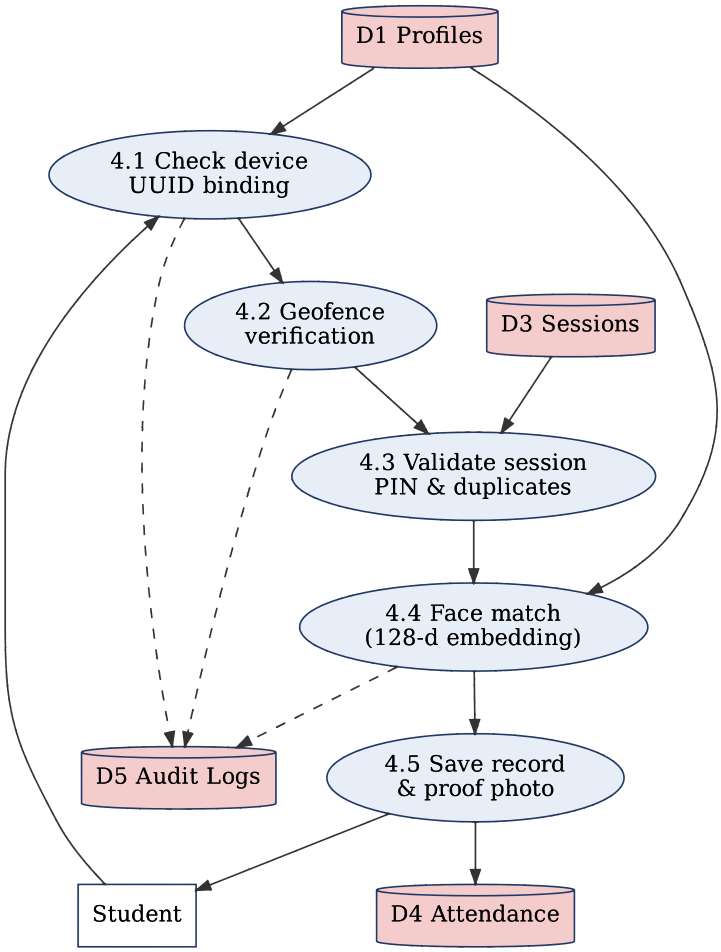


Figure 5.6 DFD Level 2 for Attendance Verification (Process 4.0)

Data element	Flow description
Registration request	USN, name, e-mail, mobile, semester, section, password hash, photograph; stored with status PENDING / APPROVED / REJECTED.
Session record	Faculty, subject, section, period, six-digit PIN, active flag and expiry time.
Check-in payload	Base64 camera frame, latitude, longitude, accuracy, mock flag, device UUID, PIN, subject code.
Attendance record	Student, subject, date, time, period, status, coordinates, distance, confidence, liveness flag, snapshot file, marked-by.
Audit event	Event type, username, role, coordinates, details and time.

Table 5.1 Data Flow Elements

5.5 Use Case Diagram

Figure 5.7 shows the actors (Student, Teacher, HOD and Master Admin) and the main use cases. Student use cases include registration, login, marking attendance and viewing attendance. Teacher use cases include launching sessions, live roster, override and analytics. The HOD approves registrations, manages faculty and timetable, and unbinds devices. The Master Admin appoints HODs and calibrates the geofence.

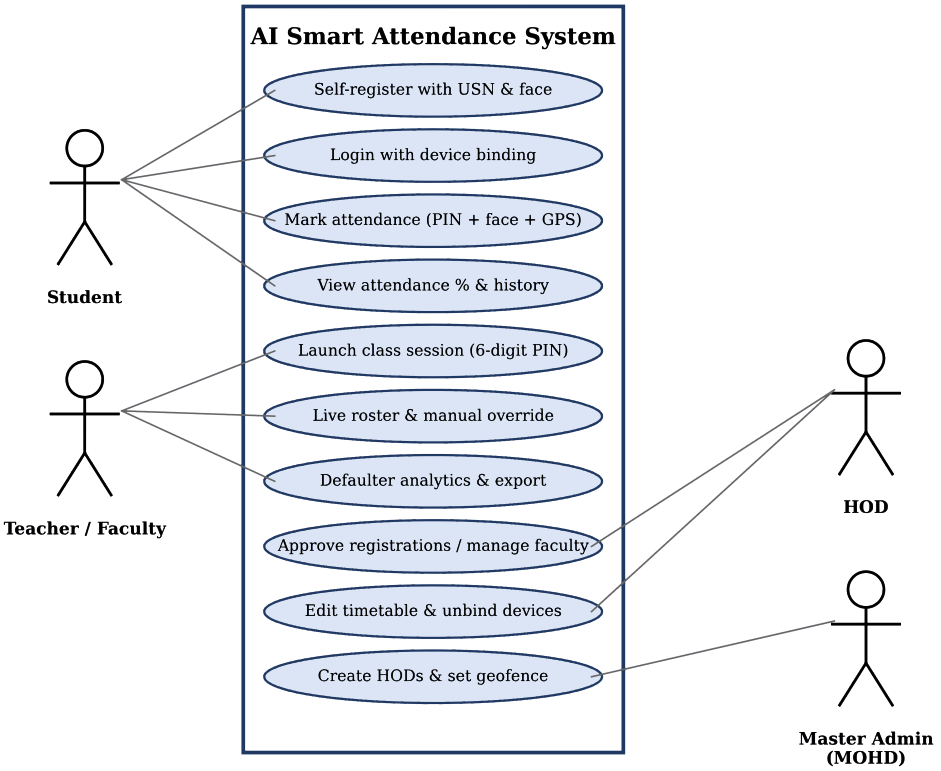


Figure 5.7 Use Case Diagram

5.6 Sequence Diagram

The sequence diagram (Figure 5.8) shows the order of messages when a student marks attendance, from the PIN entry on the client to the database insert and the success card shown to the student.

Order of Messages in the Sequence Diagram

- The student enters the PIN; the client sends the frame, GPS location and device UUID.
- The server checks the device, geofence, session PIN and duplicate rule, then the face.
- On a match the record and proof snapshot are saved, an audit event is logged and the success card is shown.

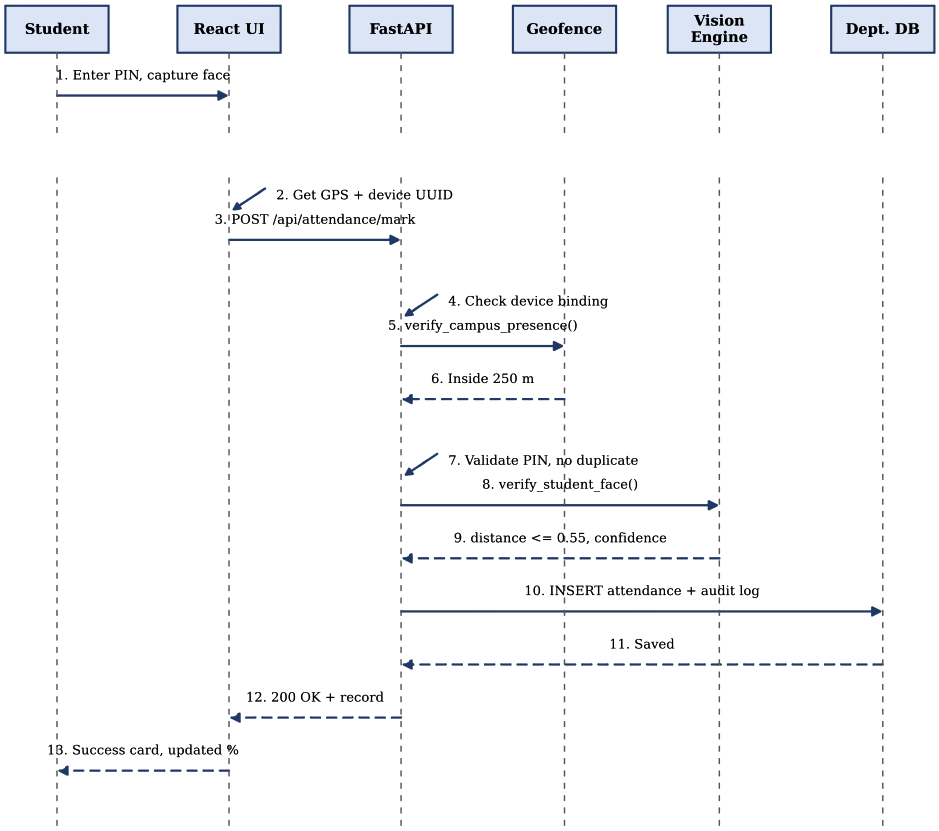


Figure 5.8 Sequence Diagram for Live Attendance Verification

5.7 Class Diagram

The class diagram (Figure 5.9) presents the main entities and engines. User has one StudentProfile or FacultyProfile. A faculty member starts many ActiveClassSessions, a session validates AttendanceRecords, and each record uses the Vision, Geofence and Timetable engines and writes to the SecurityAuditLog.

Main Classes of the System

- **User:** holds the login details and role, and has one StudentProfile or one FacultyProfile.
- **StudentProfile and FacultyProfile:** hold the department, semester and section for students and the details of the faculty member.
- **ActiveClassSession:** started by a faculty member for a subject, section and period, with the PIN and expiry time.
- **AttendanceRecord:** stores the student, subject, date, time, period, status, coordinates, distance, confidence and snapshot.
- **Vision, Geofence and Timetable engines:** used by every record to verify the face, location and period.
- **SecurityAuditLog:** receives an entry for every success and failure.

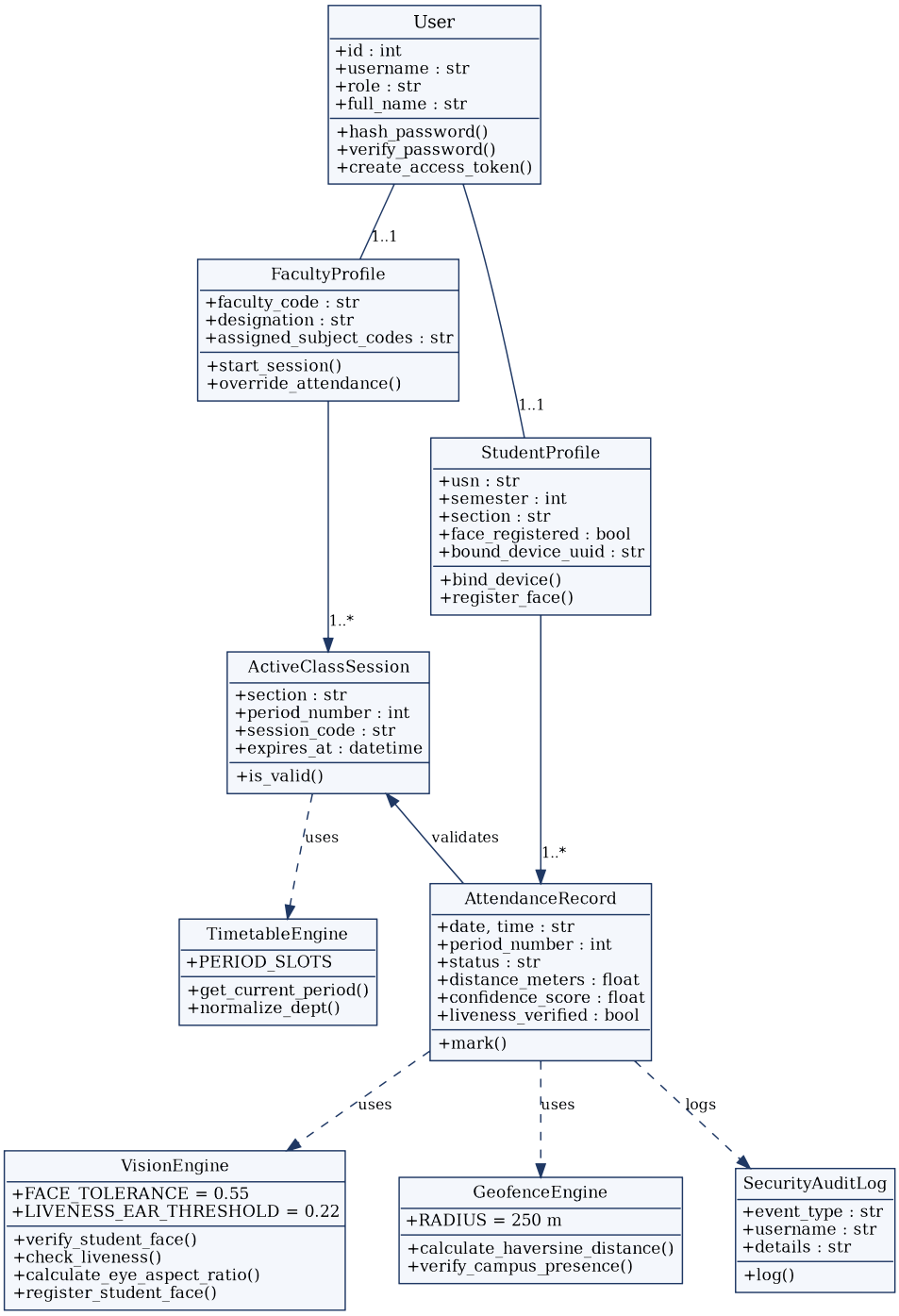


Figure 5.9 Class Diagram

5.8 Database Design (ER Diagram)

Each departmental database contains the tables shown in Figure 5.10. The tables are defined in models.py with SQLAlchemy; the master database holds system-level settings, and departmental databases hold users, profiles, subjects, timetable, sessions, attendance and audit tables.

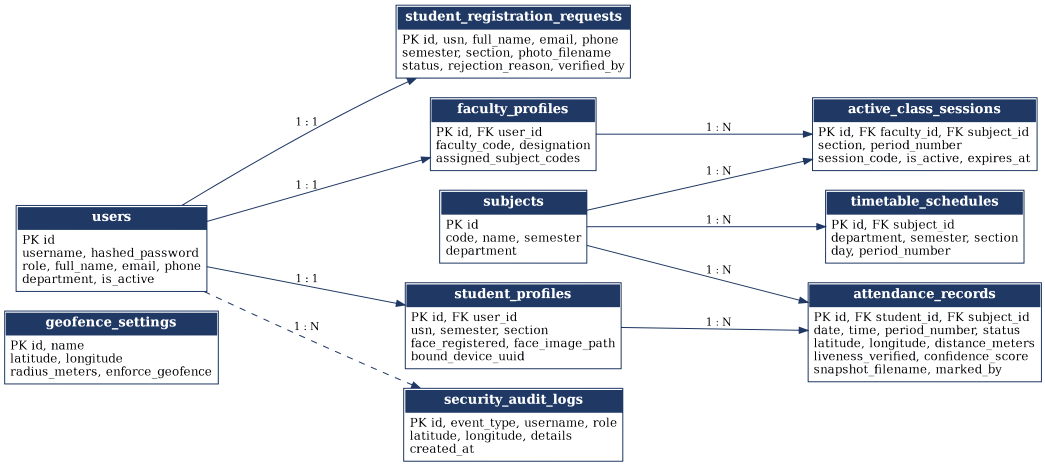


Figure 5.10 Entity Relationship Diagram

5.9 Security Design

Threat	Countermeasure in the system
Proxy attendance by another person	Face embedding comparison with the enrolled photograph (distance <= 0.55); photo proof stored.
Attendance from outside the campus	Haversine geofence (default 250 m); rejects poor accuracy (> 65 m) and mock location.
Marking without class being held	Six-digit PIN launched by the teacher; session expires; section must match.
One phone used for many students	Device UUID bound to the student on first use; mismatch is blocked and logged.
Duplicate marking	Unique check on student, subject, date and period.
Unauthorised access	JWT with role checks; separate student, faculty, HOD and master-admin dependencies.
Disputed corrections	Manual override requires a reason and is recorded with the marker.

Table 5.2 Threats and Countermeasures

5.10 System Design Summary

The design divides the system into presentation, application and data tiers with independent vision, geofence and timetable engines. The data flow, UML and database diagrams describe the structure and behaviour in detail and form the basis for the implementation described in the next chapter.

The design can be extended without changing existing modules: a stronger embedding model can replace the Vision Engine and a cloud database can be attached through the optional MongoDB mirror.

SYSTEM IMPLEMENTATION

CHAPTER 6

SYSTEM IMPLEMENTATION

The system is implemented in Python (server) and JavaScript (client). The server exposes 51 REST endpoints from a single FastAPI application. The implementation is divided into the modules described below, followed by the important source code.

6.1 Modular Approach

6.1.1 Vision Engine (vision.py)

Decodes the Base64 frame to an OpenCV image, locates faces with the HOG detector, rejects frames with no face or more than one face, extracts a 128-d embedding with dlib ResNet and calculates the Euclidean distance to the embedding of the enrolled photograph. The confidence shown to the user is (1 - distance / tolerance) x 100 and is limited to 0-100. Each attempt saves a snapshot with a bounding box; mismatches are also saved to the unknown_faces folder. The module also implements eye-aspect-ratio calculation (check_liveness) with threshold 0.22.

6.1.2 Geofence Engine (geofence.py)

Computes the great-circle distance between the device and the campus centre using the Haversine formula. Settings (centre, radius, enforcement flag) are stored in the geofence_settings table and default to latitude 13.033090, longitude 77.597786 and radius 250 m. The engine rejects mock locations and GPS accuracy worse than 65 m, and logs MOCK_GPS_DETECTED and GEOFENCE_BREACH_ATTEMPT events.

6.1.3 Timetable and Session Engine (timetable.py, main.py)

Defines the seven periods and the two breaks (Table 6.1), resolves the current period from the clock, holds the subject catalogue of each department and generates default timetables. A teacher session records subject, section, period, six-digit PIN and expiry; the PIN is validated at check-in.

Slot	Start	End	Type
Period 1	08:30 AM	09:30 AM	Lecture
Period 2	09:30 AM	10:30 AM	Lecture
Short Break	10:30 AM	10:45 AM	Break
Period 3	10:45 AM	11:45 AM	Lecture
Period 4	11:45 AM	12:45 PM	Lecture
Lunch Break	12:45 PM	01:30 PM	Break
Period 5	01:30 PM	02:30 PM	Lecture / Lab
Period 6	02:30 PM	03:30 PM	Lecture / Lab
Period 7	03:30 PM	04:30 PM	Lecture / Seminar

Table 6.1 RGIT Seven-Period Daily Timetable

6.1.4 Authentication and Access Control (auth.py)

Passwords are hashed with salted SHA-256. A JWT (HS256) valid for seven days is issued on login. Dependencies require_student, require_faculty, require_admin (admin or HOD) and require_master_admin restrict the endpoints.

6.1.5 Multi-Tenant Data Layer (database.py, models.py)

A master database and nine department databases (EE, CSE, ECE, BME, ME, CSE-AI, CSE-IoT/CS/BT, ISE, AIML) are created with SQLAlchemy. SQLite is run in WAL mode with a 30-second busy timeout. Tables: users, student_profiles, faculty_profiles, subjects, timetable_schedules, active_class_sessions, attendance_records, student_registration_requests, geofence_settings and security_audit_logs. An optional PyMongo module mirrors student data to MongoDB when a connection string is configured.

6.1.6 Front-end Portals

The React application contains LoginModal (four login tabs with USN builder), StudentRegisterForm (multi-step registration with camera capture), StudentPortal (dashboard, scanner, history, timetable, ID card, radar, forecaster), FacultyPortal (subjects, session launch with PIN and QR, roster, correction, reports, timetable, profile) and AdminPortal (HOD and master-admin dashboards, approvals, faculty, timetable, geofence, audit and database status). Utility components include CommandPalette (Ctrl+K), ThemeCustomizerModal, NoticeBoardModal and CampusRadar. The API layer (api.js) wraps all fetch calls and attaches the bearer token and device UUID.

6.2 Programming Code

The listings below contain only the key snippets of the project source code. Imports, repeated error returns and long validation blocks are replaced by # ....

6.2.1 Configuration (backend/rgit_app/config.py)

Holds the JWT settings, campus coordinates, geofence radius, face-match tolerance and blink threshold.

```
SECRET_KEY = os.getenv("SECRET_KEY", "<default-development-key>")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24 * 7   # 7-day session
RGIT_LATITUDE = 13.033089637756348   # campus centre
RGIT_LONGITUDE = 77.59778594970703
DEFAULT_GEOFENCE_RADIUS_METERS = 250.0
FACE_TOLERANCE = 0.55   # lower value = stricter match
LIVENESS_EAR_THRESHOLD = 0.22   # eye-aspect-ratio below this = blink
```

6.2.2 Vision engine (backend/rgit_app/vision.py)

Compares the live frame with the enrolled photograph. It rejects frames with no face or more than one face, measures the Euclidean distance between 128-d embeddings and converts it to a confidence percentage.

```
def verify_student_face(usn, live_base64_image, enrolled_path=None):
    # 1. Enrolled photo -> 128-d embedding
    enrolled_encoding = face_recognition.face_encodings(enrolled_rgb)[0]
    # 2. Live frame -> face locations (HOG detector)
    live_locations = face_recognition.face_locations(live_rgb, model="hog")
    if len(live_locations) == 0:
        return {"verified": False, "error_code": "NO_FACE_DETECTED", ...}
    if len(live_locations) > 1:
        return {"verified": False, "error_code": "MULTIPLE_FACES", ...}
    live_encoding = face_recognition.face_encodings(live_rgb, live_locations)[0]
    # 3. Distance -> match decision and confidence (0-100 %)
    distance = float(face_recognition.face_distance(
        [enrolled_encoding], live_encoding)[0])
    is_match = distance <= FACE_TOLERANCE
    confidence = max(0.0, min(100.0, (1.0 - distance / FACE_TOLERANCE) * 100.0))
```

```
    # 4. Save proof snapshot (green box = match, red box = mismatch)
    if not is_match:
        return {"verified": False, "error_code": "FACE_MISMATCH", ...}
    return {"verified": True, "distance": round(distance, 3),
            "confidence": round(confidence, 1), ...}
```

6.2.3 Geofence engine (backend/rgit_app/geofence.py)

Uses the Haversine formula for the great-circle distance between the student and the campus centre. Mock GPS and GPS accuracy worse than 65 m are rejected first.

```
def calculate_haversine_distance(lat1, lon1, lat2, lon2) -> float:
    phi1, phi2 = math.radians(lat1), math.radians(lat2)
    delta_phi = math.radians(lat2 - lat1)
    delta_lambda = math.radians(lon2 - lon1)
    a = (math.sin(delta_phi / 2.0) ** 2 +
         math.cos(phi1) * math.cos(phi2) * math.sin(delta_lambda / 2.0) ** 2)
    c = 2.0 * math.atan2(math.sqrt(a), math.sqrt(1.0 - a))
    return round(6371000.0 * c, 2)   # Earth radius in metres


def verify_campus_presence(user_lat, user_lon, accuracy_meters=None,
                           is_mock=False, db=None, username=None):
    # 1. Anti-mock-GPS guard
    if is_mock:
        return {"valid": False, "error_code": "MOCK_LOCATION_DETECTED", ...}
    # 2. GPS accuracy guard
    if accuracy_meters and accuracy_meters > 65.0:
        return {"valid": False, "error_code": "POOR_GPS_ACCURACY", ...}
    # 3. Distance check against the allowed radius
    if distance > settings.radius_meters and settings.enforce_geofence:
        return {"valid": False, "error_code":
        "OUTSIDE_CAMPUS_PERIMETER", ...}
    return {"valid": True, "distance_meters": distance, "is_inside": True, ...}
```

6.2.4 Timetable periods (backend/rgit_app/timetable.py)

Defines the seven daily periods and the two breaks and resolves the current slot from the clock. During a break the class is locked.

```
PERIOD_SLOTS = [
    {"period": 1, "start": time(8, 30), "end": time(9, 30),
     "label": "08:30 AM - 09:30 AM"},
    {"period": "SHORT_BREAK", "start": time(10, 30), "end": time(10, 45), ...},
    {"period": "LUNCH_BREAK", "start": time(12, 45), "end": time(13, 30), ...},
    # ... periods 2 to 7 follow the same pattern
]
def get_current_period(now_time=None):
    for slot in PERIOD_SLOTS:
        if slot["start"] <= now_time < slot["end"]:
            if isinstance(slot["period"], str) and "BREAK" in slot["period"]:
                return {"is_class_time": False, "is_break": True, ...}
            return {"is_class_time": True, "period_number": slot["period"], ...}
```

6.2.5 Authentication and role guards (backend/rgit_app/auth.py)

Passwords are stored as salted SHA-256 hashes. A JWT (HS256) is issued at login and decoded on every request; require_role() returns HTTP 403 when the role is not allowed.

```
def hash_password(password: str, salt: Optional[str] = None) -> str:
    salt = salt or "<application-salt>"
    return hashlib.sha256((salt + password).encode("utf-8")).hexdigest()

def require_role(allowed_roles: list):
    def role_checker(current_user: User = Depends(get_current_user)) -> User:
        if current_user.role not in allowed_roles:
            raise HTTPException(status_code=403, detail="Access forbidden ...")
        return current_user
    return role_checker

require_admin = require_role(["admin", "hod"])
require_faculty = require_role(["faculty", "admin", "hod"])
require_student = require_role(["student"])
```

6.2.6 Database models (backend/rgit_app/models.py)

StudentProfile stores face enrolment and device binding; AttendanceRecord keeps the location, confidence and snapshot proof of each mark; ActiveClassSession holds the teacher's live PIN and expiry.

```
class StudentProfile(Base):
    usn = Column(String(32), unique=True, index=True, nullable=False)
    section = Column(String(8), default="A")
    face_registered = Column(Boolean, default=False)
    face_image_path = Column(String(256), nullable=True)
    bound_device_uuid = Column(String(128), nullable=True)   # anti-proxy

class AttendanceRecord(Base):
    student_id = Column(Integer, ForeignKey("student_profiles.id"))
    date = Column(String(16), index=True, nullable=False)
    period_number = Column(Integer, nullable=False)
    distance_meters = Column(Float, nullable=True)
    confidence_score = Column(Float, nullable=True)
    snapshot_filename = Column(String(256), nullable=True)

class ActiveClassSession(Base):
    session_code = Column(String(16), nullable=False)   # 6-digit PIN
    is_active = Column(Boolean, default=True)
    expires_at = Column(DateTime, nullable=False)
```

6.2.7 Launching a teacher session (backend/rgit_app/main.py)

The server generates a random six-digit PIN, closes any earlier active session of that teacher and stores the new session with an expiry time (default 15 minutes).

```
@app.post("/api/sessions/start")
def start_attendance_session(payload: StartSessionRequest,
                             current_user: User = Depends(require_faculty),
                             db: Session = Depends(get_db)):
    session_pin = f"{random.randint(100000, 999999)}"   # dynamic 6-digit PIN
    db.query(ActiveClassSession).filter(          # expire old sessions
        ActiveClassSession.faculty_id == fac_profile.id,
```

```
        ActiveClassSession.is_active == True).update({"is_active": False})
    expires_at = datetime.utcnow() + timedelta(minutes=payload.duration_minutes)
    new_session = ActiveClassSession(faculty_id=fac_profile.id, ...,
        session_code=session_pin, is_active=True, expires_at=expires_at)
    db.add(new_session)
    db.commit()
    return {"success": True, "session_code": session_pin, ...}
```

6.2.8 Marking attendance (backend/rgit_app/main.py)

The central endpoint. A request passes six checks in order: device binding, GPS geofence, teacher session PIN, duplicate check, face verification and finally the save with an audit-log entry. Any failed check stops the request with a clear error message.

```
@app.post("/api/attendance/mark")
def mark_attendance(payload: MarkAttendanceRequest,
                    current_user: User = Depends(require_student),
                    db: Session = Depends(get_db)):
    # 1. Device binding: first device is bound, any other device is blocked
    if not student_profile.bound_device_uuid:
        student_profile.bound_device_uuid = payload.device_uuid
    elif student_profile.bound_device_uuid != payload.device_uuid:
        raise HTTPException(403, "Proxy Attendance Blocked: ...")
    # 2. Geofence check (mock GPS, accuracy, distance)
    geo_result = verify_campus_presence(payload.latitude, payload.longitude, ...)
    if not geo_result["valid"]:
        raise HTTPException(403, geo_result["message"])
    # 3. Teacher's active session must match the PIN and the student's section
    active_session = db.query(ActiveClassSession).filter(...).first()
    if not active_session:
        raise HTTPException(400, "Invalid session code / no active session ...")
    # 4. Duplicate check for the same subject, date and period
    if duplicate:
        raise HTTPException(400, "Attendance already marked ...")
    # 5. Face biometric verification
    face_result = verify_student_face(usn, payload.image_base64,
```

```
                                                    student_profile.face_image_path)
        if not face_result["verified"]:
            raise HTTPException(400, face_result["message"])
        # 6. Save the record and write an audit log
        db.add(AttendanceRecord(status="PRESENT", ...))
        db.add(SecurityAuditLog(event_type="ATTENDANCE_SUCCESS", ...))
        db.commit()
```

6.2.9 Front-end check-in action (frontend/src/components/StudentPortal.jsx)

Validates the 6-digit PIN, captures a selfie frame from the live camera, sends the image with GPS data to the server and shows confetti and vibration on success.

```
const handleMarkAttendance = async () => {
  if (sessionCodeInput.trim().length < 6) {
    alert('The session code must be a 6-digit PIN.');
    return;
  }
  const imageBase64 = captureFrameBase64(videoRef);   // canvas -> JPEG base64
  const res = await api.markAttendance({
    latitude: locationState.lat, longitude: locationState.lon,
    imageBase64, sessionCode: sessionCodeInput.trim().toUpperCase() });
  confetti({ particleCount: 90, spread: 60, origin: { y: 0.6 } });
  setMarkResult({ success: true, message: res.message, record: res.record });
};
```

6.2.10 Multi-tenant database layer (backend/rgit_app/database.py)

A master database and nine department databases are created, one SQLite file per department. Each connection runs in WAL mode with a 30-second busy timeout.

```
DEPARTMENT_DB_REGISTRY = {   # one SQLite file per department (nine in all)
    "EE":  {"name": "B.E in Electrical Engineering", "filename": "attendance_EE.db"},
    "CSE": {"name": "B.E in Computer Science and Engineering",
              "filename": "attendance_CSE.db"},   # ... ECE, BME, ME, ISE, AIML
}
def get_department_session(dept_name_or_code: str):
    key = resolve_department_key(dept_name_or_code)
    return DEPARTMENT_SESSIONS.get(key, MasterSessionLocal)()
```

6.2.11 Request schemas (backend/rgit_app/schemas.py)

Pydantic models validate every incoming request automatically. The geofence radius is limited to 50-5000 m.

```
class MarkAttendanceRequest(BaseModel):
    latitude: float
    longitude: float
    accuracy_meters: Optional[float] = None
    is_mock_location: Optional[bool] = False
    device_uuid: Optional[str] = None
    image_base64: str   # captured face image
    session_code: Optional[str] = None


class GeofenceUpdateRequest(BaseModel):
    radius_meters: float = Field(..., ge=50, le=5000)
    enforce_geofence: bool
```

6.2.12 Front-end API layer (frontend/src/services/api.js)

Generates a persistent device ID for anti-proxy binding and attaches the bearer token to every request.

```
export const getDeviceUUID = () => {
  let uuid = localStorage.getItem('rgit_device_uuid');
  if (!uuid) {
    uuid = 'DEV_' + Math.random().toString(36).substring(2, 10).toUpperCase()
          + '_' + Date.now().toString(36).toUpperCase();
    localStorage.setItem('rgit_device_uuid', uuid);
  }
  return uuid;
};
```

6.2.13 Face enrolment endpoint (backend/rgit_app/main.py)

A student may enrol only their own face. The photograph is validated by the vision engine, the profile is marked as registered and the event is logged.

```
@app.post("/api/students/register-face")
def enroll_student_face(payload: RegisterFaceRequest, ...):
    if current_user.role == "student" and \
```

```
            current_user.username.upper() != payload.usn.strip().upper():
                raise HTTPException(403, "Students can only enroll their own facial
                biometric")
        reg_result = register_student_face(payload.usn, payload.image_base64)
        if not reg_result["success"]:
            raise HTTPException(400, reg_result["message"])
        student_profile.face_registered = True
        student_profile.face_image_path = reg_result["file_path"]
        db.add(SecurityAuditLog(event_type="FACE_ENROLLED", ...))
        db.commit()
```

6.2.14 Defaulters calculation (backend/rgit_app/main.py)

Counts the distinct classes conducted for the section and calculates each student's attendance percentage. Students below the threshold are listed for the teacher and the HOD.

```
    total_conducted = max(sess_query.distinct().count(), 1)   # distinct classes held
    for s in students:
        attended = db.query(AttendanceRecord).filter(
            AttendanceRecord.student_id == s.id).count()
        pct = min(100.0, round((attended / total_conducted) * 100.0, 1))
        defaulters.append({"usn": s.usn, "attendance_percentage": pct, ...})
```

6.3 System Implementation Summary

This chapter presented the key source code of the vision engine, geofence engine, timetable resolver, authentication, data models, session launch and the attendance-marking endpoint, together with the multi-tenant database layer, schemas, login, enrolment, defaulter logic, QR scanner and Android packaging. A check-in is accepted only when the device, location, teacher PIN and face all pass verification.

TESTING

CHAPTER 7

TESTING

7.1 Validation and System Testing

7.1.1 Software Testing

Software testing verifies that every module behaves as specified and that the modules work together. The testing of this project was organised at three levels:

- **Unit testing** of the Haversine distance, geofence decisions, period resolution, EAR calculation and password/JWT functions.
- **Integration testing** of the API flows: login, device binding, session launch, attendance marking guards, defaulter computation, dashboard statistics and audit logs, using the automated test script (backend/test_suite.py) which drives the FastAPI application through its test client.
- **System testing** of the complete workflow through the web portals: registration, approval, session launch, check-in from a phone and reporting.

7.1.2 Validation

Validation checks that the system meets the needs of its users. The portals were exercised for each role using the sample CSE Semester 7 data (students, faculty, subjects and timetable) to confirm that attendance from check-in is visible in the student dashboard, teacher roster, HOD dashboard and audit ledger.

7.1.3 Reasons for performing software validation

- To confirm that the proxy-prevention checks cannot be bypassed through the API.
- To confirm that percentages and defaulter flags are computed correctly for the VTU 75% rule.
- To verify role separation between student, teacher, HOD and administrator.
- To make sure that failures give clear messages and are recorded in the audit log.

7.2 Test Cases

Table 7.1 lists the test cases defined in the automated test suite. Each case has an expected result that the test asserts when the suite is run (python test_suite.py from the backend folder).

Test ID	Test scenario	Expected result
TC-01	Admin / HOD login with valid credentials	HTTP 200 and access token
TC-02	Faculty login with valid credentials	HTTP 200 and access token
TC-03	Student login with device UUID	HTTP 200; device bound on first use
TC-04	Login with wrong password	HTTP 401 Unauthorized
TC-05	Geofence status at campus centre	is_inside = true
TC-06	Geofence status about 700 m away	is_inside = false
TC-07	Mock GPS flag set	Rejected with mock-location message
TC-08	Current timetable period	Period information returned for current time
TC-09	Faculty launches session (BCS701, Sec A, Period 1, 15 min)	Session created with six-digit code
TC-10	Query active session for Section A	Active session with the same code returned
TC-11	Defaulter computation for Section A	Percentages calculated; students < 75% flagged
TC-12	Student roster for Section A	Roster returned
TC-13	Dashboard statistics and audit logs	Totals and geofence radius returned; audit records listed
TC-14	Attendance request from outside campus	HTTP 403 Forbidden (geofence)
TC-15	Attendance request from an unbound device	HTTP 403 Forbidden (device mismatch)
TC-16	Attendance request without session code	HTTP 400 "Faculty session code is required"
TC-17	Attendance request with wrong session code	HTTP 400 invalid session code

Table 7.1 Test Cases of the Automated Test Suite

7.3 Operating Parameters and Evaluation Metrics

The behaviour of the system is controlled by the parameters in Table 7.2. The recognition quality of the vision engine is evaluated with the metrics in Table 7.3; their values depend on the enrolled population and capture conditions and are to be recorded by repeated trials in the target classrooms.

Parameter	Value in the implementation
Face distance tolerance	0.55 (Euclidean distance between 128-d embeddings)
Liveness EAR threshold	0.22 (consecutive frames parameter: 2)
Geofence centre	Latitude 13.033090, Longitude 77.597786
Geofence radius	250 m (configurable by master admin)
Maximum GPS accuracy accepted	65 m
Session PIN	6 digits; default validity 15 minutes
Token validity	7 days (JWT HS256)
Multi-frame confirmation (scanner UI)	5 frames

Table 7.2 Operating Parameters of the System

Metric	Definition
Recognition accuracy	Correct accept and correct reject decisions / total decisions
False Acceptance Rate (FAR)	Impostor attempts accepted / total impostor attempts
False Rejection Rate (FRR)	Genuine attempts rejected / total genuine attempts
Verification latency	Time from request received to response sent
Geofence decision precision	Correct inside/outside decisions at boundary test points

Table 7.3 Evaluation Metrics for the Vision and Geofence Modules

7.4 Testing Summary

This chapter described the testing strategy, the automated test cases for authentication, geofence, session, device and PIN guards, and the parameters and metrics used to evaluate the system. The guards that prevent proxy attendance are covered by explicit negative test cases.

Observations from the Test Cases

The negative cases TC-14 to TC-17 confirm that each guard rejects an invalid request: a request from outside the campus or from an unbound device gets HTTP 403, and a missing or wrong session code gets HTTP 400. The positive cases TC-01 to TC-13 confirm login, geofence, timetable, session and report functions. Repeated trials with a larger classroom population and different lighting are recommended to record the metrics of Table 7.3.

SAMPLE OUTPUT

CHAPTER 8

SAMPLE OUTPUT

8.1 Snapshots

The following screens show the working of the system for each role; every screen is shown once. The sample data belongs to the CSE Semester 7 Section A class of RGIT.

8.1.1 Login portals

Student login with the RGIT banner, the nine-branch department selector, semester buttons and the 10-character VTU USN builder. A shortcut leads new students to the registration form.

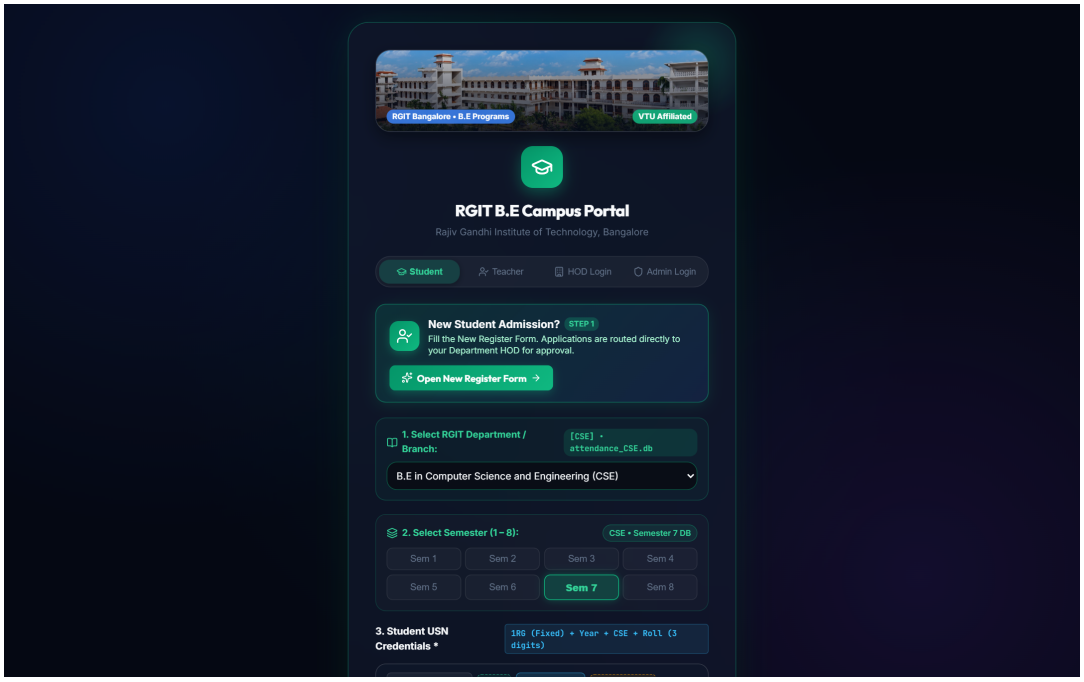


Figure 8.1 Student Portal Login Screen

Details of the Student Login Screen

After the department and semester are selected, the USN builder assembles the ten-character VTU seat number of the student, for example 1RG23CS027: the college code, the year of joining, the branch code and the serial number. On the first login the device UUID is bound to the student, and attendance can afterwards be marked only from that device. New students can reach the registration form through the shortcut on the same screen.

Teacher login for the selected department and semester, with quick-select faculty cards and the password assigned by the HOD.

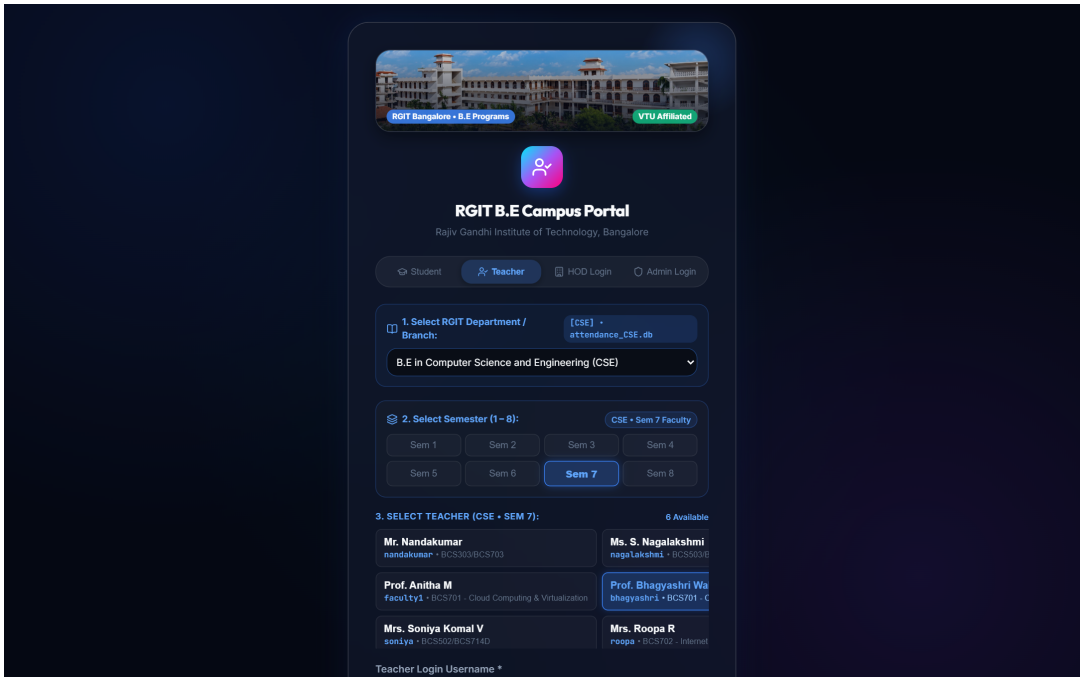


Figure 8.2 Teacher Login Portal

Head-of-Department login showing the appointed HOD of the selected branch; the password is assigned by the master administrator.

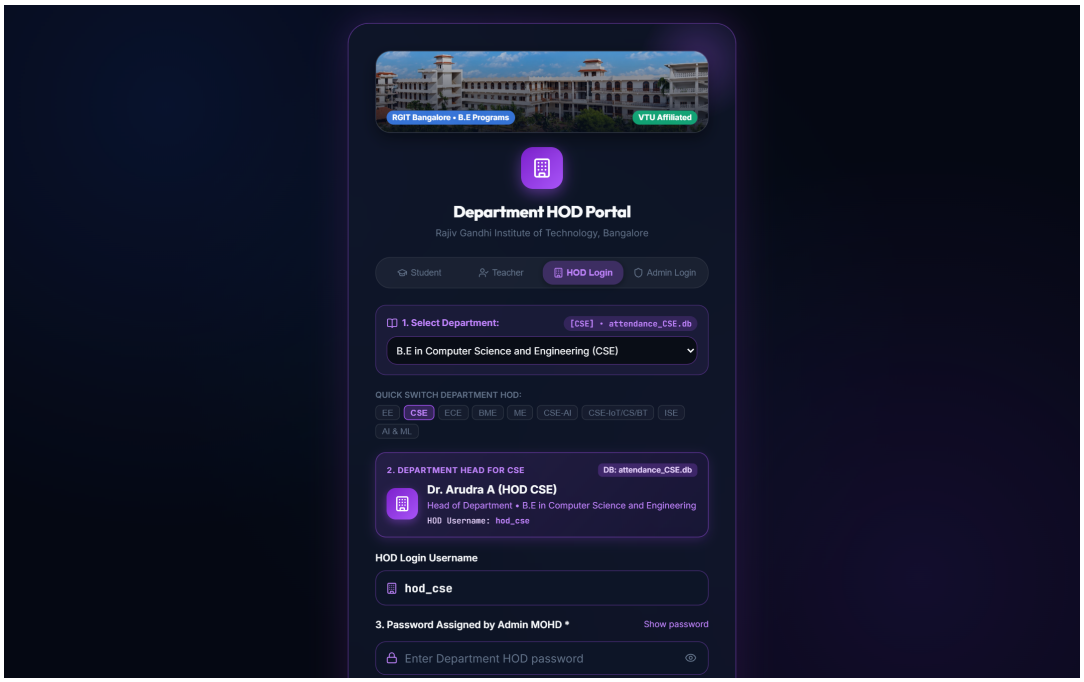


Figure 8.3 HOD Login Portal

Root-privilege login for the master system administrator who governs all nine B.E. departments.

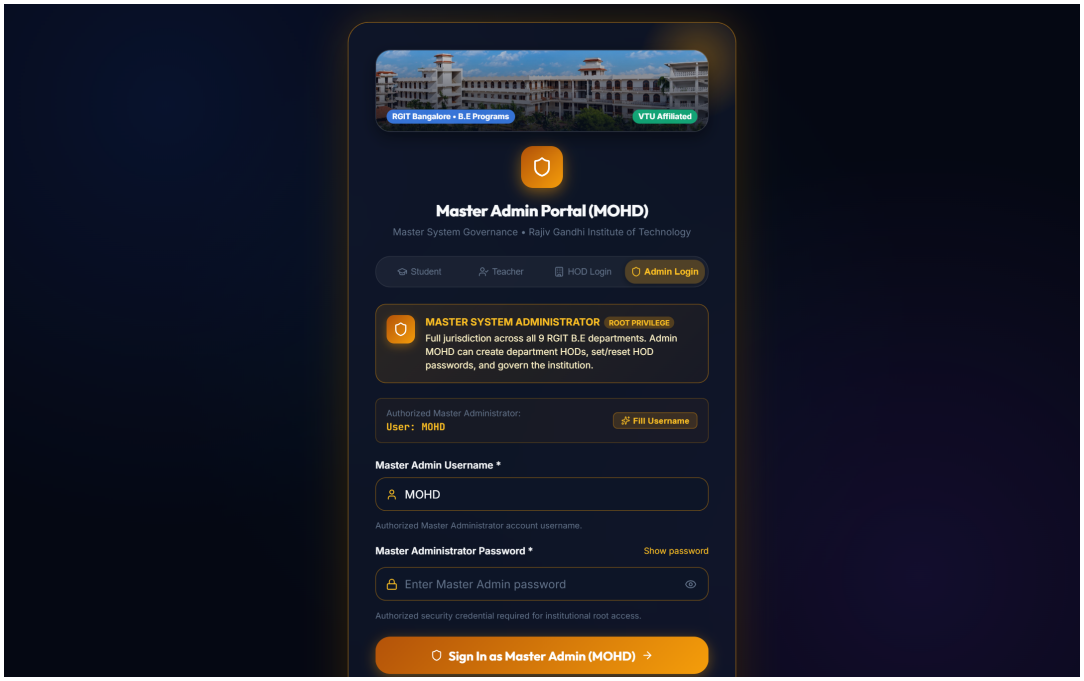


Figure 8.4 Master Admin (MOHD) Login Portal

8.1.2 Student self-registration

Self-registration form: department and semester selection, the USN builder and personal details, followed by camera capture of the face photograph.

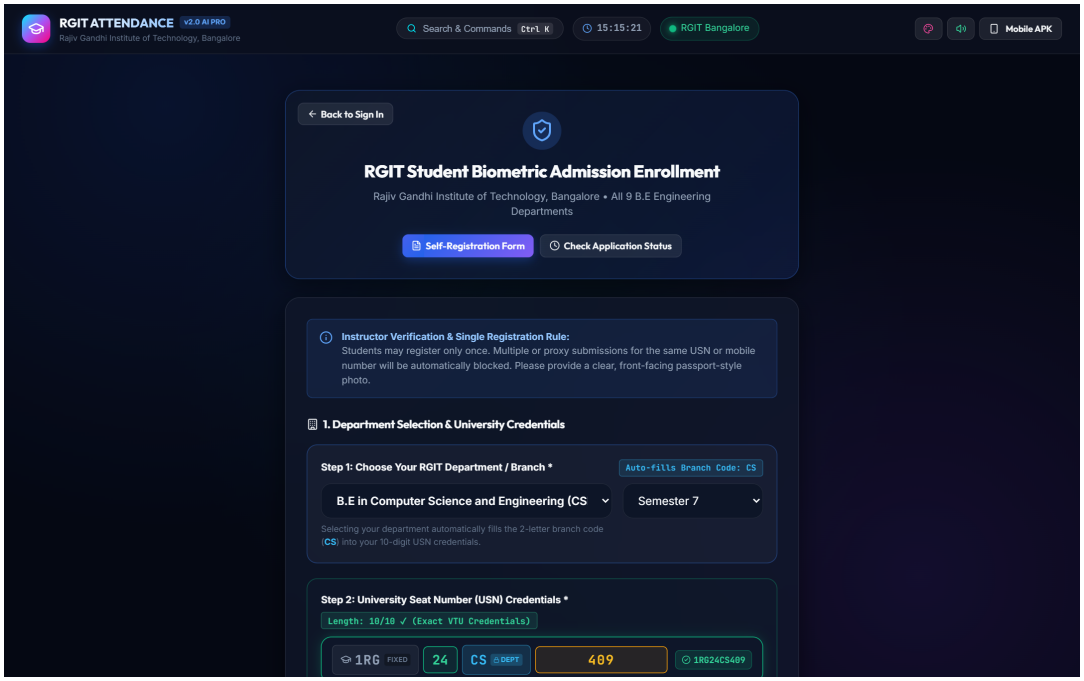


Figure 8.5 Student Biometric Admission Enrollment Form

The student enters the USN to check whether the application is pending, approved or rejected, and sees the approving faculty.

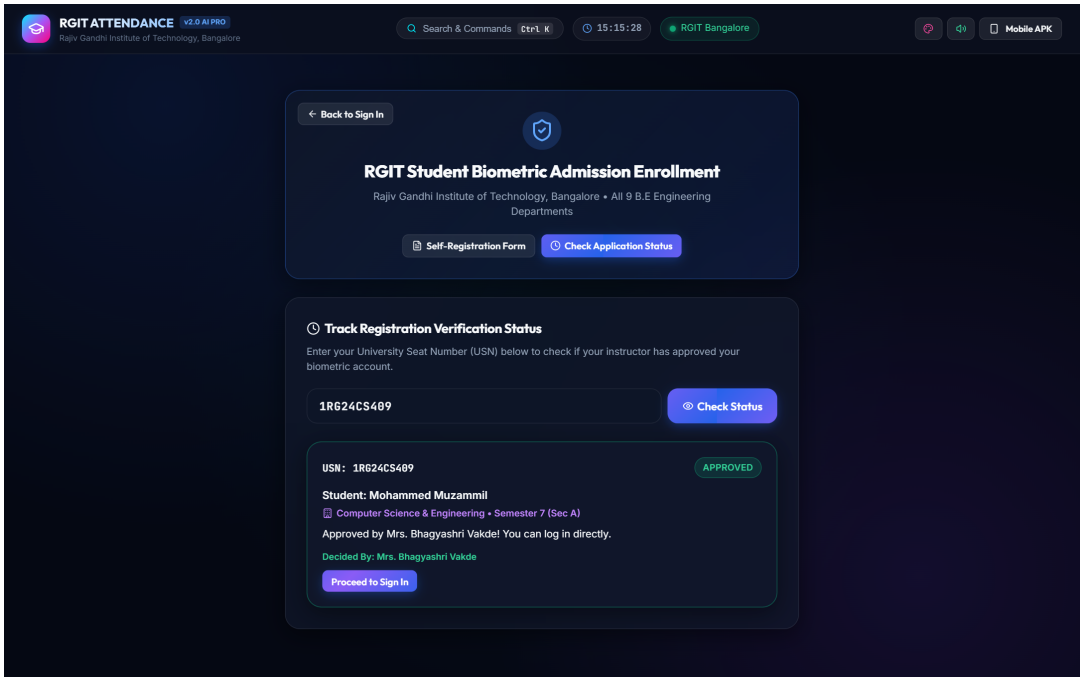


Figure 8.6 Registration Status Tracker

8.1.3 Student portal

Dashboard with profile and device-lock badge, VTU statutory attendance gauge, next scheduled class, AI attendance forecaster, campus radar and subject-wise attendance.

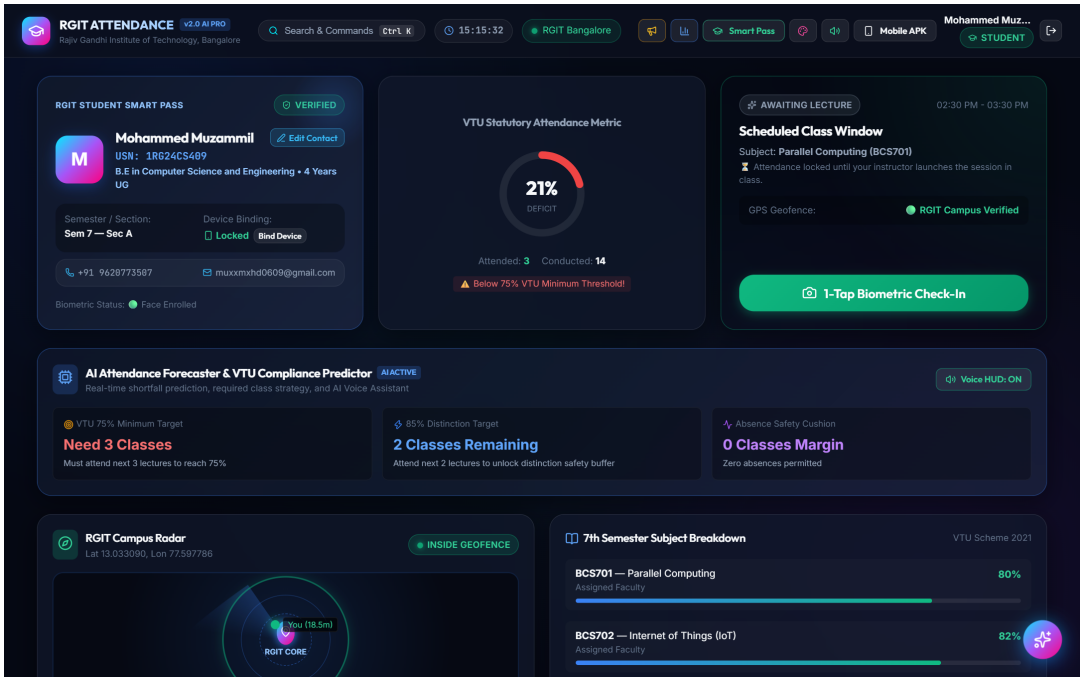


Figure 8.7 Student Main Dashboard

Student dashboard during a scheduled class window. The two ways of marking attendance are offered side by side: entering the six-digit session PIN or scanning the faculty QR code.

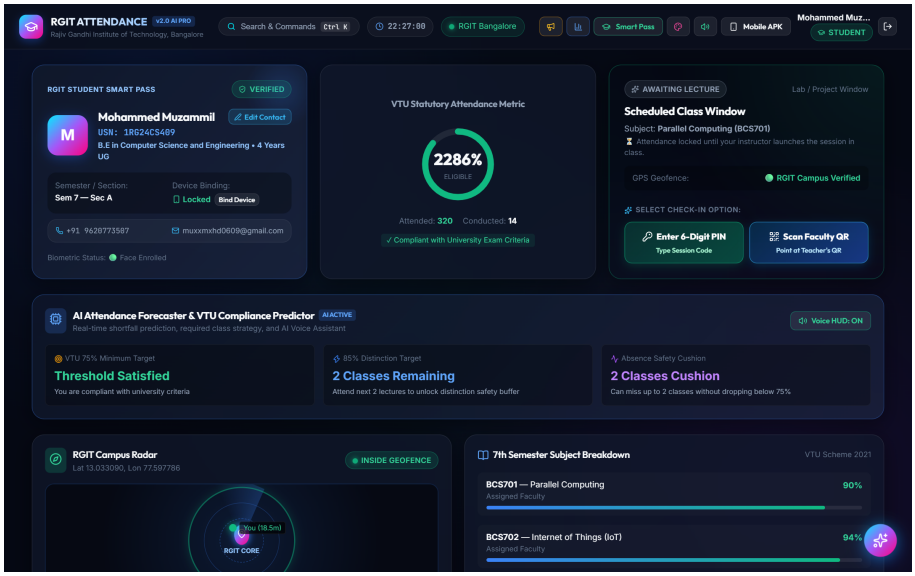


Figure 8.8 Student Dashboard with Check-in Options

Scanner window opened by the student to read the faculty QR code shown in the classroom. A saved image of the QR code can also be uploaded when the camera is not available.

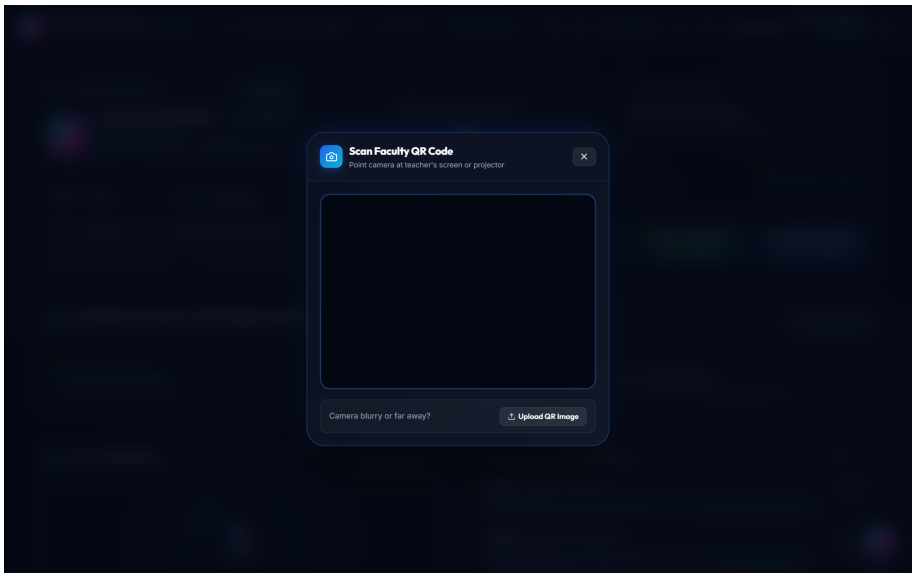


Figure 8.9 Student QR Scanner Modal

Scanner modal showing the geofence status (distance from campus centre against the allowed 250 m), the six-digit session PIN field, subject selection and the face-alignment frame.

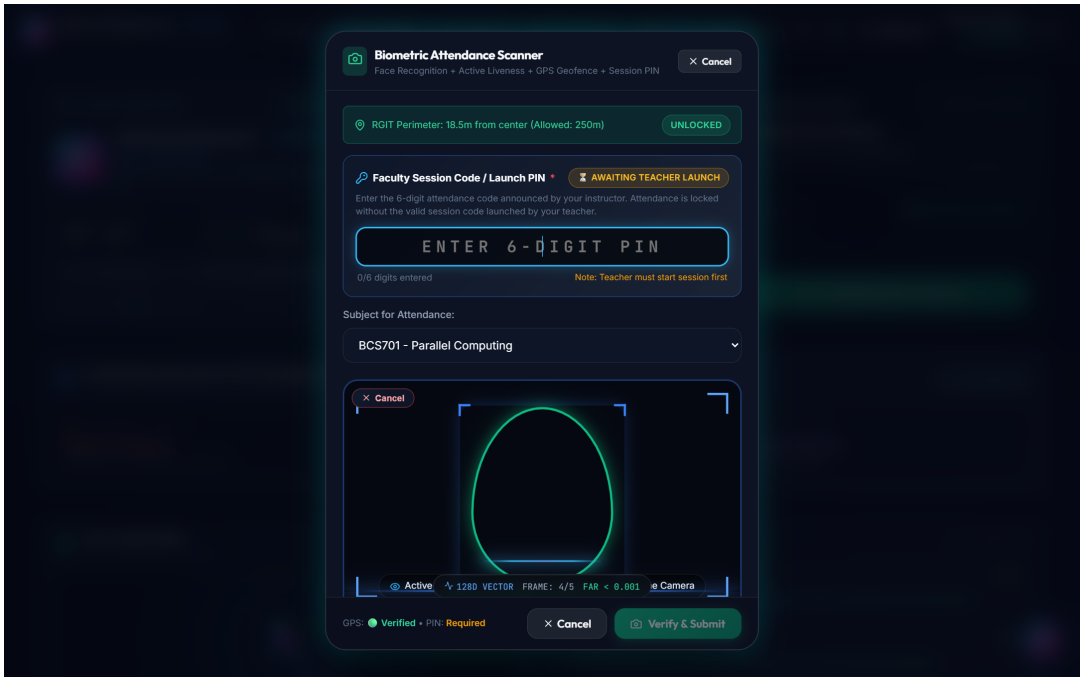


Figure 8.10 Biometric Attendance Scanner with Session PIN

Biometric attendance scanner after the six-digit session PIN has been typed. The phone camera is then used to capture the face before the attendance is submitted.

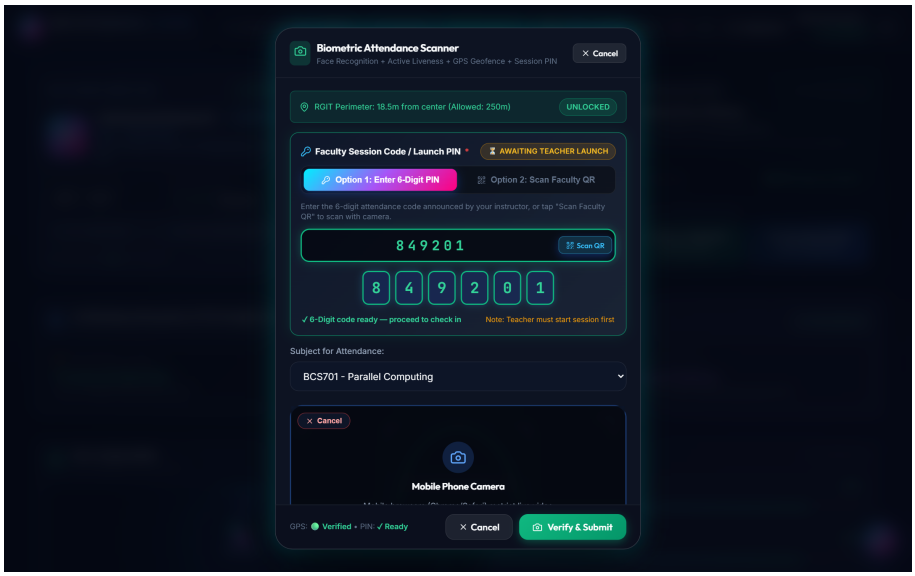


Figure 8.11 Session PIN Entry with Biometric Capture

Scanner after the QR code has been read. The session PIN is picked up automatically from the faculty link and filled in, so the student only has to verify the face.

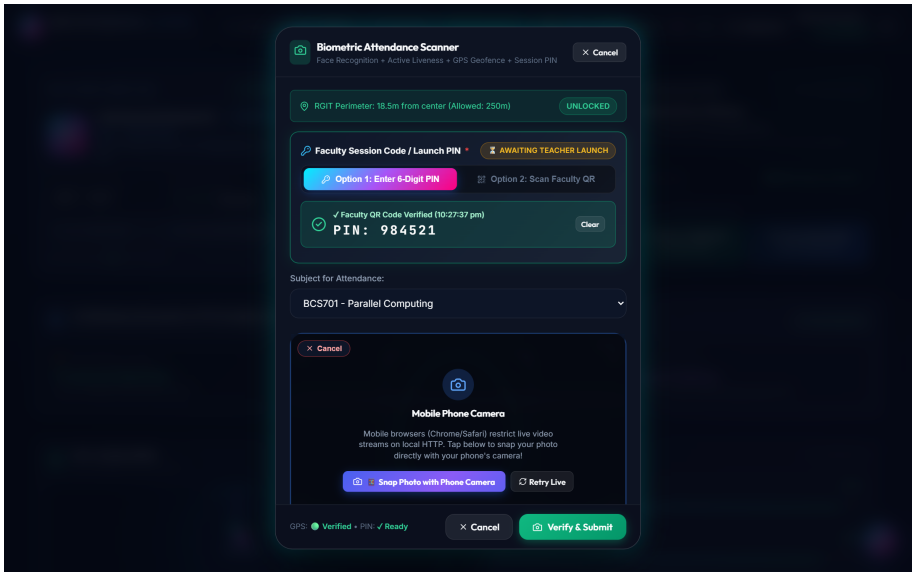


Figure 8.12 PIN Auto-detected from the QR Link

Alternative capture mode in which the student takes a selfie with the phone camera before Verify and Submit.

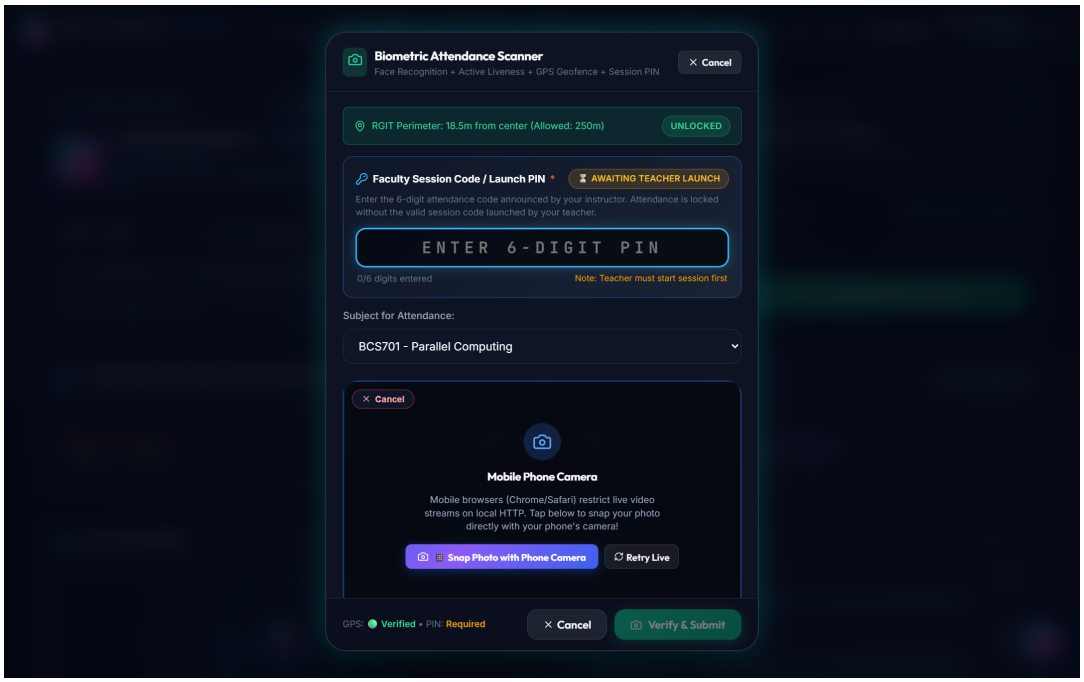


Figure 8.13 Scanner with Phone-Camera Capture

Students may update only their mobile number and e-mail; USN, name and branch are locked and controlled by the HOD.

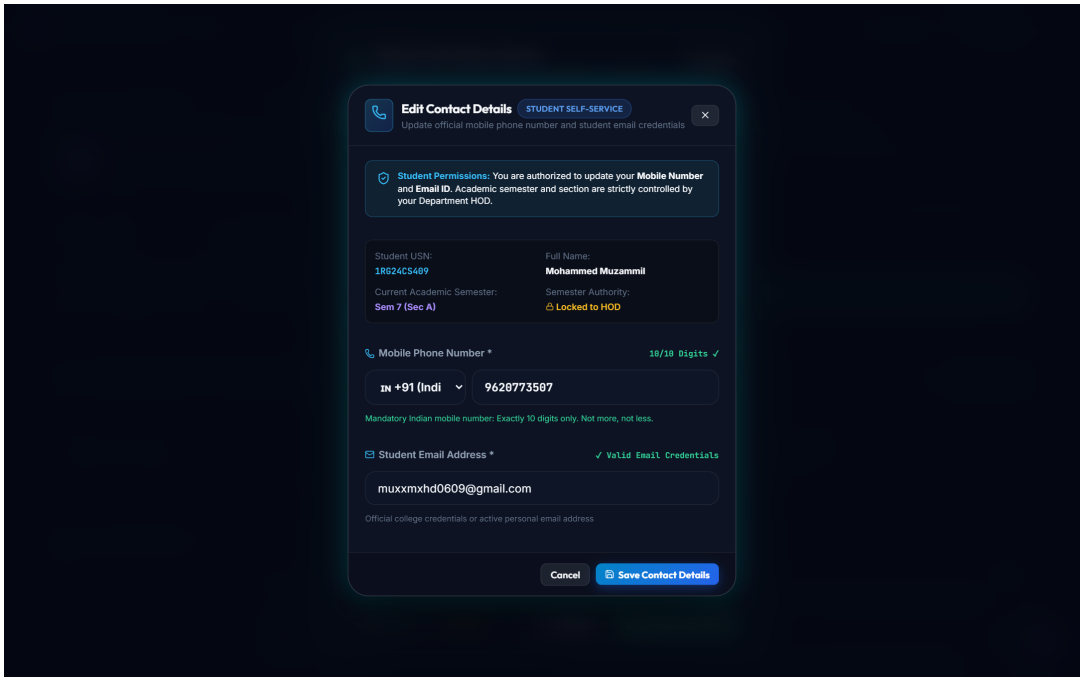


Figure 8.14 Student Contact Edit Modal

8.1.4 Teacher portal

Class strength, present and absent counts, attendance rate and the day's teaching schedule for the selected semester and section.

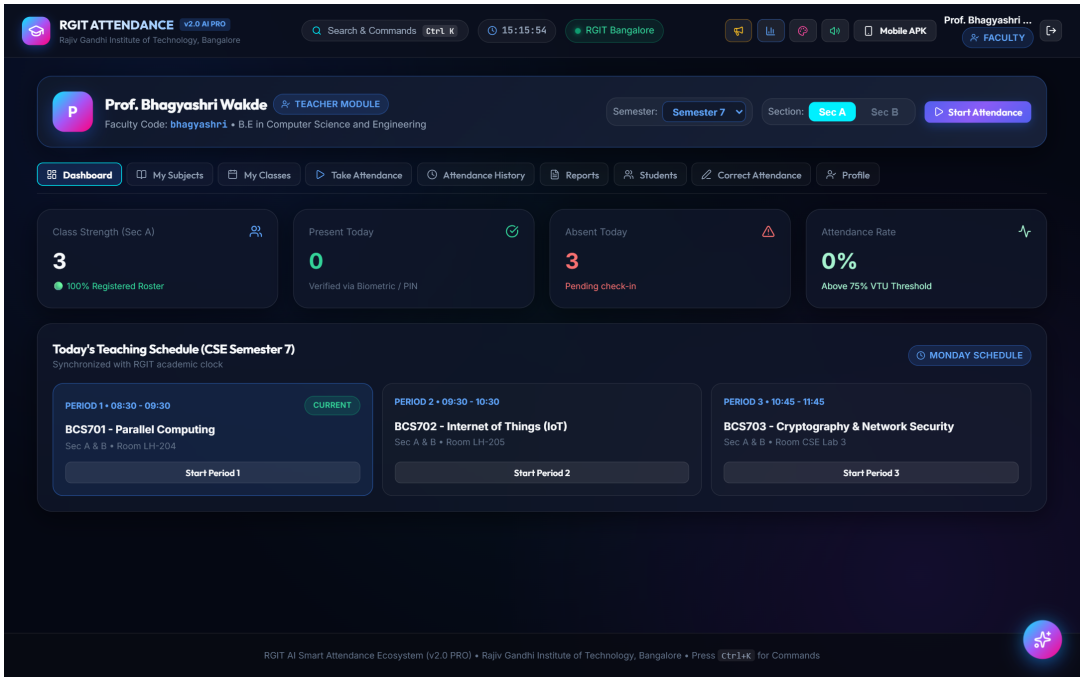


Figure 8.15 Teacher Dashboard

Subjects assigned by the HOD (BCS701, BCS702, BCS703) with credits, average attendance and a start-class button.

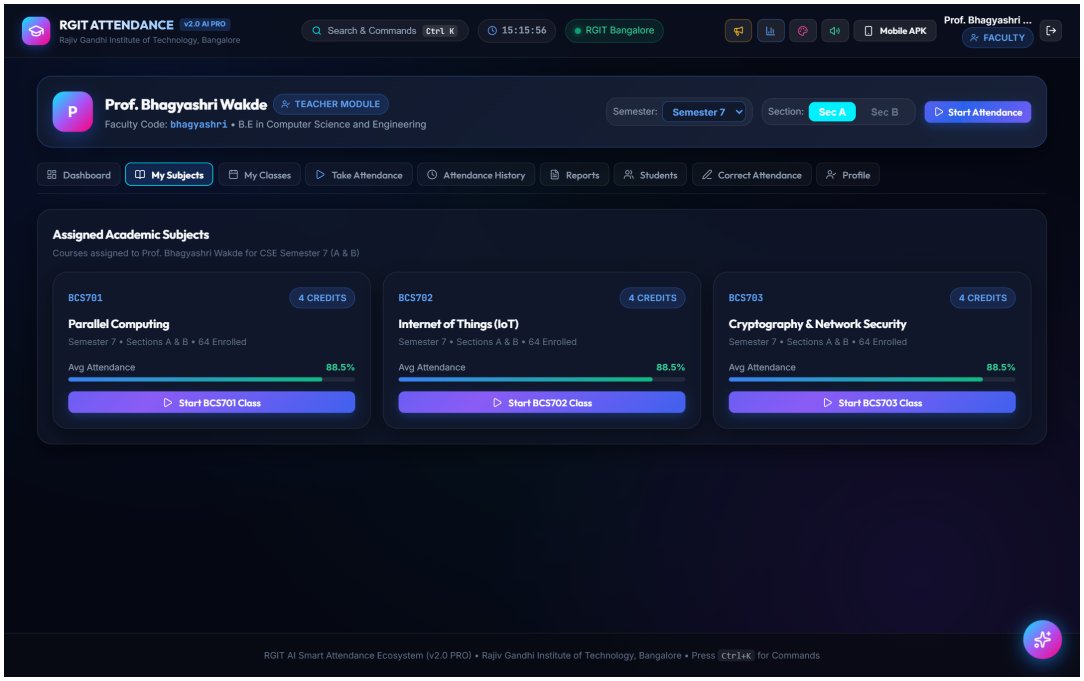


Figure 8.16 Teacher Assigned Subjects

The teacher selects subject, section and period and launches the session; a dynamic classroom QR code and the live check-in stream appear.

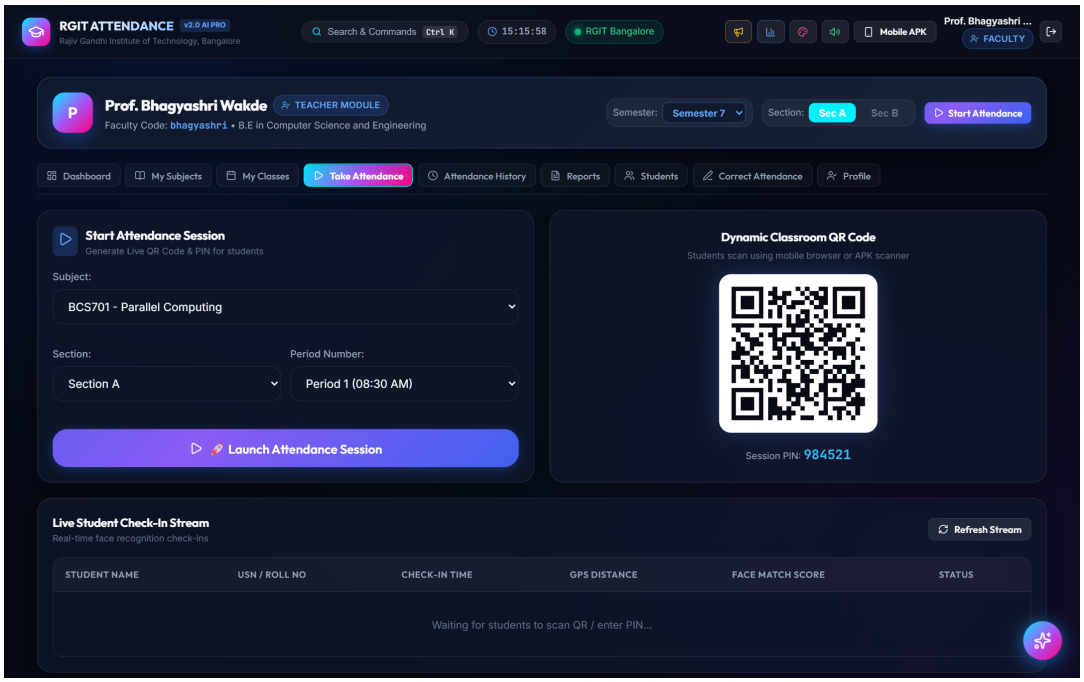


Figure 8.17 Launching an Attendance Session with Dynamic QR Code

Projector view of the session QR code and the six-digit PIN that students must enter.

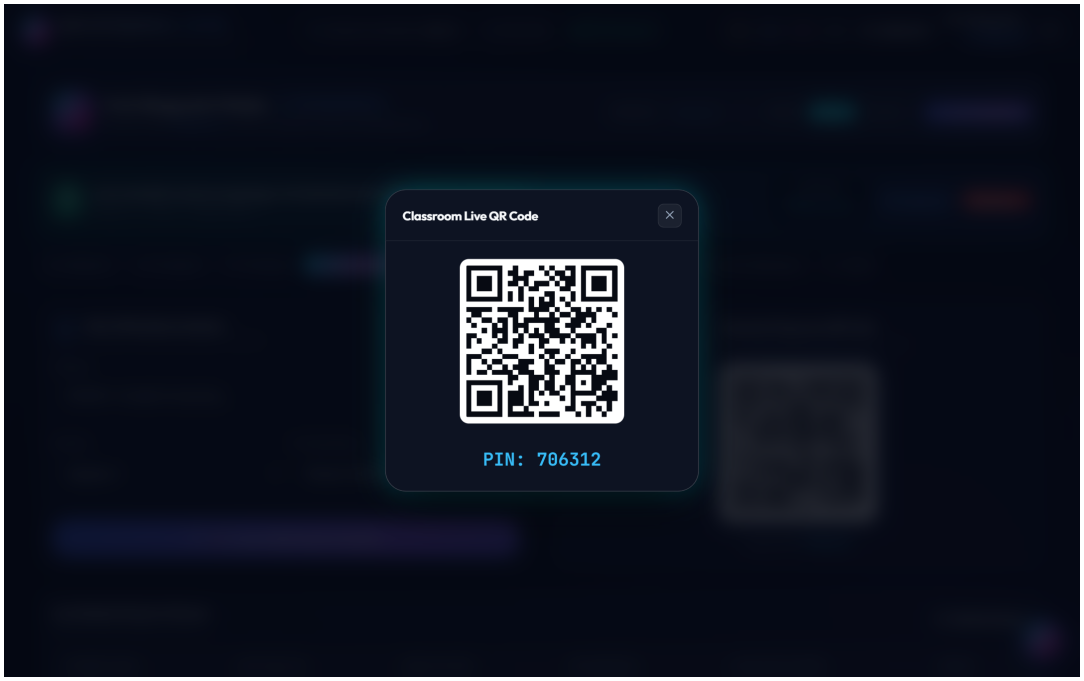


Figure 8.18 Classroom Live QR Code and Session PIN

Take Attendance tab of the teacher after launching a session. The live QR code and the six-digit PIN are shown together with the stream of students who have checked in.

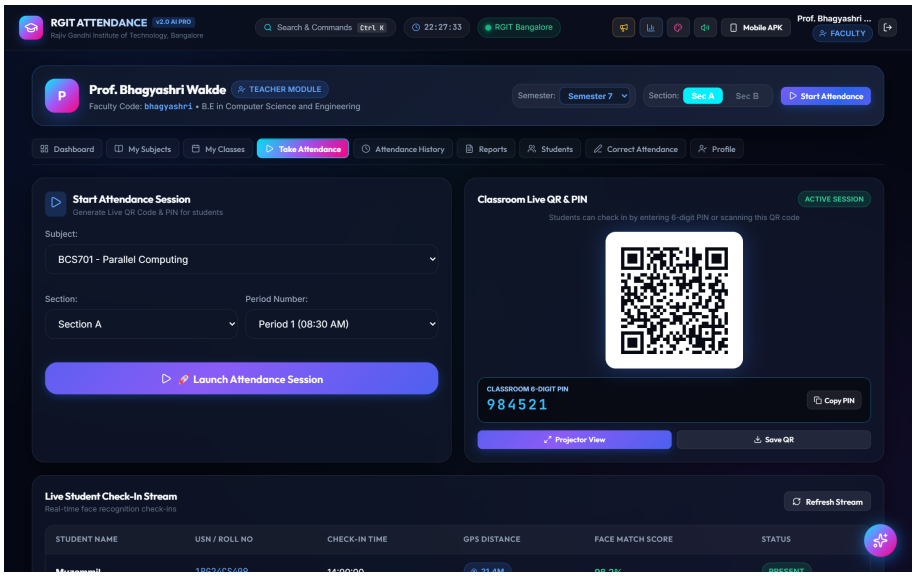


Figure 8.19 Teacher Session with Live QR Code and PIN

Full-screen classroom view meant for the projector. The large QR code and PIN let every student in the room join the session, and the window can be copied or closed from the buttons below.

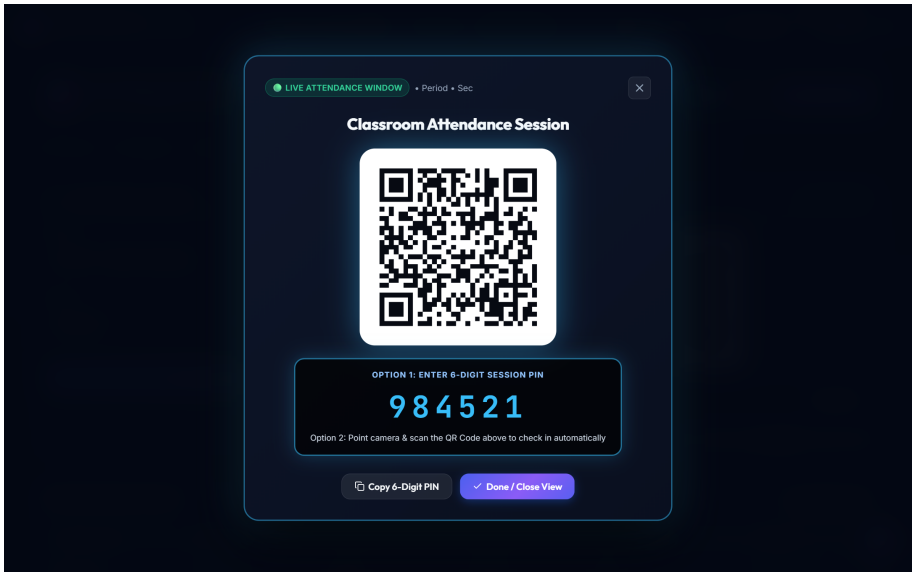


Figure 8.20 Projector Full-screen Session View

Attendance records can be exported as an official Excel sheet, a CSV sheet or a day-to-day register. Official Excel attendance sheet for BCS702, Section A (October 2026), with the daily marks of every student, the totals of present, absent and late classes and the attendance percentage.

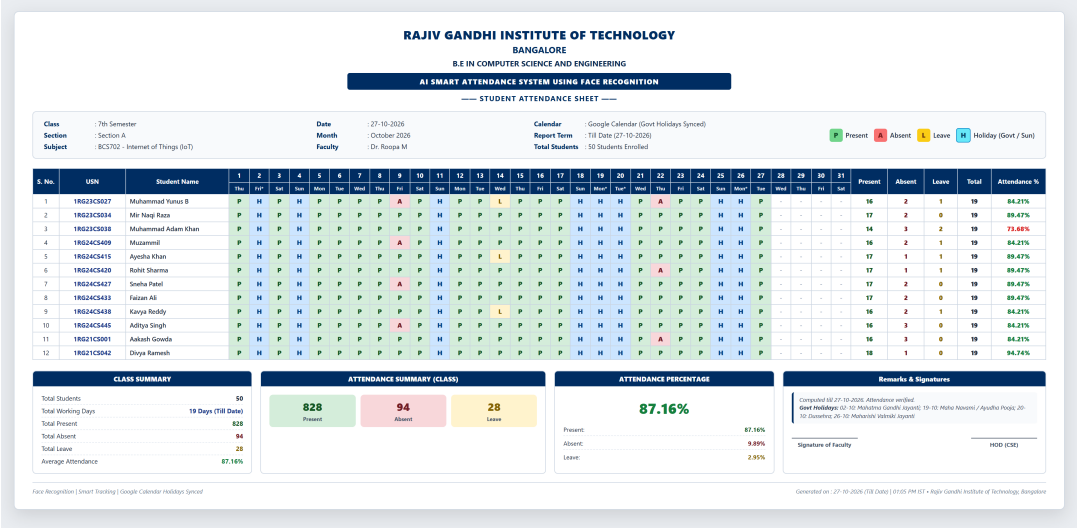


Figure 8.21 Excel Attendance Sheet

On-screen preview of the monthly attendance sheet in CSV form, showing the report header and the date-wise marks for each student before the file is downloaded.

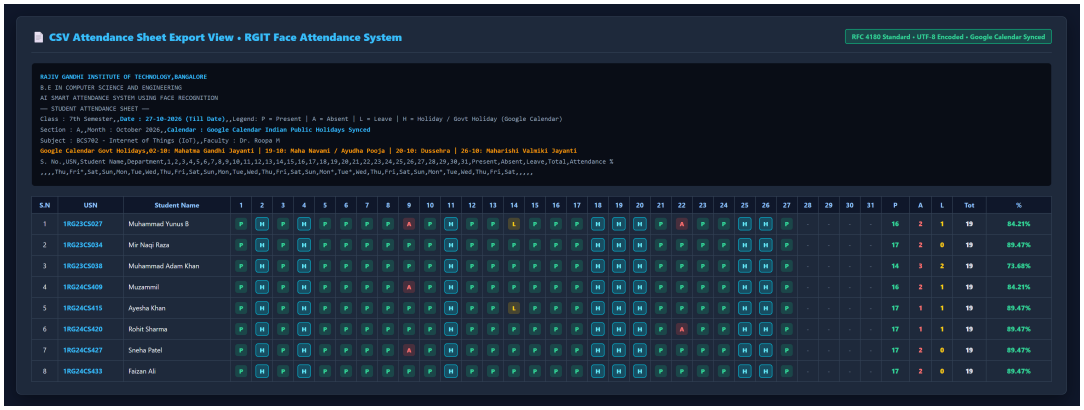


Figure 8.22 CSV Attendance Sheet Preview

Day-to-day attendance register for one session, listing the time, subject, student, status, verification method and audit note for every record.

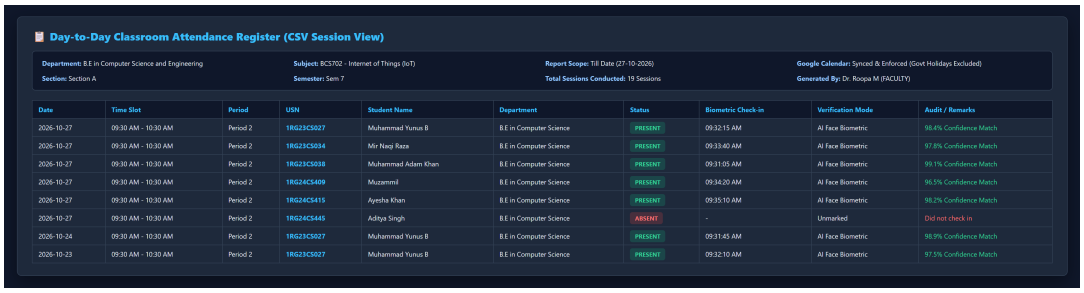


Figure 8.23 Day-to-Day Attendance Register

Export panel of the teacher portal. The subject, month, section and semester are selected and the Excel sheet, the CSV sheet or the live preview can be downloaded.

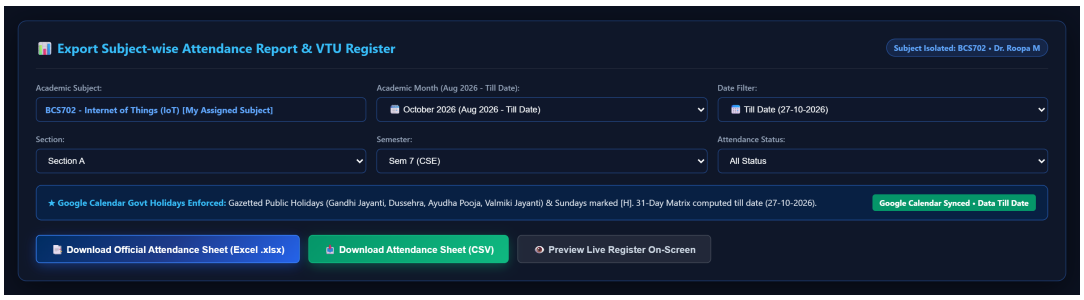


Figure 8.24 Faculty Export Panel

8.1.5 HOD portal

Registered students, biometric enrolment, live classroom sessions and defaulters, with the system-integrity panel and the registered subject catalogue.

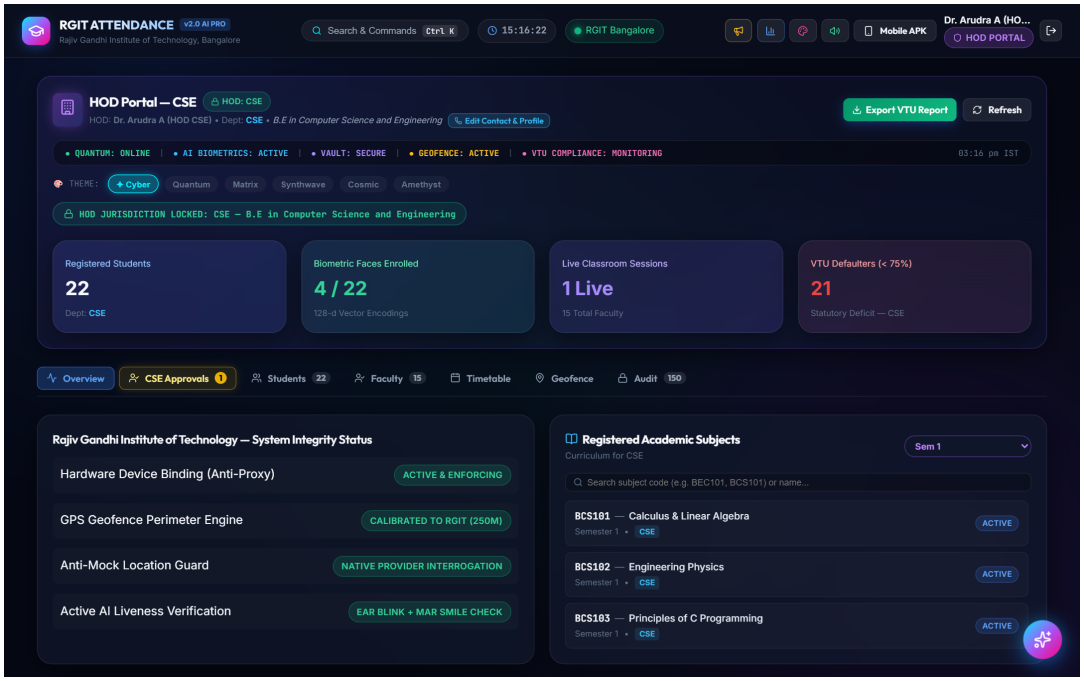


Figure 8.25 HOD Department Overview Dashboard

Searchable student list filtered by semester, section and attendance status, with pending admission alerts and batch approval.

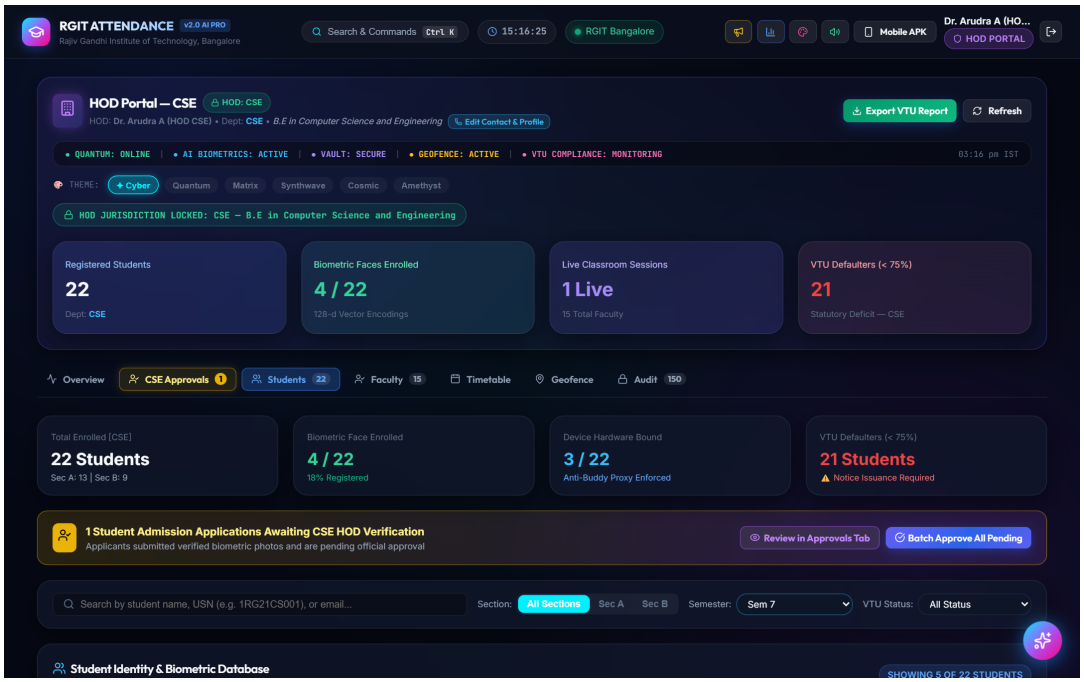


Figure 8.26 HOD Student Directory

The responsive layout of the HOD portal in the mobile frame mode used for the Android application.

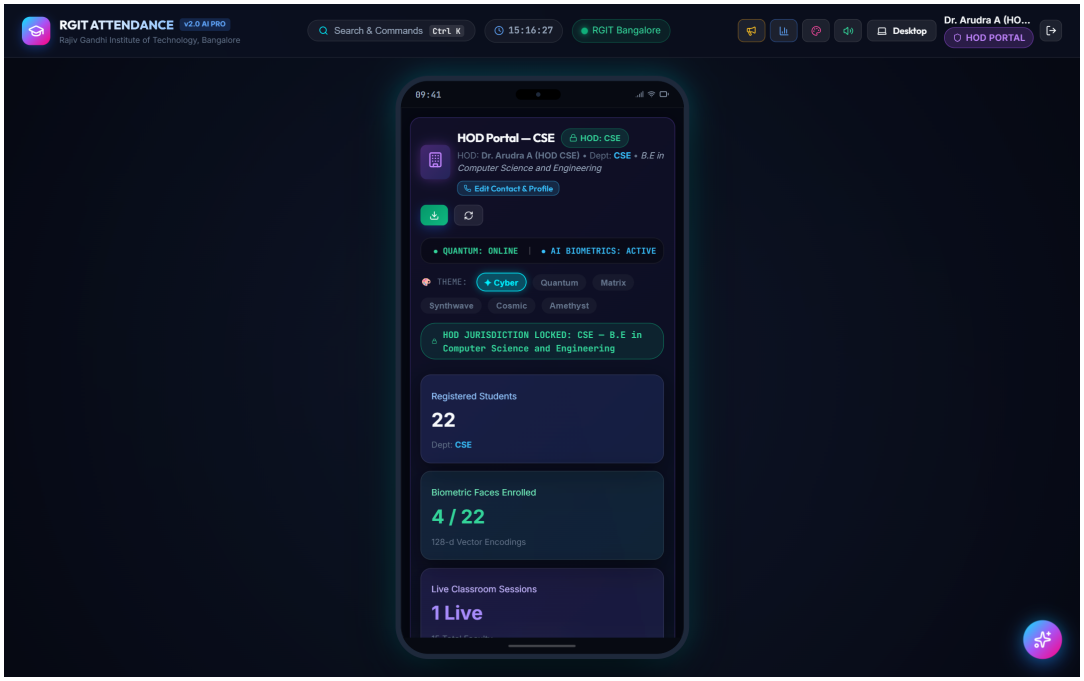


Figure 8.27 HOD Portal in Mobile View

Modal used to update the HOD profile and set a new password issued by the master administrator.

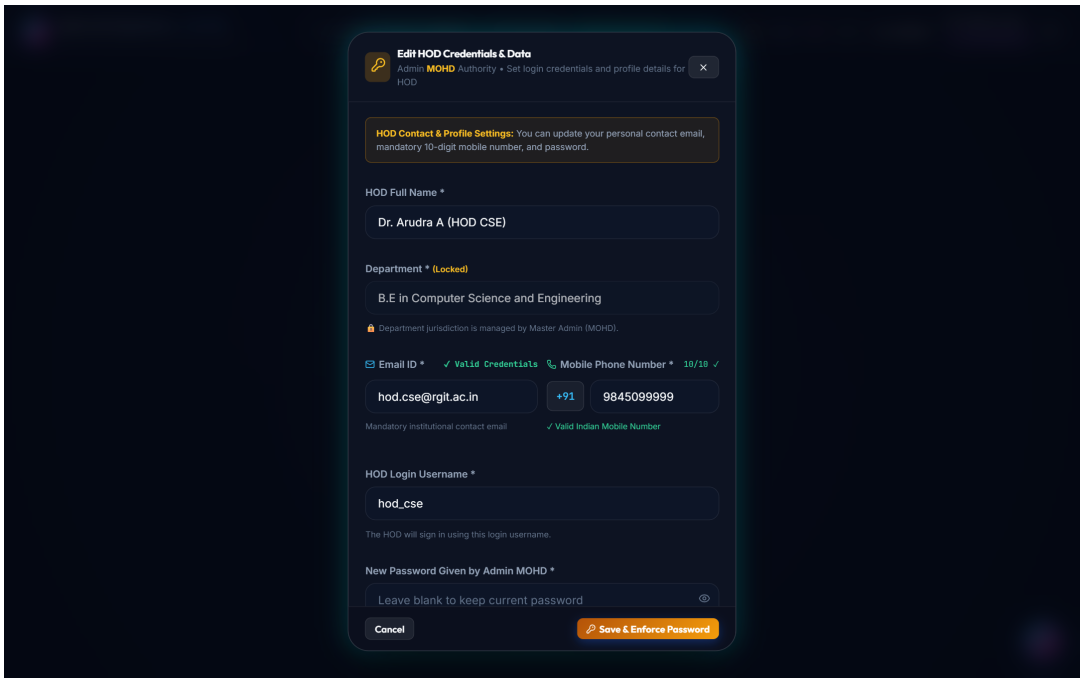


Figure 8.28 HOD Edit Credentials and Profile Modal

When the HOD rejects an admission request, standard reasons such as blurry photo, multiple faces or wrong semester can be chosen or a custom reason written.

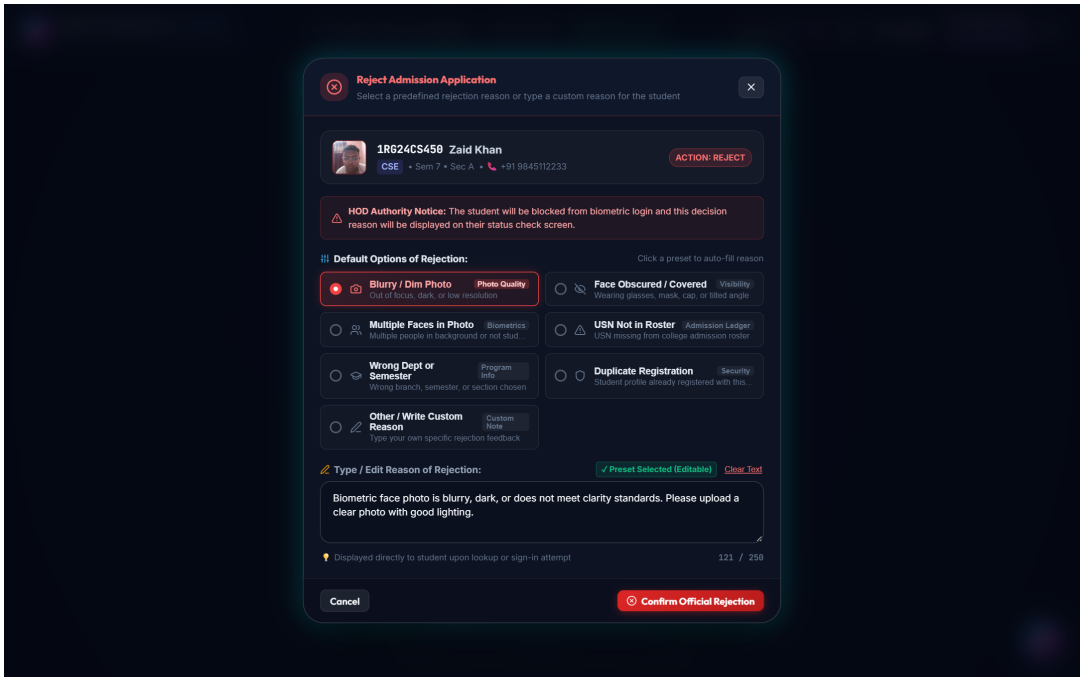


Figure 8.29 Registration Rejection Reasons

Multi-select of accredited subjects to assign to a faculty member.

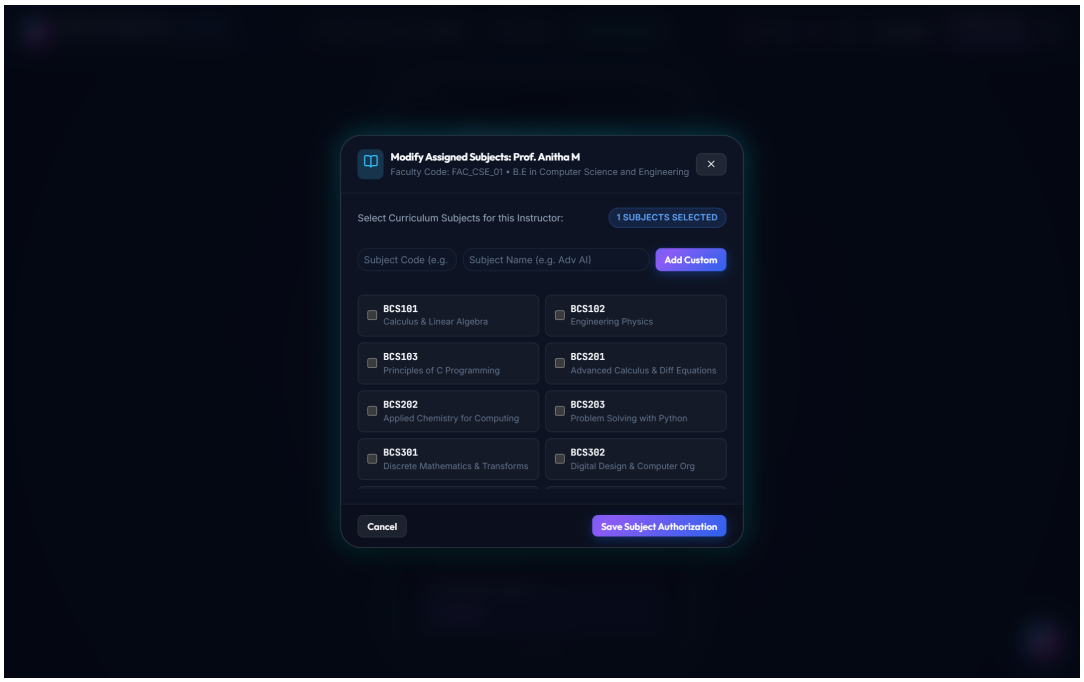


Figure 8.30 Assign Faculty Subjects Modal

Export and control buttons of the institutional governance centre used by the HOD and the master administrator to download department-level reports.

Figure 8.31 Institutional Export Controls

8.1.6 Master administrator portal

Institution-wide view of all departments with registration counts, live sessions, defaulters and the system-integrity status.

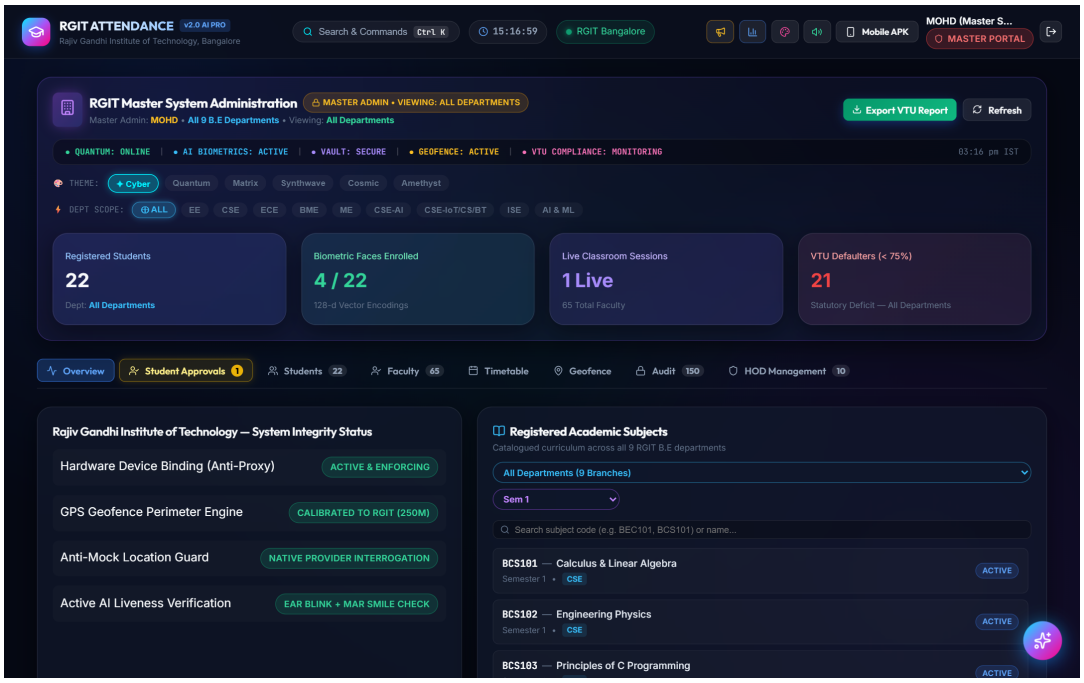


Figure 8.32 Master Admin Dashboard

Directory of Department Heads for all programmes, with options to appoint a new HOD and manage credentials.

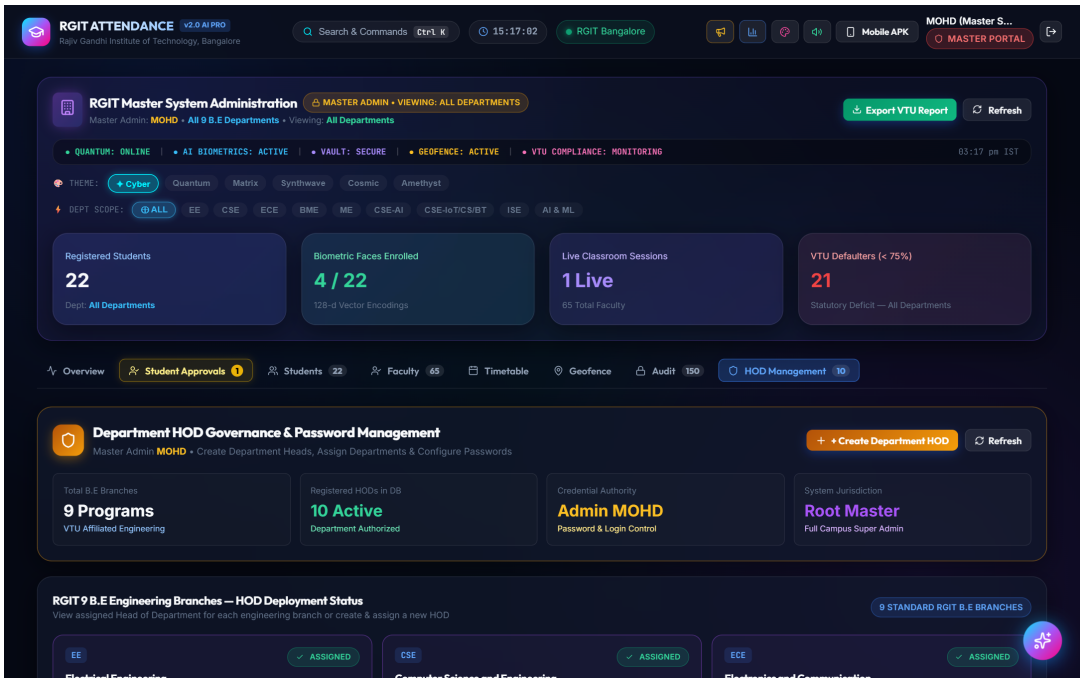


Figure 8.33 HOD Governance Directory

Calibrates the campus geofence radius (250 m) and enforcement setting used by the geofence

engine.

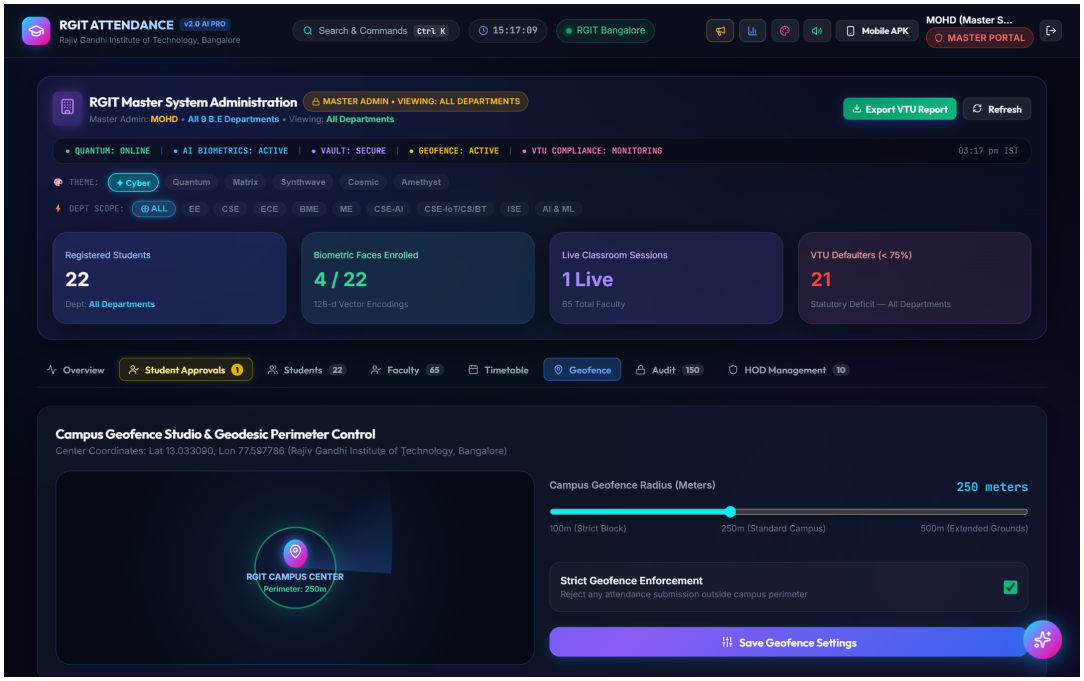


Figure 8.34 Campus Geofence Perimeter Studio

Chronological list of security events such as logins, attendance success and blocked attempts with user, role, time and details.

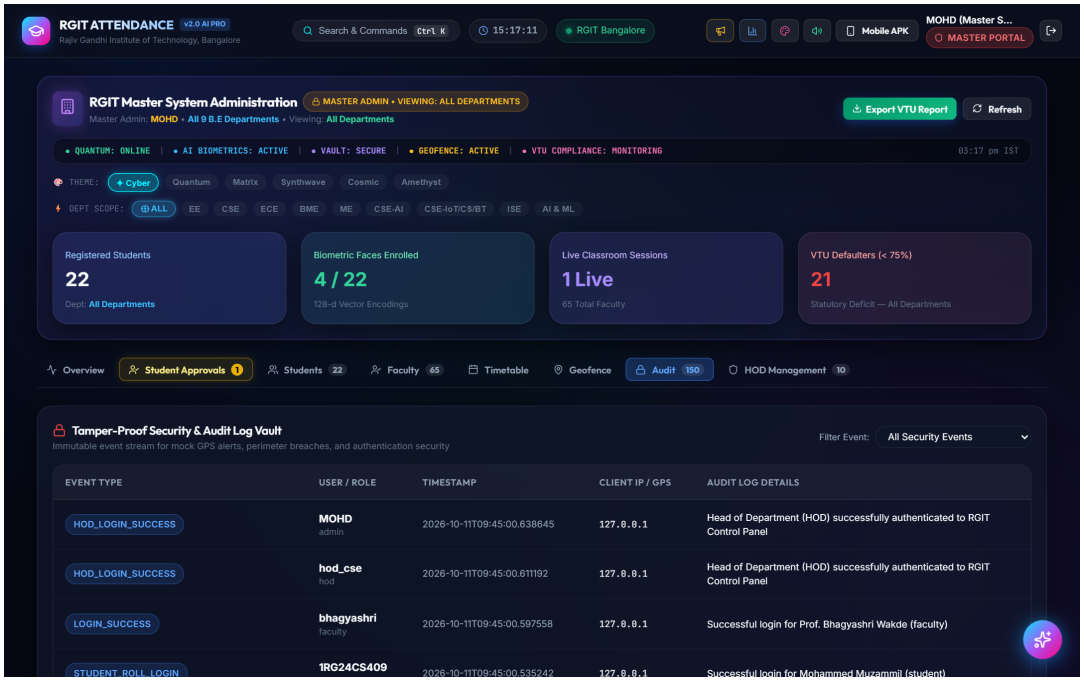


Figure 8.35 Security Audit Ledger

Card showing the synchronisation of public holidays from Google Calendar, so that sessions are not scheduled and attendance is not expected on holidays.

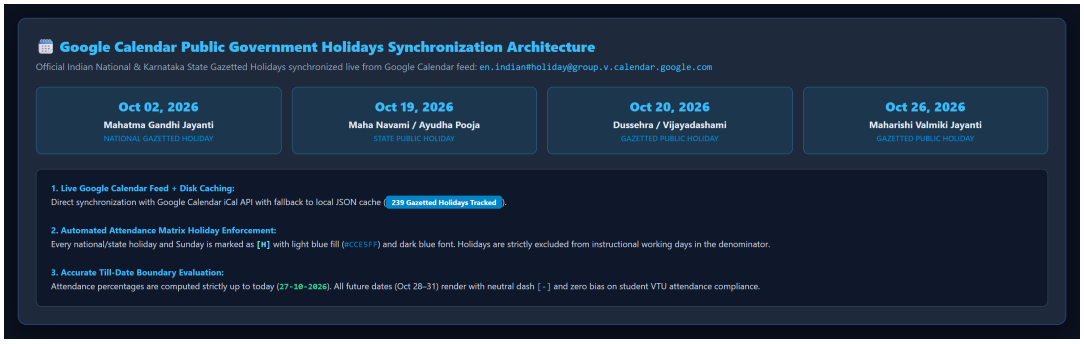


Figure 8.36 Google Calendar Holiday Synchronisation

8.1.7 Utilities

Keyboard-driven palette to open the scanner, campus radar, theme studio and other pages quickly.

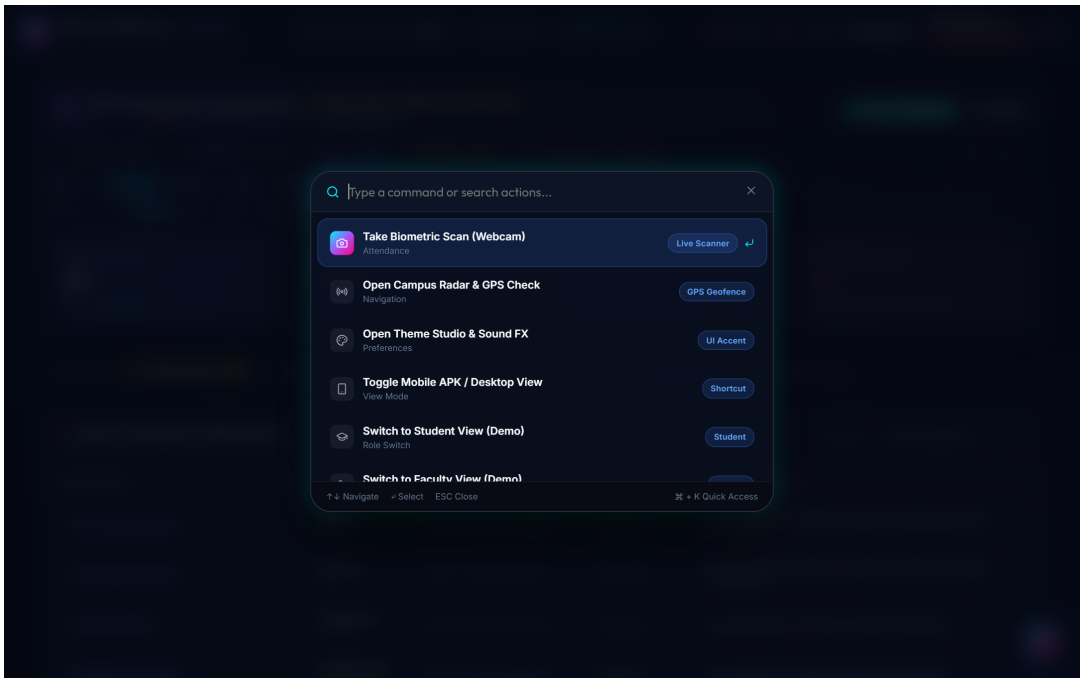


Figure 8.37 Command Palette (Ctrl+K)

Colour accents, audio feedback and ambient blur settings for the interface.

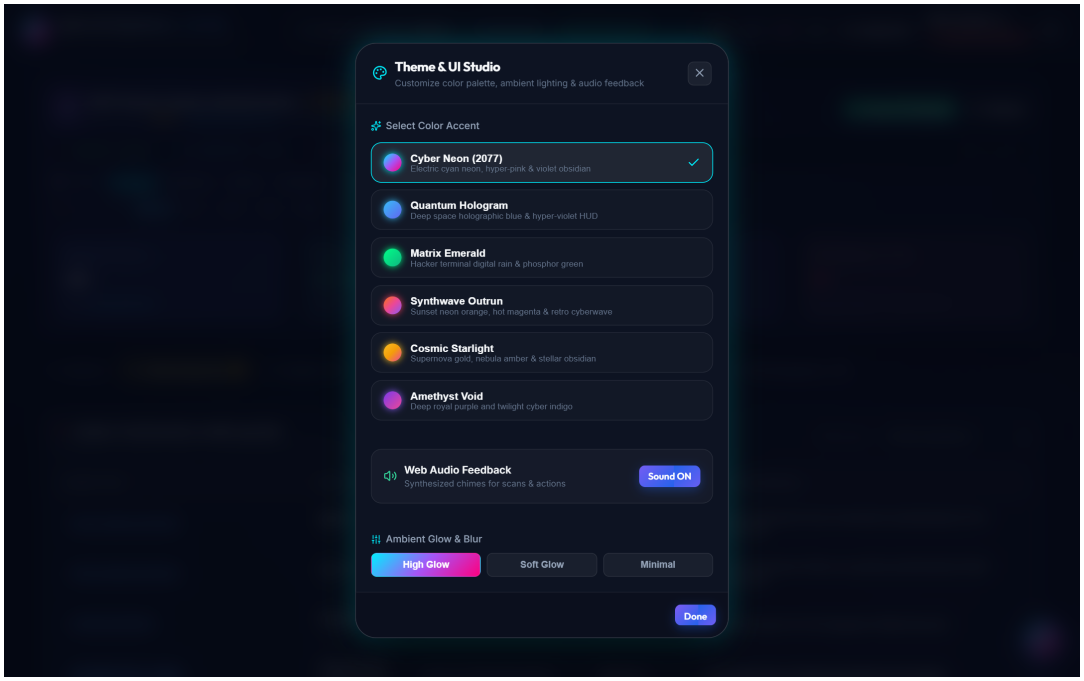


Figure 8.38 Theme and UI Studio

Circulars and notices with priority badges for the campus community.

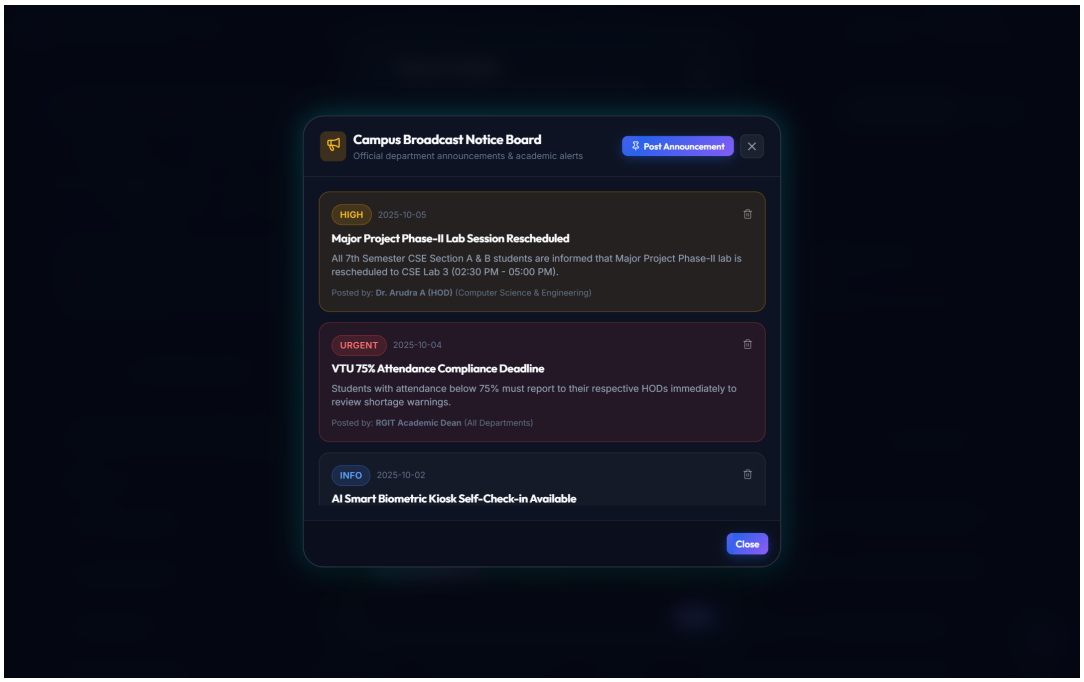


Figure 8.39 Campus Notice Board

8.2 Sample Output Summary

The screens demonstrate that the student can register with a face photograph, check in with the six-digit PIN or by scanning the faculty QR code, with location and face verification, and review proof-backed attendance; the teacher can launch sessions with a live QR code and PIN, monitor and correct attendance, identify defaulters and export Excel and CSV reports;

the HOD can approve registrations and manage faculty, subjects and timetable; and the master administrator can govern all departments, calibrate the geofence, synchronise holidays and review the audit ledger.

Role-wise Summary of the Screens

- **Student:** login portal (Figure 8.1), biometric admission enrolment form and registration status tracker (Figures 8.5 and 8.6), main dashboard and the dashboard with PIN and QR check-in options (Figures 8.7 and 8.8), QR scanner modal (Figure 8.9), biometric scanner with session PIN, PIN entry with biometric capture and PIN auto-detected from the QR link (Figures 8.10 to 8.12), phone-camera capture and the contact edit window (Figures 8.13 and 8.14).
- **Teacher:** login portal (Figure 8.2), dashboard and assigned subjects (Figures 8.15 and 8.16), launching of an attendance session with a dynamic QR code and PIN, the live classroom display and the projector full-screen view (Figures 8.17 to 8.20), and the Excel sheet, CSV preview, day-to-day register and faculty export panel (Figures 8.21 to 8.24).
- **HOD:** login portal (Figure 8.3), department overview dashboard, student directory and mobile view of the portal (Figures 8.25 to 8.27), credentials and profile editing, registration rejection reasons and the window for assigning faculty to subjects (Figures 8.28 to 8.30), and the institutional export controls (Figure 8.31).
- **Master administrator:** login portal (Figure 8.4), dashboard, HOD governance directory, campus geofence perimeter studio and security audit ledger (Figures 8.32 to 8.35), Google Calendar holiday synchronisation (Figure 8.36), and the command palette (Ctrl+K), theme and UI studio and campus notice board (Figures 8.37 to 8.39).

Observations on the Interface

The screens show that every role sees only the functions that belong to it, and that important events are recorded with their details in the audit ledger. A student can mark attendance in two ways, by typing the session PIN or by scanning the faculty QR code, and in both cases the geofence and the face capture are checked before the attendance is accepted. The teacher can show the QR code and PIN on the projector and download the attendance of a subject as an Excel sheet or a CSV sheet. The portals are responsive: the HOD portal is shown in its mobile view, and the scanner can use the phone camera directly. Interface features such as the command palette, theme and UI studio, notice board and the holiday synchronisation card make the system convenient for daily use by the college community.

Relation to the Conclusion

The sample output confirms that the functions described in the earlier chapters have been implemented as working screens. The conclusion that follows summarises what the system achieves, its limitations and the possible future work.

CONCLUSION

CONCLUSION

The AI Smart Attendance System Using Face Recognition replaces manual roll calls with an automated and verifiable process. A student’s attendance is accepted only when the registered phone is used, the student is inside the RGIT campus perimeter, a teacher has launched the class session, the attendance has not already been marked, and the face matches the enrolled photograph. Each accepted record stores the subject, period, time, coordinates, distance, match confidence and a photographic proof, and every failed attempt is written to the audit log, so that every decision can be traced and every correction needs a recorded reason. The system saves lecture time because the roll call is no longer needed, it removes the chance of proxy attendance through the combined use of face verification, geofence, session PIN and device binding, and it gives teachers and Heads of Department an immediate view of attendance percentages and of the students who fall below the 75% requirement of the university. The role-based portals give students, teachers, HODs and the master administrator the information and controls they need, and the department-wise databases suit the structure of an engineering college with nine branches while the master administrator keeps oversight of all of them. The automated test cases confirm that the guards for login, location, session, device and PIN behave as intended, and the sample screens show that every function described in this report is available as a working portal. The system uses only free software and ordinary hardware, which keeps it affordable for the institution, and the work has also been published as a research paper in the IRE Journals.

Future Scope

In the future, the system can be strengthened and extended in several directions. Server-side active liveness detection using the eye aspect ratio over consecutive frames, together with texture analysis against screen replay, can make the system more resistant to photographs and videos. Passive classroom attendance from CCTV streams can be added with a face-detection model such as YOLO-Face, and a stronger embedding model of the ArcFace type can be adopted if larger populations or harder conditions require it. Automatic SMS or WhatsApp alerts can inform parents when attendance falls below 75%. The system can be deployed on the cloud with managed databases, encryption of stored face images and clear retention policies, and it can be integrated with the university examination and eligibility systems. Attention and engagement analytics from facial landmarks can further help teachers understand the participation of students in the classroom.

REFERENCES

REFERENCES

[1] M. Mohammed, M. A. Khan, M. Y. B., M. N. Raza, B. Wakde and S. Komal V, "AI Smart Attendance System Using Face Recognition," IRE Journals, vol. 10, issue 4, October 2026, Paper ID 1723595, ISSN 2456-8880.

[2] B. Wakde, M. Y. B., M. N. Raza, M. A. Khan, and M. Muzammil, "AI Smart Attendance System Using Face Recognition," International Journal of Scientific Research in Engineering and Management (IJSREM), vol. 10, no. 4, pp. 1–7, April 2026, ISSN 2582-3930.

[3] J. W. Yap and M. S. Jawad, "Real Time Face Recognition Attendance System Using Python and OpenCV," Applied Information Technology and Computer Science, vol. 5, no. 1, pp. 1310–1329, 2024.

[4] J. Deng, J. Guo, N. Xue and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4690–4699, 2019.

[5] A. Rao, "AttenFace: A Real Time Attendance System Using Face Recognition," in Proceedings of the IEEE 6th Conference on Information and Communication Technology (CICT), 2022.

[6] D. Sunaryono, J. Siswantoro and R. Anggoro, "An Android Based Course Attendance System Using Face Recognition," Journal of King Saud University – Computer and Information Sciences, vol. 33, no. 3, pp. 304–312, 2021.

[7] K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778, 2016.

[8] T. Soukupova and J. Cech, "Real-Time Eye Blink Detection Using Facial Landmarks," in Proceedings of the 21st Computer Vision Winter Workshop (CVWW), 2016.

[9] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), vol. 1, pp. 886–893, 2005.

[10] K. Zhang, Z. Zhang, Z. Li and Y. Qiao, "Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks," IEEE Signal Processing Letters, vol. 23, no. 10, pp. 1499–1503, 2016.

[11] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779–788, 2016.

[12] F. Schroff, D. Kalenichenko and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in Proceedings of the IEEE Conference on Computer

Vision and Pattern Recognition (CVPR), pp. 815–823, 2015.

[13] O. M. Parkhi, A. Vedaldi and A. Zisserman, "Deep Face Recognition," in Proceedings of the British Machine Vision Conference (BMVC), 2015.

[14] D. E. King, "Max-Margin Object Detection," arXiv preprint arXiv:1502.00046, 2015.

[15] Z. Boulkenafet, J. Komulainen and A. Hadid, "Face Anti-Spoofing Based on Color Texture Analysis," in Proceedings of the IEEE International Conference on Image Processing (ICIP), pp. 2636–2640, 2015.

[16] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, 2015.

[17] K. Kazemi and J. Sullivan, "One Millisecond Face Alignment with an Ensemble of Regression Trees," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1867–1874, 2014.

[18] Y. Taigman, M. Yang, M. Ranzato and L. Wolf, "DeepFace: Closing the Gap to Human-Level Performance in Face Verification," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1701–1708, 2014.

[19] G. B. Mohapatra, J. Dsouza, Y. Sangoiy and N. Shah, "Smart Attendance System Using Real Time Face Recognition," International Journal of Technology, vol. 8, no. 1, pp. 1–5, 2018.

[20] D. E. King, "Dlib-ml: A Machine Learning Toolkit," Journal of Machine Learning Research, vol. 10, pp. 1755–1758, 2009.

[21] G. B. Huang, M. Ramesh, T. Berg and E. Learned-Miller, "Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments," University of Massachusetts, Amherst, Technical Report 07-49, 2007.

[22] T. Ahonen, A. Hadid and M. Pietikainen, "Face Description with Local Binary Patterns: Application to Face Recognition," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 28, no. 12, pp. 2037–2041, 2006.

[23] A. K. Jain, A. Ross and S. Prabhakar, "An Introduction to Biometric Recognition," IEEE Transactions on Circuits and Systems for Video Technology, vol. 14, no. 1, pp. 4–20, 2004.

[24] P. Viola and M. Jones, "Rapid Object Detection Using a Boosted Cascade of Simple Features," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), vol. 1, pp. 511–518, 2001.

[25] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.

[26] M. Turk and A. Pentland, "Eigenfaces for Recognition," Journal of Cognitive Neuroscience, vol. 3, no. 1, pp. 71–86, 1991.

PUBLISHED RESEARCH PAPER

© OCT 2026 | IRE Journals | Volume 10 Issue 4 | ISSN: 2456-8880
DOI: https://doi.org/10.64388/IREV10I4-1723595

AI Smart Attendence System Using Face Recognition

MOHAMMED MUZAMMIL[1], MUHAMMAD ADAM KHAN[2], MAHAMMAD YUNUS B[3], MIR NAQI RAZA[4], BHAGYASHRI WAKDE[5], SONIYA KOMAL V[6]
[1, 2, 3, 4] *Dept. of Computer Science Engineering, Rajiv Gandhi Institute of Technology, Bengaluru*
[5]*Professor, Dept. of Computer Science Engineering, Rajiv Gandhi Institute of Technology, Bengaluru*
[6]*Professor, Dept. of Computer Science Engineering, Rajiv Gandhi Institute of Technology, Bengaluru*

Abstract- ***Attendance is an essential academic activity, but conventional roll calls and manual records consume classroom time and may introduce recording errors and opportunities for proxy attendance. This paper presents an AI Smart Attendance System using face recognition that automates attendance through real-time camera processing and timetable-based validation. The implemented prototype is developed using Python and Streamlit with OpenCV, the face_recognition library, WebRTC camera streaming, SQLite, Pandas and Matplotlib. The system provides administrator and student authentication, student registration, section management, timetable checking, face-based attendance, multi-frame confirmation, duplicate-attendance prevention, security logging, attendance filtering, analytics and CSV reporting. The face verification workflow is configured to require multiple confirming frames before an attendance record is created. The proposed enhanced architecture extends the prototype with mobile access, GPS/geofencing, separate subject-teacher accounts and an HOD dashboard. Under this architecture, attendance is accepted only when the student's identity, scheduled subject and permitted location satisfy the configured rules. Teachers can monitor attendance for their assigned subjects, while the HOD can obtain department-level visibility across teachers, subjects, sections and students. The paper describes the system architecture, methodology, database design, security controls, implementation status and planned enhancements. Rather than claiming unmeasured accuracy or deployment results, the study identifies the measurable evaluation parameters required for future experimental validation.***

Keywords - Artificial Intelligence, Face Recognition, Smart Attendance, Timetable Validation, WebRTC, SQLite, GPS Geofencing, Role-Based Access Control.

I. INTRODUCTION

Attendance management is a routine but important part of academic administration. In a conventional classroom, the teacher may record attendance by calling student names, passing a register, or using a manual signature process. These methods require repeated human effort and can make it difficult to maintain consistent, searchable records. They can also permit proxy attendance when one student responds or signs on behalf of another. Automated biometric attendance provides an opportunity to reduce these limitations by linking an attendance event to an identifiable student.

Face recognition is particularly suitable for this purpose because it is contactless and can operate through an ordinary camera. A practical attendance system, however, must do more than recognize a face. It must determine whether the recognized student is attending the correct class, whether an attendance record already exists, and whether the event should be accepted by the institution's rules. The supplied implementation addresses these operational requirements through student and administrator login, timetable validation, camera-based recognition, duplicate checking, security logging and attendance analytics. The implementation uses Python, Streamlit, OpenCV, face_recognition, WebRTC, NumPy, Pandas, SQLite and Matplotlib.

The enhanced system proposed in this paper introduces an additional institutional structure. Students use restricted accounts and submit only their own attendance. Subject teachers receive separate accounts with access limited to their assigned subjects and sections. An HOD account provides department-level monitoring. GPS or geofencing is proposed as a second identity-context check so that attendance can be accepted only from an approved college or classroom area. These enhanced modules are presented as proposed extensions rather than as features already implemented in the supplied prototype.

© OCT 2026 | IRE Journals | Volume 10 Issue 4 | ISSN: 2456-8880
DOI: https://doi.org/10.64388/IREV10I4-1723595

II. PROBLEM STATEMENT AND OBJECTIVES

The system addresses the following problems:

- Time consumed by manual attendance recording.
- Possibility of proxy or impersonated attendance.
- Difficulty in connecting an attendance record to the correct subject and period.
- Limited visibility of attendance records for different institutional roles.
- Repeated manual preparation of attendance summaries and reports.
- The principal objectives are:
- Automate attendance using real-time face recognition.
- Verify the active timetable period before attendance is accepted.
- Restrict students to their own attendance workflow.
- Prevent duplicate attendance records.
- Maintain searchable records, analytics and security logs.
- Extend the system with GPS verification and role-based teacher/HOD dashboards.

III. LITERATURE SURVEY

Automated attendance has been studied using RFID, fingerprint, iris, NFC and face recognition. RFID and NFC systems can simplify identification but depend on physical cards or tags, which may be transferred between students. Fingerprint systems provide biometric identification but require physical contact with a sensor. Iris systems can provide strong biometric discrimination but generally require specialized acquisition hardware. Face recognition offers a contactless alternative and can be performed through cameras already available in many classrooms.

Classical computer-vision approaches include the Viola–Jones object detection framework for rapid face detection and Local Binary Pattern Histogram (LBPH) methods for face description and recognition. [2], [3] More recent approaches use learned facial embeddings and deep neural networks. FaceNet, for example, maps faces into an embedding space in which distance can be used for identity comparison. [4] The choice of recognition method depends on accuracy, computational resources, data quality, illumination, pose and deployment requirements.

The proposed project follows a practical implementation path rather than claiming that one algorithm is universally optimal. The supplied code uses OpenCV and the face_recognition library and performs recognition from a live WebRTC camera stream. The attendance decision is additionally constrained by timetable information and duplicate-record checks. This combination shifts the focus from face recognition alone to an attendance transaction containing identity, subject, period, date, time and evidence.

IV. PROPOSED SYSTEM

The proposed system contains five logical layers: user/client layer, identity and context verification layer, attendance engine, role-based dashboards, and database and reporting layer. In the current prototype, the client and attendance engine are implemented through Streamlit. The camera is accessed through WebRTC, facial processing is performed using OpenCV and face_recognition, and persistent data are maintained using SQLite. The enhanced architecture adds mobile access, GPS verification and separate teacher/HOD authorization.

A. Complete Attendance Ecosystem

The final design is organized around one attendance rule shared by every client: the student may mark attendance only for the student's own account and only for the subject and period permitted by the academic timetable. The website and mobile application are two access channels to the same attendance service. This keeps the system consistent even when a student changes devices.

B. Student Website and Mobile Access

Students can log in through the college website using a browser-enabled camera. The proposed mobile application provides the same attendance workflow on a phone. After login, the student sees the subjects associated with the student's section, selects the currently scheduled subject and starts attendance. The interface does not allow the student to select an unrelated subject or edit an existing attendance record.

C. Subject-Level Attendance

Every attendance transaction is subject-specific. The backend links Student → Section → Subject → Period → Date → Time. When teacher assignment is enabled, Teacher is also linked to the transaction. This structure allows a student to attend one subject

© OCT 2026 | IRE Journals | Volume 10 Issue 4 | ISSN: 2456-8880
DOI: https://doi.org/10.64388/IREV10I4-1723595

without accidentally creating attendance in another subject and allows reports to be generated independently for every subject.

D. Institutional Roles

The enhanced platform contains Student, Subject Teacher, HOD and Administrator roles. Students submit their own attendance. Subject teachers see only the subjects and sections assigned to them. The HOD can monitor all teachers, subjects, sections and students in the department. The Administrator manages registration, timetable, system configuration, security information and controlled attendance administration.

E. Location and Session Security

GPS/geofencing is an additional validation layer for the mobile and location-aware web workflow. A configurable college or classroom zone can be checked before face verification is completed. A short-lived QR token can optionally be used to launch the active attendance session, but the QR code does not replace face verification. This keeps the QR mechanism as a session-control feature rather than a transferable attendance credential.

V. METHODOLOGY

A. Student Registration

The registration phase creates the identity record required for later recognition. Student information such as name, roll number, class and section is stored in the database, while facial information is associated with the student. The administrator interface in the current implementation provides student registration and editing functions. The registration data form the reference set against which the live camera input is evaluated.

B. Attendance Processing

When a student enters the attendance workflow, the application checks the current section and timetable. If a valid period is available, the camera interface is activated. Frames from the live stream are processed for face detection and recognition. The current configuration requires five confirmation frames within an eight-second confirmation window before a recognition event is treated as sufficiently consistent for attendance processing. A duplicate check prevents the same student from being recorded repeatedly for the same attendance context.

C. Timetable Validation

Timetable validation connects attendance to an academic period rather than treating attendance as an unrestricted transaction. The current implementation contains teaching periods and break periods and maps subjects to sections and weekdays. This mechanism is important because a valid face match alone should not automatically create attendance for an arbitrary subject or time.

D. GPS / Geofencing Extension

GPS verification is a future enhancement and is not required in the current website-based workflow. The application obtains the device location and compares it with an approved campus or classroom geofence. If location permission is denied, location data are unavailable, or the device is outside the configured area, the attendance request is rejected. If the location is valid, the request proceeds to face verification and timetable validation. The supplied prototype does not currently implement this GPS layer; it is an enhancement for the next version.

E. Role-Based Access

The current prototype provides administrator and student login types. The enhanced architecture adds two institutional roles. A subject teacher can view attendance only for assigned subjects and sections and can generate subject-specific reports. The HOD can monitor teachers, subjects, sections and student attendance across the department. This separation reduces unnecessary access to attendance records and establishes clearer responsibility for attendance management.

F. Subject-Wise Website Attendance

Attendance is subject-specific in the enhanced design. A student can log in through the institutional website or through the proposed mobile application. After authentication, the system displays the subjects available for the student's section and the active timetable context. The student selects the current subject and starts the attendance workflow.

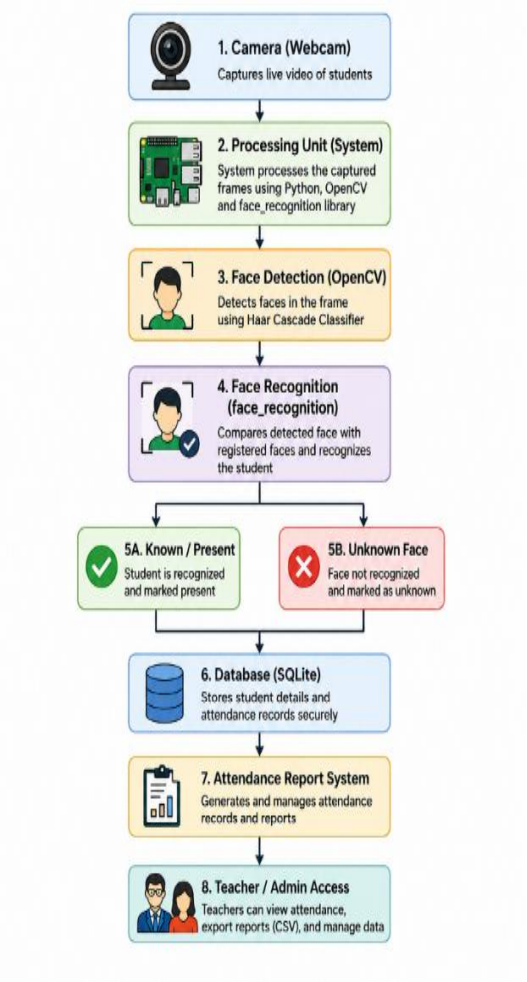
© OCT 2026 | IRE Journals | Volume 10 Issue 4 | ISSN: 2456-8880
DOI: https://doi.org/10.64388/IREV10I4-1723595

The same rules are applied to both clients: student identity is verified by face recognition, the active period and subject are checked against the timetable, and duplicate attendance is rejected. The mobile phone camera can be used directly through the website; GPS/geofencing remains a future enhancement. The resulting record is associated with Student + Section + Subject + Period + Date + Time and, where enabled, Face Verification + Location Verification.

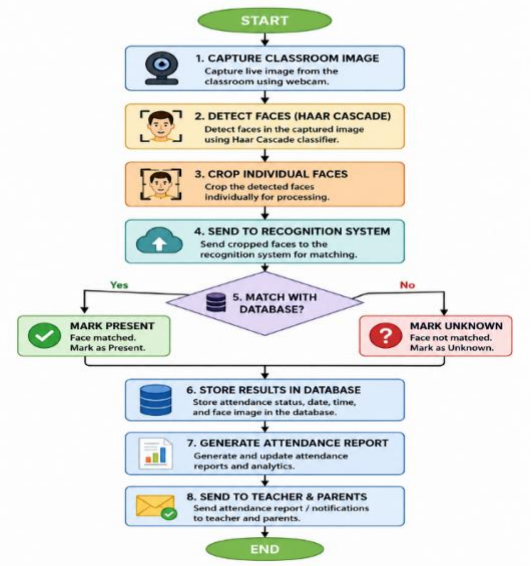
The subject teacher receives a restricted dashboard for the assigned subject and section. The HOD receives department-level visibility. Students cannot edit stored attendance or submit attendance on behalf of another student. The website remains available as an alternative when a native mobile application is not installed.

After The proposed system presents an intelligent and automated attendance management solution using face recognition technology integrated with cloud computing and embedded systems. Based on an extensive literature survey, existing systems were analyzed to identify limitations such as low accuracy, high manual effort, and lack of real-time monitoring. To overcome these issues, an efficient and scalable approach is designed using a combination of image processing, machine learning, and cloud-based services. The system is implemented using a Raspberry Pi 3B+ integrated with a high-resolution webcam, which captures real-time classroom images. The face detection process is performed using the ViolaJones algorithm (Haar Cascade Classifier), which efficiently detects human faces from the captured image. Once faces are detected, the recognition process is carried out using Local Binary Pattern Histogram (LBPH) along with Convolutional Neural Network (CNN)-based cloud recognition through AWS Rekognition for higher accuracy. The methodology is divided into two main phases: Student Registration and Attendance Processing.

A. System Architecture Diagram



B. System Flowchart



© OCT 2026 | IRE Journals | Volume 10 Issue 4 | ISSN: 2456-8880
DOI: https://doi.org/10.64388/IREV10I4-1723595

C. Student Registration Process

The registration process is essential for building the facial database of students. Initially, student details such as name, ID, and class are collected. The system captures multiple images of each student using the camera module. These images are then uploaded to Amazon S3 along with metadata. An AWS Lambda function is automatically triggered upon image upload, which processes the image, extracts facial features, and generates a unique face print. This face print is a mathematical representation of the student's face and is stored in Amazon DynamoDB along with the corresponding student information. This ensures quick and efficient retrieval during attendance marking.

D. Attendance Processing

The attendance process begins with capturing a real-time image of the classroom. The system detects all faces present in the image and crops them individually for further processing. These cropped images are sent to AWS Rekognition, which analyzes facial features using deep learning models and compares them with stored face prints in the database. If a match is found, the student is marked as present; otherwise, the face is labeled as unknown. The system automatically generates a list of present and absent students. Teachers are given the option to manually verify and correct attendance records if necessary. The final attendance data is stored securely in the cloud database.

E. Hardware Requirements

The system requires minimal and cost-effective hardware components, including:

- Raspberry Pi 3B+sentence.
- Logitech C270 Webcam (8MP)
- Power Supply
- 16GB Micro SD Card
- Monitor (Screen)
- Keyboard and Mouse
- HDMI Cable

F. Cloud Integration (AWS Services)

The system leverages multiple AWS services for efficient operation:

- Amazon S3 for image storage
- AWS Rekognition for facial recognition
- AWS Lambda for automated processing
- DynamoDB for storing face prints and student data

This cloud-based architecture ensures scalability, high accuracy, and real-time processing capabilities.

G. Parent Access and Monitoring Feature (NEW ADDITION)

A key enhancement of the proposed system is the integration of a Parent Access Module, which improves transparency and student monitoring. In this feature, parents are provided with secure access (via mobile app or web portal) to view their child's attendance records in real time. The system automatically sends notifications to parents regarding their child's attendance status, including daily presence, absence alerts, and overall attendance percentage. This allows parents to track academic participation and take necessary actions in case of irregular attendance. Additionally, periodic reports can be generated and shared with parents via email or mobile notifications, enhancing communication between educational institutions and families.

VI. SOFTWARE IMPLEMENTATION

The prototype is implemented as a Python Streamlit application. OpenCV is used for image-processing operations, while the face_recognition library provides face encoding and comparison functionality. WebRTC provides live browser-based camera streaming from the laptop or mobile device. NumPy supports numerical operations, Pandas supports data handling and reporting, Matplotlib supports analytics visualization, and SQLite provides persistent storage.

a) Python – application development and control logic.

b) Streamlit – user interface and dashboard layer.

c) OpenCV – camera/image processing.

d) face_recognition – facial encoding and recognition.

e) WebRTC – live browser camera stream.

f) SQLite – local relational database.

g) Pandas – attendance filtering and report generation.

h) Matplotlib – attendance analytics and visualization.

VII. DATABASE DESIGN AND SECURITY

The current database contains tables for users, students, attendance, security logs and administrator face information. Attendance records contain identity

© OCT 2026 | IRE Journals | Volume 10 Issue 4 | ISSN: 2456-8880
DOI: https://doi.org/10.64388/IREV10I4-1723595

and academic-context fields such as student name, roll number, class, section, subject, period, date, time and image path. The security-log structure records timestamp, event, roll, section, IP and details. These records provide an audit trail for important system events.

For the enhanced system, the data model should be extended with teacher accounts, HOD accounts, subject-teacher assignments, geofence configuration and location-verification fields. A normalized attendance record can therefore be represented conceptually as: Student + Section + Subject + Teacher + Period + Date + Time + Face Verification + Location Verification. Sensitive biometric and location information should be protected through appropriate access controls, retention rules and secure transmission/storage mechanisms during real deployment.

VIII. ROLE-BASED WORKFLOW

Student: logs in, checks the active subject/period, permits location access, performs face verification and receives attendance confirmation.

Subject Teacher: logs in to the assigned subject account, views present/absent students, filters records by date/period/section and generates reports.

HOD: views department-level attendance across teachers, subjects, sections and students and reviews analytics and audit information.

Administrator: registers students, manages timetable and records, reviews attendance, analytics and security logs.

IX. SECURITY AND ANTI-PROXY MEASURES

The system combines several controls rather than relying on face recognition alone. Face verification links an event to the registered student identity. Timetable checking links the event to an active academic period. Multiple-frame confirmation reduces acceptance based on a single unstable frame, while duplicate checking prevents repeated records. Security logs capture important events such as failed login attempts. In the enhanced design, GPS/geofencing adds a location constraint. Together, these controls are intended to make unauthorized attendance more difficult, although no biometric system can guarantee complete elimination of spoofing without dedicated liveness and anti-spoofing mechanisms.

X. RESULT AND DISCUSSION

The developed system provides a user-friendly graphical interface using Streamlit, allowing users to efficiently manage attendance through face recognition. The system mainly consists of three modules: student registration, attendance marking, and attendance viewing. In the student registration module, users are required to enter the student's name and roll number and upload a face image. Once the register button is clicked, the image is stored in the dataset folder and the details are saved in the SQLite database. The registered students are displayed in a gallery format with their images resized to a uniform size, ensuring consistency for face recognition. Each student is also provided with a remove option, which allows deletion of both the database record and the corresponding image from the dataset. Additionally, a retrieve student feature is implemented where users can search for a student using their roll number and view their details along with the stored image. This functionality is illustrated in Fig.1, which shows the dataset of registered students and the interface for managing them

Fig.1. Registered Students Dataset and Gallery View

During the attendance process, the user selects the take attendance option and activates the webcam by clicking the start camera checkbox. The system captures live video frames and detects faces in real-time using the face_recognition library. The detected faces are encoded and compared with the stored dataset images. If a match is found, the system

© OCT 2026 | IRE Journals | Volume 10 Issue 4 | ISSN: 2456-8880
DOI: https://doi.org/10.64388/IREV10I4-1723595

identifies the student and displays their name on the video frame. If the face is not registered, it remains unrecognized. Once a student is identified, their attendance is automatically recorded along with the current date and time. The system ensures that attendance is marked only once per day for each student, preventing duplicate entries. The real-time face detection and recognition process is shown in Fig.2, where recognized students are highlighted on the screen.

Fig.2. Live Face Recognition for Attendance Marking

The attendance records are stored in the database and can be accessed through the view attendance module. The records are displayed in a structured tabular format containing details such as student name, roll number, date, and time. This allows easy tracking and verification of attendance data. Furthermore, the system provides an option to download the attendance records in CSV format for external use such as reporting or analysis. The attendance table displayed in the interface is shown in Fig.3, which represents the successfully recorded attendance data after the recognition process.

Fig.3. Attendance Records Displayed in Tabular Format

Overall, the system demonstrates an effective implementation of face recognition technology for automated attendance management. It eliminates the need for manual attendance marking and reduces the chances of proxy attendance. The system is efficient, easy to use, and capable of handling student data effectively. However, its performance depends on factors such as lighting conditions, camera quality, and the availability of clear facial images. Despite these limitations, the system provides a reliable and accurate solution for modern attendance management.

XI. ADVANTAGES

Contactless attendance using the built-in laptop or mobile-phone camera.

Reduced dependence on manual roll calls.

Timetable-based subject and period control.

Multiple-frame confirmation and duplicate prevention.

Searchable attendance records and analytics.

Security logging for important events.

Usable on laptops and mobile phones without requiring a CCTV camera.

XII. LIMITATIONS

Recognition performance may vary with lighting, pose, camera quality and image conditions.

The supplied implementation is a Streamlit/web prototype accessed through a laptop or mobile

browser; a native Android application is not required for the current workflow.

GPS/geofencing is an optional future enhancement and is not implemented in the supplied code.

Separate subject-teacher and HOD roles are proposed extensions; the current login model supports administrator and student roles.

Dedicated liveness/anti-spoofing and large-scale deployment evaluation require further development and testing.

XIII. FUTURE ENHANCEMENT

Future work will focus on converting the student attendance workflow into a mobile application, adding GPS/geofencing, implementing teacher-subject authorization, and introducing an HOD dashboard. The database can be extended for teacher assignments and location audit information. Liveness detection can be incorporated to strengthen protection against presentation attacks. Additional work may investigate cloud deployment, secure remote storage, institutional ERP integration, parent/guardian access, automated notifications and longitudinal attendance analytics. These extensions should be evaluated experimentally before being presented as measured performance improvements.

XIV. CONCLUSION

This paper presents an AI Smart Attendance System using face recognition to automate classroom attendance through a website accessible on a laptop or mobile phone. The implemented prototype uses Python, Streamlit, OpenCV, face_recognition, WebRTC, and SQLite to support student registration, timetable-controlled subject attendance, live camera-based face verification, multi-frame confirmation, duplicate prevention, security logging, attendance records, analytics, and reporting. A student authenticates through the website, the system identifies the student's class and section, verifies the current day, period, and scheduled subject, and activates the built-in laptop or mobile-phone camera. The captured face is compared with the registered face encoding, and multiple frames are confirmed before attendance is accepted. The system then checks for duplicate attendance and records the student identity, section, subject, period, date, time, and supporting evidence. The student receives an attendance confirmation or rejection message, while authorized administrators can review attendance and security events. As future enhancements, the system can include GPS/geofencing, subject-teacher access, and HOD-level monitoring. The proposed extensions provide a path toward a more complete institutional attendance platform, while future experimental evaluation can measure recognition performance, processing time, usability, and security under real classroom conditions.

REFERENCES

[1] S. Hapani, M. Shah, and V. Kanani, "Automated Attendance System Using Image Processing," in *2018 4th International Conference on Computing Communication Control and Automation (ICCUBEA)*, IEEE, 2018.

[2] M. S. Akbar, P. Sarker, A. T. Mansoor, A. M. Al Ashray, and J. Uddin, "Face Recognition and RFID Verified Attendance System," in *2018 International Conference on Computing, Electronics & Communications Engineering (iCCECE)*, IEEE, 2018, pp. 168–172.

[3] K. O. Okokpujie, E. Noma-Osaghae, O. J. Okesola, S. N. John, and O. Robert, "Design and Implementation of a Student Attendance System Using Iris Biometric Recognition," in *2017 International Conference on Computational Science and Computational Intelligence (CSCI)*, IEEE, 2017, pp. 563–567. IEEE

[4] H. Rathod et al., "Automated Attendance System Using Machine Learning Approach," in *2017 International Conference on Nascent Technologies in Engineering (ICNTE)*, IEEE, 2017. IEEE

[5] A. R. S. Siswanto, A. S. Nugroho, and M. Galinium, "Implementation of Face Recognition Algorithm for BiometricsBased Time Attendance System," in *2014 International Conference on ICT for Smart Society (ICISS)*, IEEE, 2014. IEEE

[6] S. Lukas, A. R. Mitra, R. I. Desanti, and D. Krisnadi, "Student Attendance System in Classroom Using Face Recognition Technique," in *2016 International Conference on Information and Communication Technology Convergence (ICTC)*, IEEE, 2016, pp. 1032–1035. IEEE

[7] O. A. R. Salim, R. F. Olanrewaju, and W. A. Balogun, "Class Attendance Management System Using Face Recognition," in *2018 7th International Conference on Computer and Communication Engineering (ICCCE)*, IEEE, 2018. IEEE

[8] K. Solanki and P. Pittalia, "Review of Face Recognition Techniques," *International Journal of Computer Applications*, vol. 133, no. 12, pp. 20–24, 2016. International Journal of Computer Applications

[9] C. Stoll et al., "Face Recognition is Shaped by the Use of Sign Language," *Journal of Deaf Studies & Deaf Education*, vol. 23, no. 1, pp. 1–9, 2018.

[10] W. Deng, J. Hu, and J. Guo, "Face Recognition via Collaborative Representation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 10, 2017.

[11] T. Pei et al., "Decision Pyramid Classifier for Face Recognition," *Pattern Recognition*, vol. 64, pp. 305–313, 2016.

[12] H. Shi et al., "Cross-Modality Face Recognition via Heterogeneous Joint Bayesian," *IEEE Signal Processing Letters*, vol. 24, no. 1, pp. 81–85, 2017. IEEE

[13] C. Ding and D. Tao, "Trunk-Branch Ensemble CNN for VideoBased Face Recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2016.

[14] V. Nemirovskiy et al., "Face Recognition Based on Proximity Measure Clustering," *Cybernetics Institute Journal*, vol. 40, no. 5, pp. 740–745, 2016.

[15] T. K. Taniya, N. M. Nidhi, and T. Nandini, "Automated HR and Attendance Management Using Face Recognition," *IJSRSET*, vol. 16, no. 4, pp. 847–853, 2016.

[16] D. Wu et al., "Robust Face Recognition Using Deep Learning," *Journal of Optoelectronics Laser*, vol. 27, no. 6, pp. 655–661, 2016.

[17] Y. Sun, J. Zhao, and Y. Hu, "Supervised Sparsity Preserving Projections for Face Recognition," *Proceedings of SPIE*, vol. 8009, 2017.

[18] L. Best-Rowden and A. K. Jain, "Longitudinal Study of Automatic Face Recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 1, pp. 148–162, 2018. IEEE

[19] T. Valentine, M. B. Lewis, and P. J. Hills, "Face-Space: A Unifying Concept in Face Recognition Research," *Quarterly Journal of Experimental Psychology*, vol. 69, no. 10, pp. 1996–2019, 2016. Taylor & Francis

[20] Y. Duan et al., "Context-Aware Local Binary Feature Learning for Face Recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2017.

[21] S. Chakraborty, S. K. Singh, and P. Chakraborty, "Local Gradient Hexa Pattern for Face Recognition," *IEEE Transactions on Circuits and Systems for Video Technology*, 2016.

CERTIFICATES OF PUBLICATION

CERTIFICATES OF PUBLICATION

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

MOHAMMED MUZAMMIL

has published the research paper entitled

AI SMART ATTENDENCE SYSTEM USING FACE RECOGNITION

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 4	October 2026

PAPER ID
1723595
www.irejournals.com

ISSN
2456-8880

Sanket Patel
EDITOR-IN-CHIEF
IRE Journals

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

MUHAMMAD ADAM KHAN

has published the research paper entitled

AI SMART ATTENDENCE SYSTEM USING FACE RECOGNITION

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 4	October 2026

PAPER ID
1723595
www.irejournals.com

ISSN
2456-8880

Sanket Patel
EDITOR-IN-CHIEF
IRE Journals

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

MAHAMMAD YUNUS B

has published the research paper entitled

AI SMART ATTENDENCE SYSTEM USING FACE RECOGNITION

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 4	October 2026

PAPER ID
1723595
www.irejournals.com

ISSN
2456-8880

Sanket Patel
EDITOR-IN-CHIEF
IRE Journals

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

MIR NAQI RAZA

has published the research paper entitled

AI SMART ATTENDENCE SYSTEM USING FACE RECOGNITION

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 4	October 2026

PAPER ID
1723595
www.irejournals.com

ISSN
2456-8880

Sanket Patel
EDITOR-IN-CHIEF
IRE Journals

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

BHAGYASHRI WAKDE

has published the research paper entitled

AI SMART ATTENDENCE SYSTEM USING FACE RECOGNITION

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 4	October 2026

PAPER ID

1723595

www.irejournals.com

ISSN

2456-8880

Sanket Patel

EDITOR-IN-CHIEF

IRE Journals

IRE JOURNALS

ICONIC RESEARCH AND ENGINEERING JOURNALS

AN INTERNATIONAL OPEN ACCESS · PEER-REVIEWED · REFEREED JOURNAL

Certificate of Publication

This is to certify that

SONIYA KOMAL V

has published the research paper entitled

AI SMART ATTENDENCE SYSTEM USING FACE RECOGNITION

PUBLISHED IN	VOLUME & ISSUE	MONTH OF ISSUE
e-Journal	Volume 10 Issue 4	October 2026

PAPER ID

1723595

www.irejournals.com

ISSN

2456-8880

Sanket Patel

EDITOR-IN-CHIEF

IRE Journals